# Workbook

## DATABASE MANAGEMENT SYSTEMS

## (SE –204)


| | |
|---|---|
| **Name** | |
| **Roll No.** | |
| **Batch** | |
| **Year** | |
| **Semester** | |

**Department of Software Engineering**

**NED University of Engineering & Technology**

# DATABASE MANAGEMENT SYSTEMS

# (SE –204)

Prepared by

Engr. Shiza Riaz Memon

Lecturer

Department of Software Engineering

Approved by

Prof. Dr. Shehnila Zardari

Chairperson

Department of Software Engineering

**Department of Software Engineering**

**NED University of Engineering & Technology**

## TABLE OF CONTENTS

**LAB No 1.**

**Objective: Installing Oracle SQL Plus**

**Theory:**

**What is SQL?**

SQL stands for Structured Query Language. It is a domain-specific language used in programming and managing relational databases. SQL provides a standardized way to communicate with databases, allowing users to perform various operations like creating, modifying, and querying data

**Installing ORACLE SQL PLUS:**

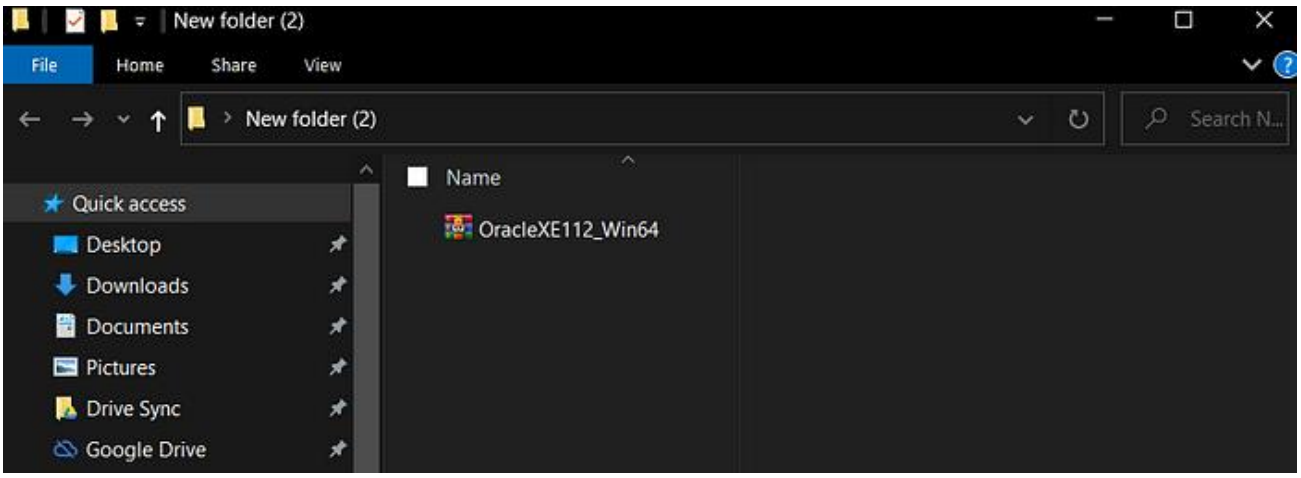
**About Oracle 11g Express Edition**

Oracle Database Express Edition, also known as Oracle Database XE is free and a small footprint edition of Oracle Database. Furthermore, Oracle Database XE is easy in terms of installation and management.

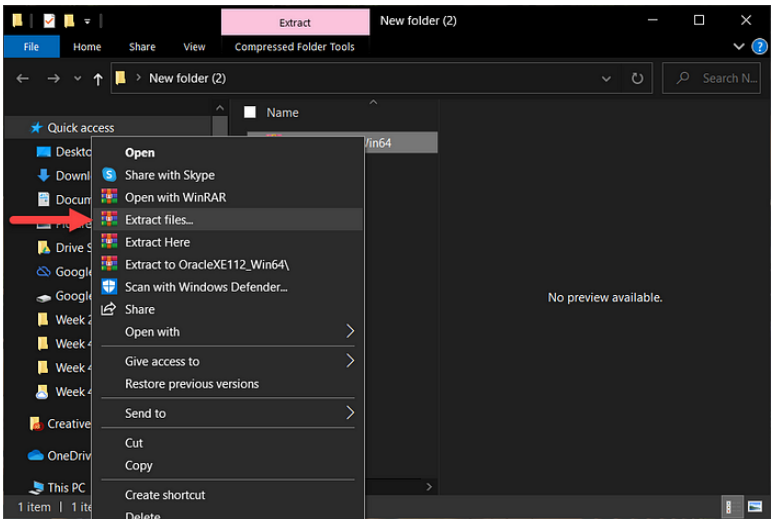
**Stepwise procedure for installing Oracle 11g Express Edition**

**Step 1: Get the licensed software from computer laboratory and download the setup on your system.**

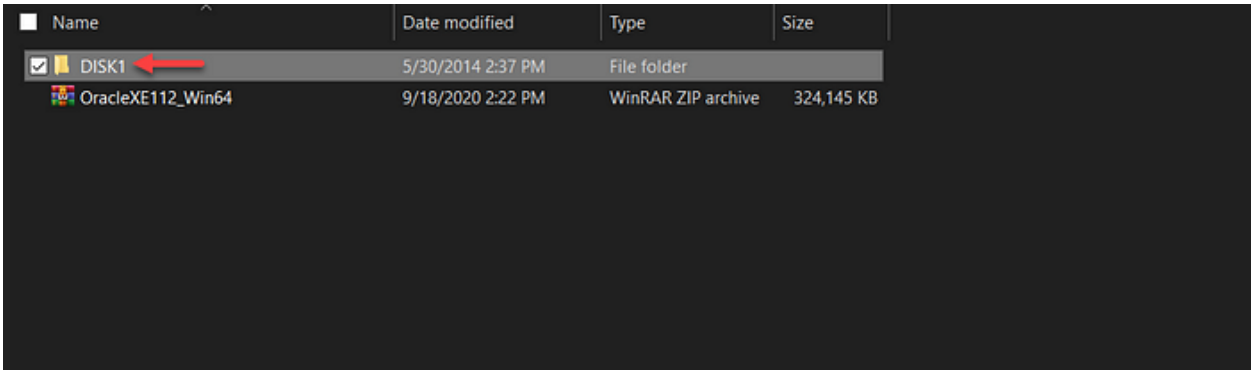
**Step 2: Now, after the file has been successfully downloaded, you can view the file in the file explorer.**



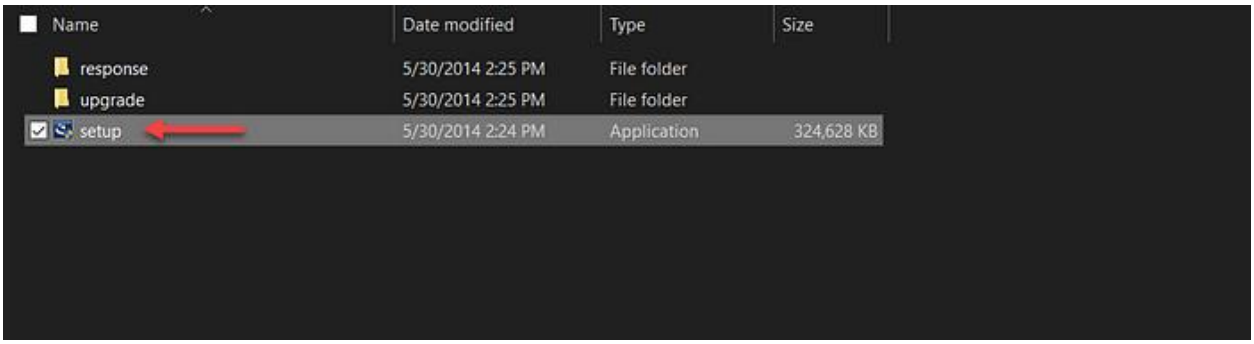
**Step 3: You are now required to extract the file present in the zip file. So, in order to unzip the file, right-click the zipped file and click on 'Extract files…' options as shown in the image below.**



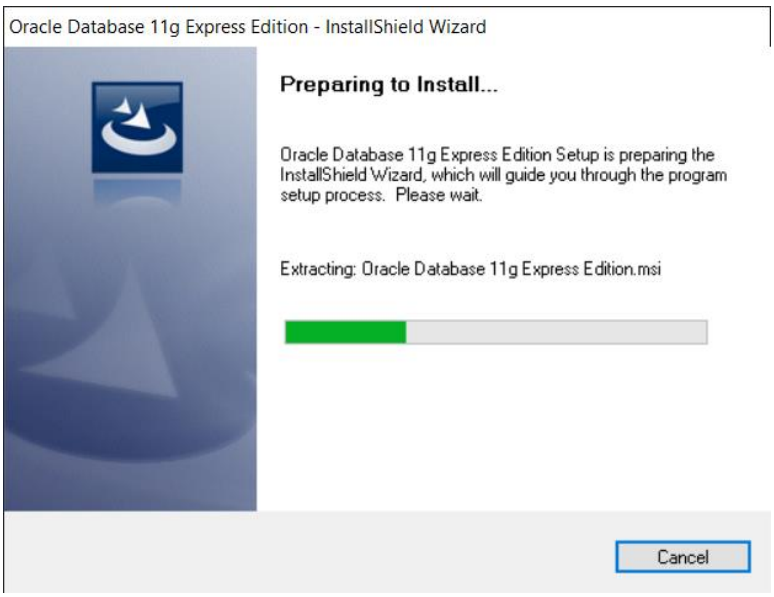

**Step 4: After you have extracted your file, you can view the contents present in the zip file as shown in the image below.**



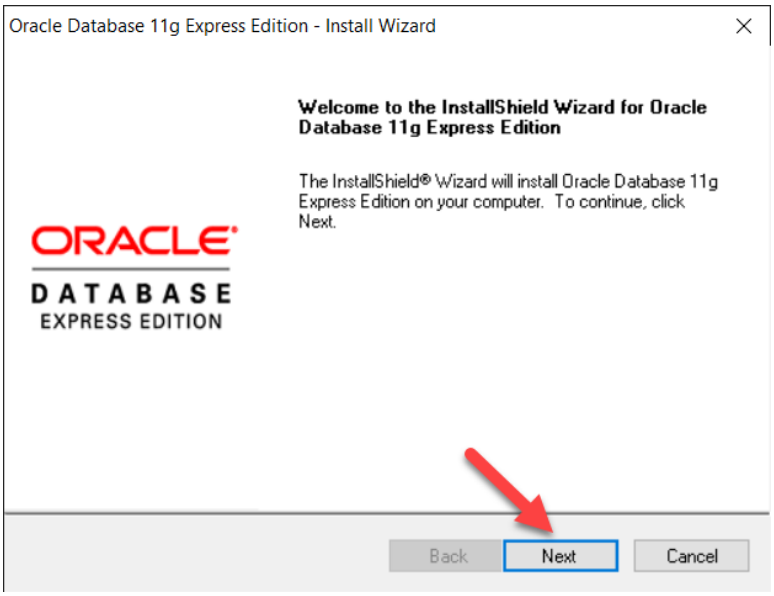
**Step 5: Now, go inside the folder name 'DISK1'. Here, execute the 'setup' file as shown in the image below.**



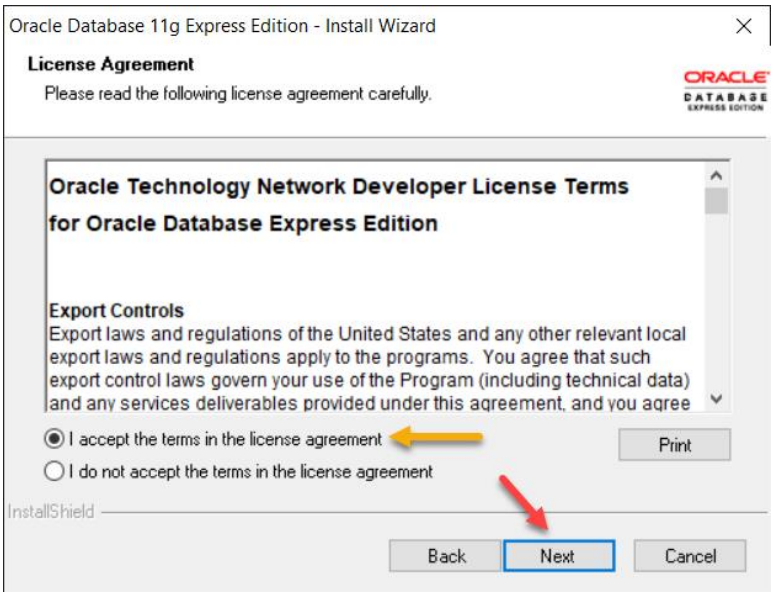

**Step 6: After you execute the setup file. A window appears as shown in the image below.**



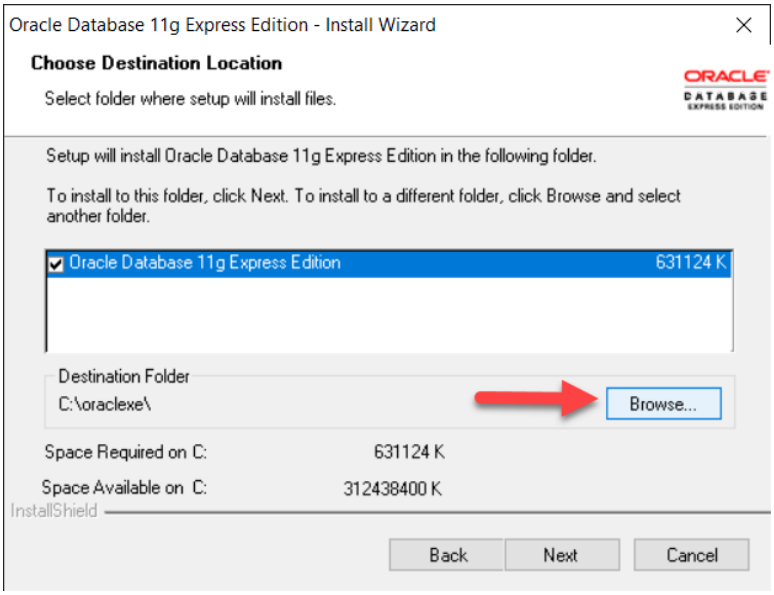
**Step 7: Now, 'Oracle Database 11g Express Edition — Install Wizard' appears as shown in the image below. Here, click on the 'Next' button.**



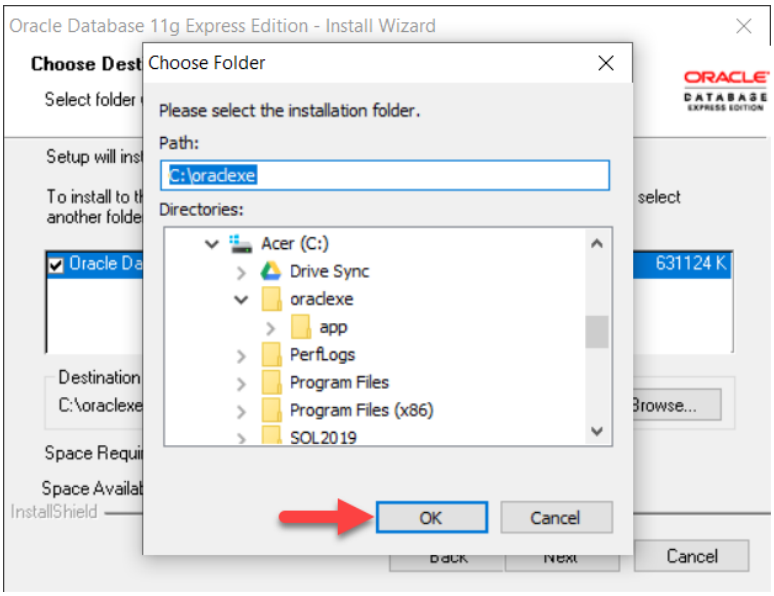

**Step 8: Here, in the License Agreement, select 'I accept the terms in the license agreement'. Then, finally, click on the 'Next' button.**



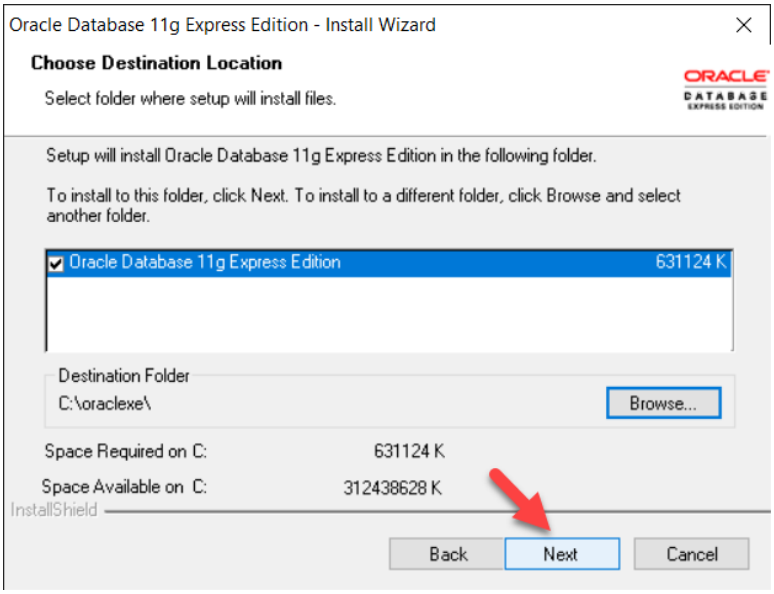
**Step 9: Here, you can click on the 'Browse…' button in order to change the location where you are willing to install the Oracle.**



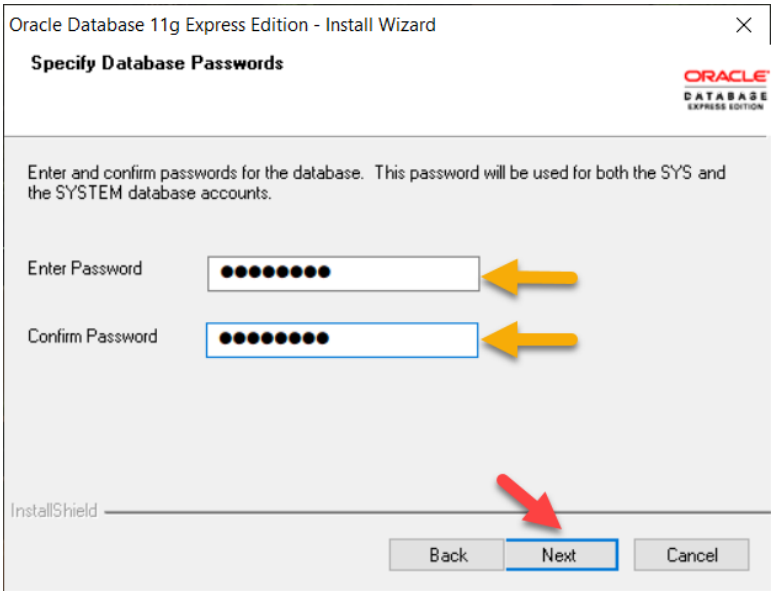

**Step 10: Here, you can select the location where you are willing to install the files. After you have selected a suitable location, you can click on the 'OK' button.**



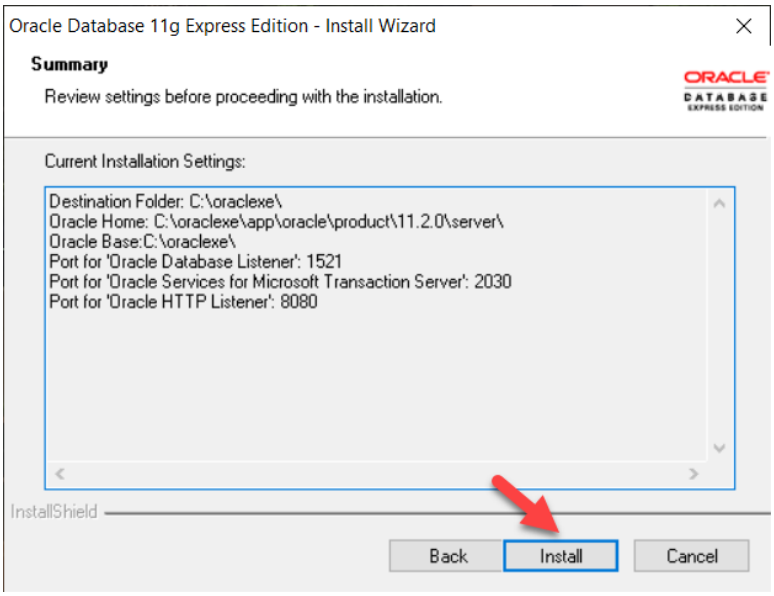
**Step 11: After selecting the file location, you can click on the 'Next' button.**



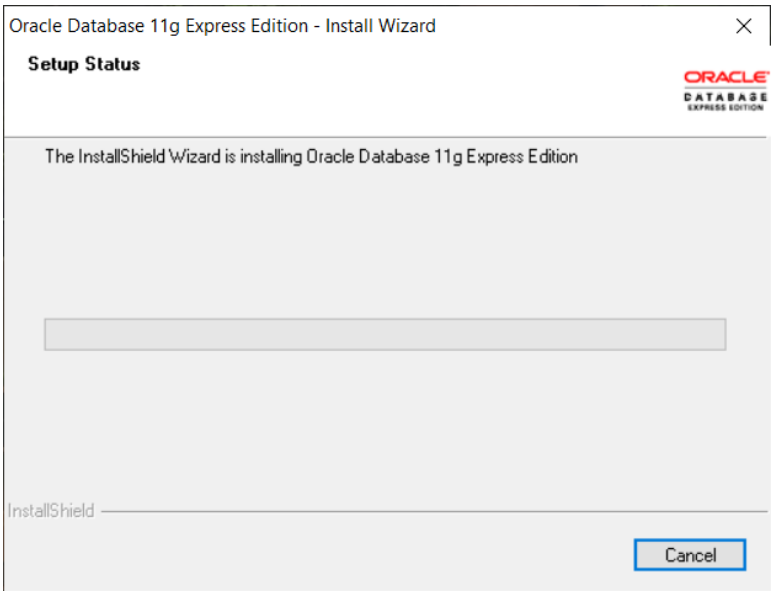

**Step 12: Now, you are required to enter the password for the Oracle database. However, in this stage, you are required to provide a password mandatorily. For instance: 'admin123'.**



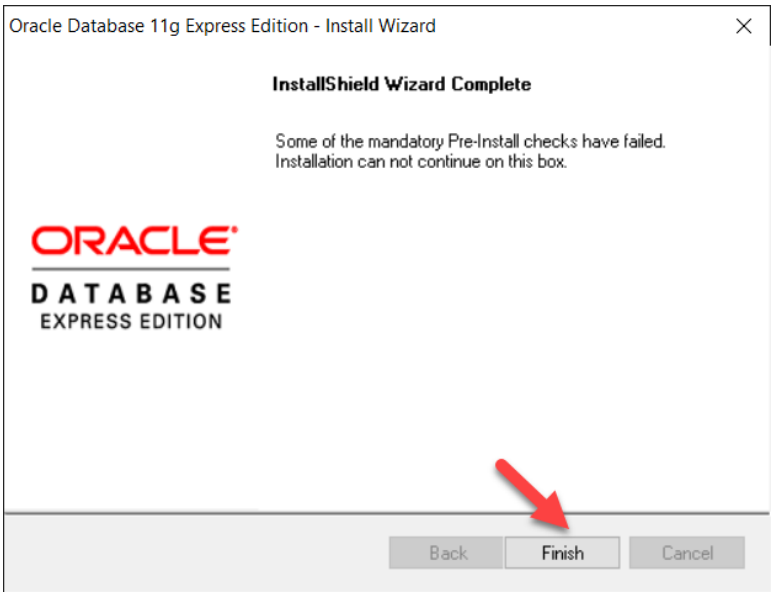
**Step 13: Here, in 'Summary', you are required to click on the 'Install' button.**



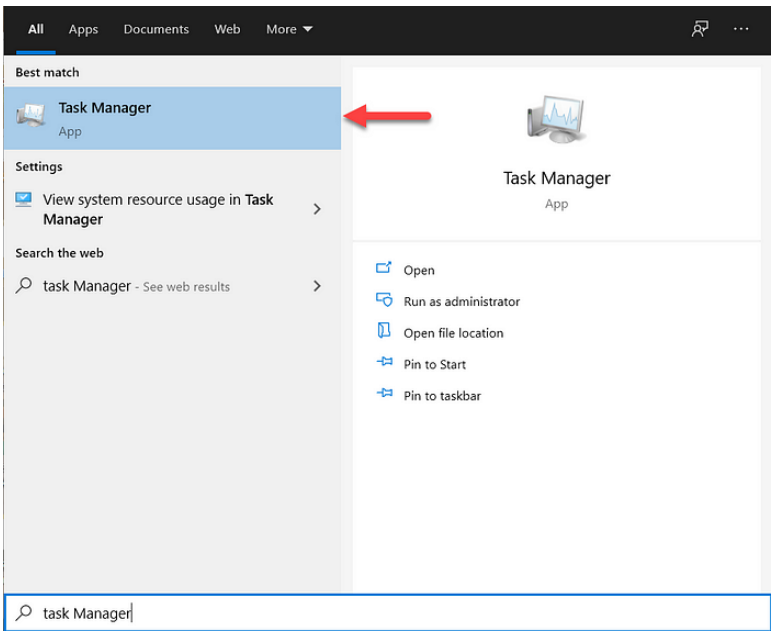

**Step 14: Now, your installation of Oracle Database 11g Express Edition begins as shown in the image below.**



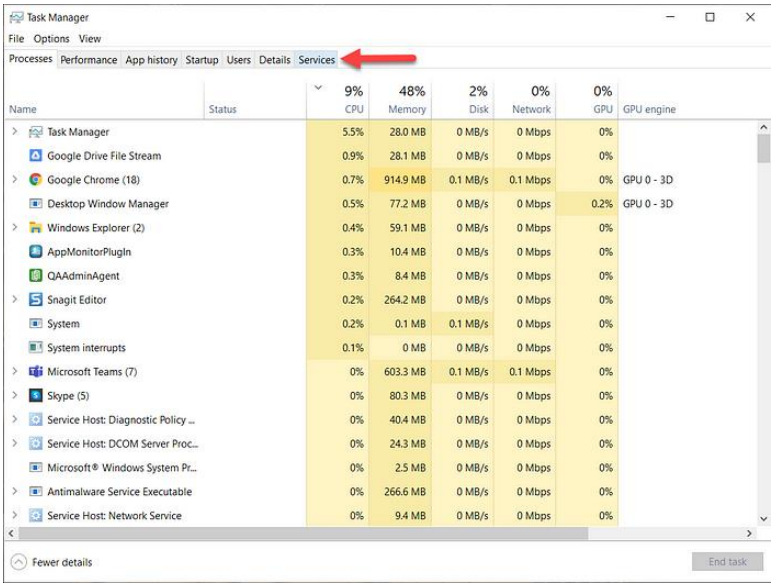
**Step 15: Finally, the 'Install Shield Wizard' is complete. Hence, you can now click on the 'Finish' button.**



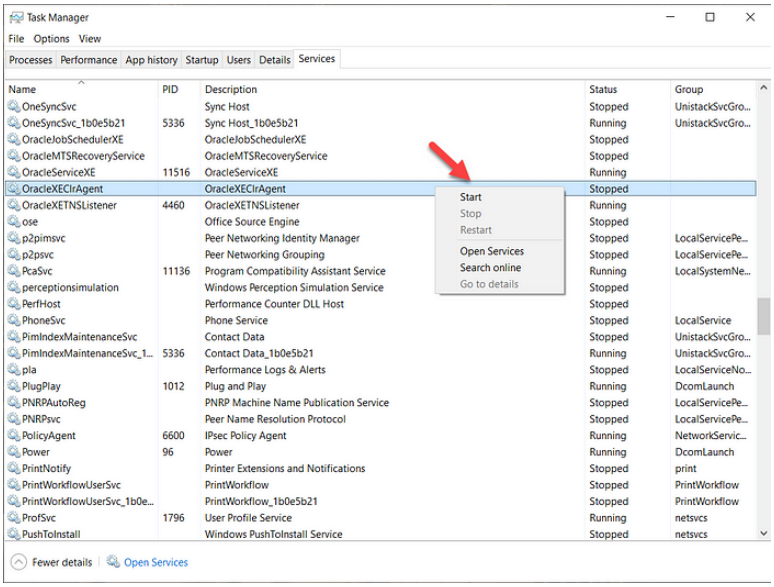

**Step 16: Now, search for the 'Task Manager' in your PC.**



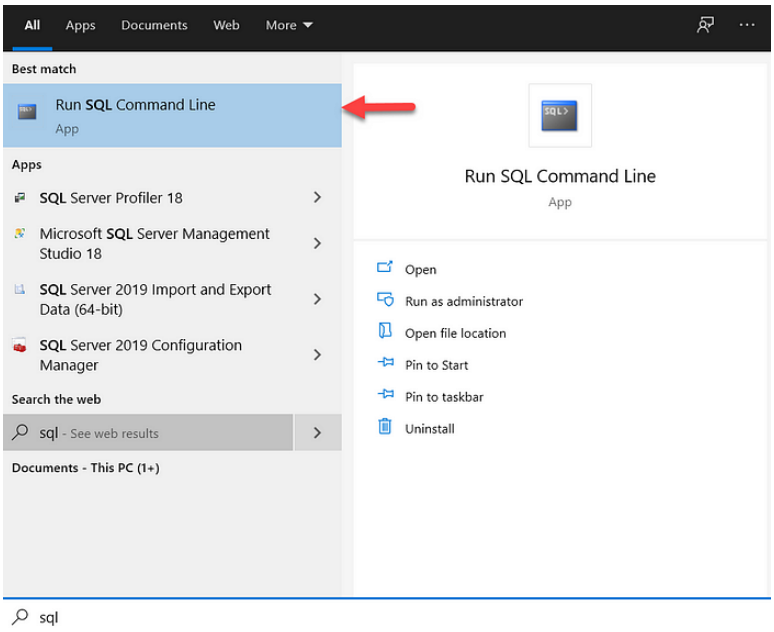
**Step 17: In the 'Task Manager', you can go to the 'Services' as shown in the image below.**



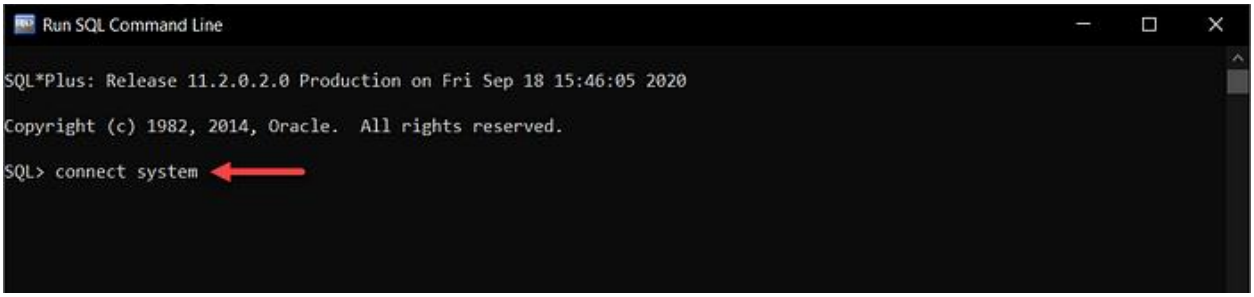

**Step 18: In 'Services', start all the services of Oracle. Hence, for starting a service, you are required to right-click the service and select 'Start' as shown in the image below. For instance: 'OracleXEClrAgent'.**



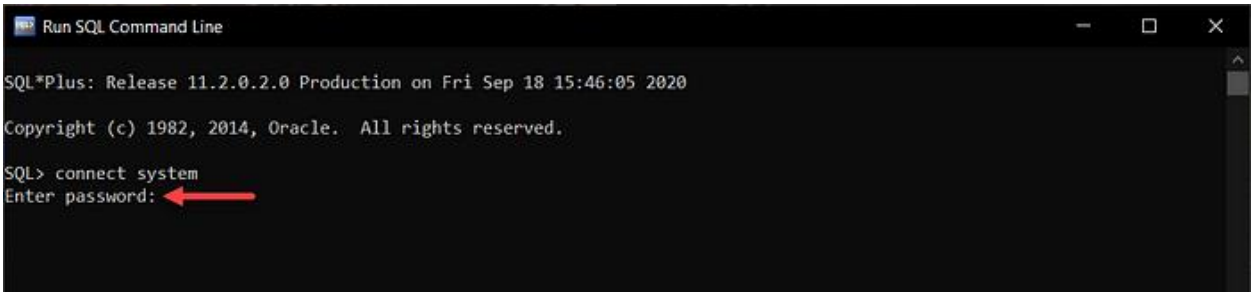
**Step 19: In your PC, you are required to search for 'SQL Command Line' as shown in the image below.**



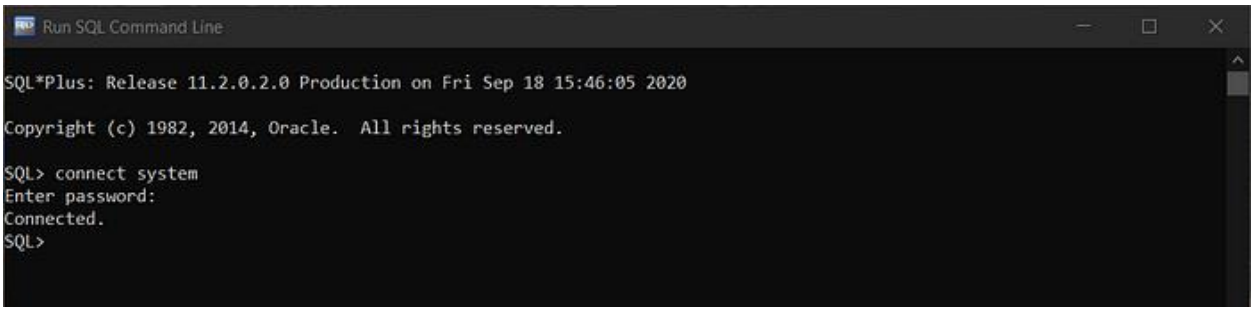

**Step 20: For ensuring that the installation has been carried out successfully, let us try to log in to the system. Hence, for connecting to the system, you can execute the query provided below.**




**Step 21: Here, you are now required to enter your password. For instance: 'admin123'**



**Step 22: You are now connected to the system.**



## LAB No 2.

**Objective: Exploring SQL Plus Commands**

**Theory:**

**Syntax of Commands**

### A) CREATE TABLE:

To make a new database, table, index, or stored query. A create statement in SQL creates an object inside of a relational database management system (RDBMS).

**CREATE TABLE <table_name>**

(Column_name1 data_type ([size]),

Column_name2 data_type ([size]),

Column_name-n data_type ([size]) );

### B) ALTER A TABLE:

To modify an existing database object. Alter the structure of the database.

To add a column in a table

**ALTER TABLE table_name ADD column_name datatype;**

To delete a column in a table

**ALTER TABLE table_name DROP column column_name;**

### C) DROP TABLE:

Delete Objects from the Database

**DROP TABLE table_name;**

### D) TRUNCATE TABLE:

Remove all records from a table, including all spaces allocated for the records are removed.

**SQL> truncate table table_name;**

### E) DESCRIBE TABLE:

**SQL> desc table_name;**

### F) VIEW TABLE:

In order to view all tables already created by the user, use the command

**SQL> select table_name from user_tables;**

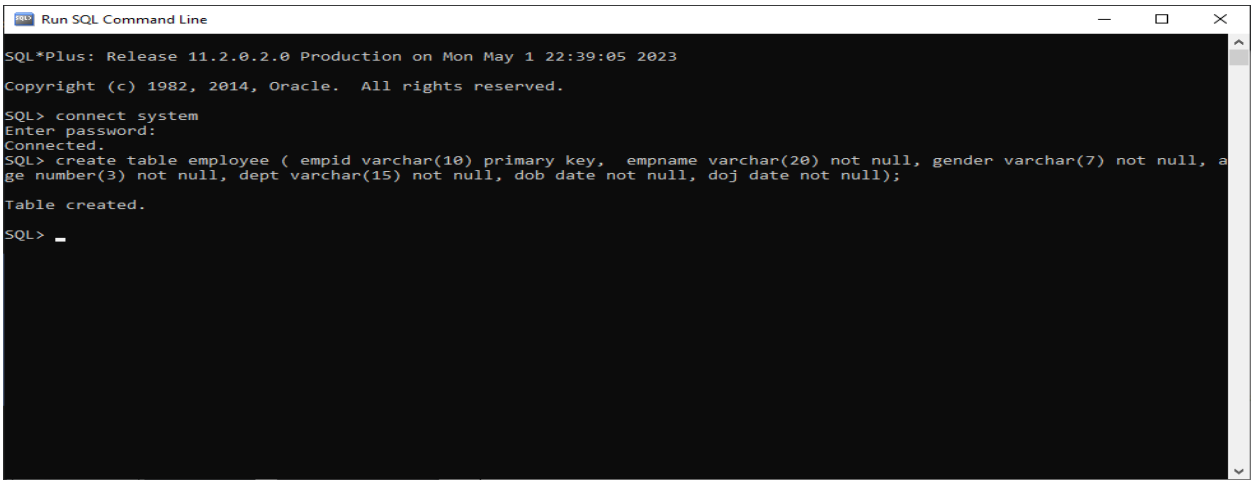
If the table is not viewable , then we should search the tables within all schemas

**SQL> select table_name from all_tables;**

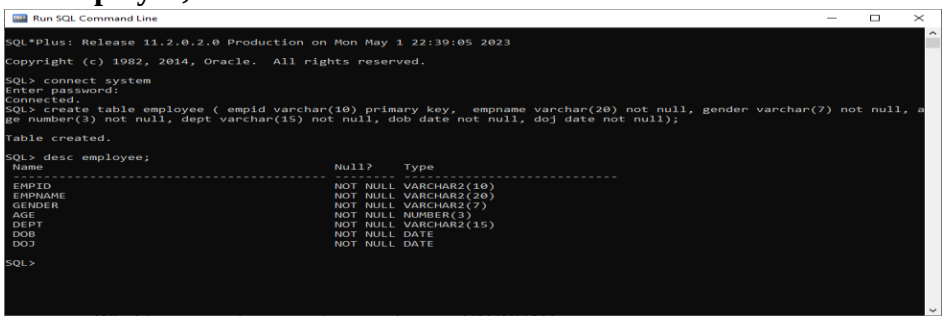
```
TABLE_NAME
------------------------------
LOGSTDBY$HISTORY
LOGSTDBY$EDS_TABLES
DEF$_AQCALL
DEF$_AQERROR
SQLPLUS_PRODUCT_PROFILE
HELP
EMPLOYEE
LOGMNR_GT_TAB_INCLUDE$
LOGMNR_GT_USER_INCLUDE$
LOGMNR_GT_XID_INCLUDE$
LOGMNRT_MDDL$

TABLE_NAME
------------------------------
```

**Exercise:**

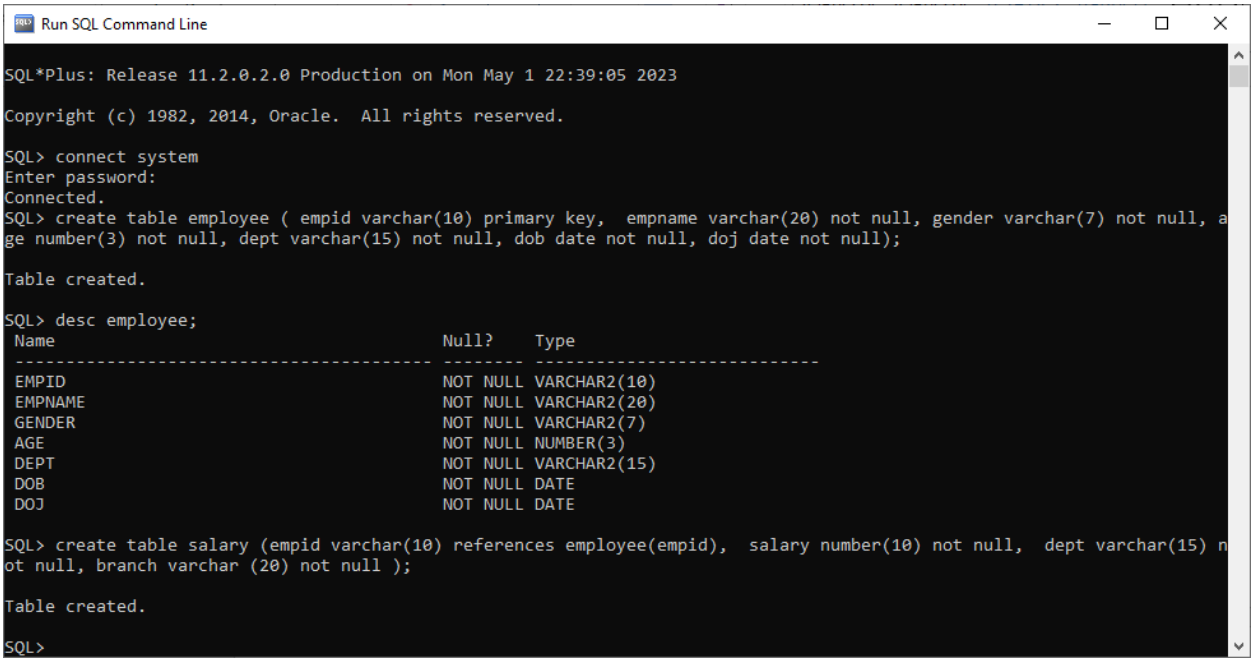
1. **Create a table**
   **SQL>create table employee ( empid varchar(10) primary key, empname varchar(20) not null, gender varchar(7) not null, age number(3) not null, dept varchar(15) not null, dob date not null, doj date not null);**



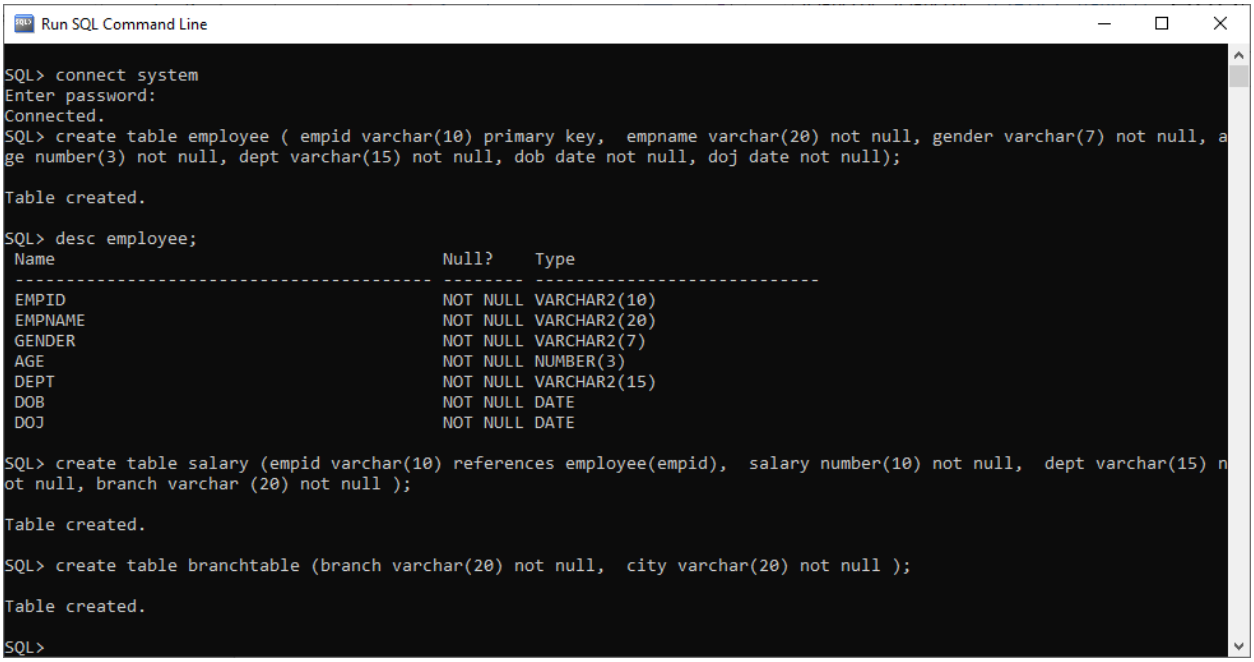

2. **Describe table:**
   **SQL> desc employee;**



3. **Create more tables:**
    - **SQL> create table salary (empid varchar(10) references employee(empid), salary number(10) not null, dept varchar(15) not null, branch varchar (20) not null );**

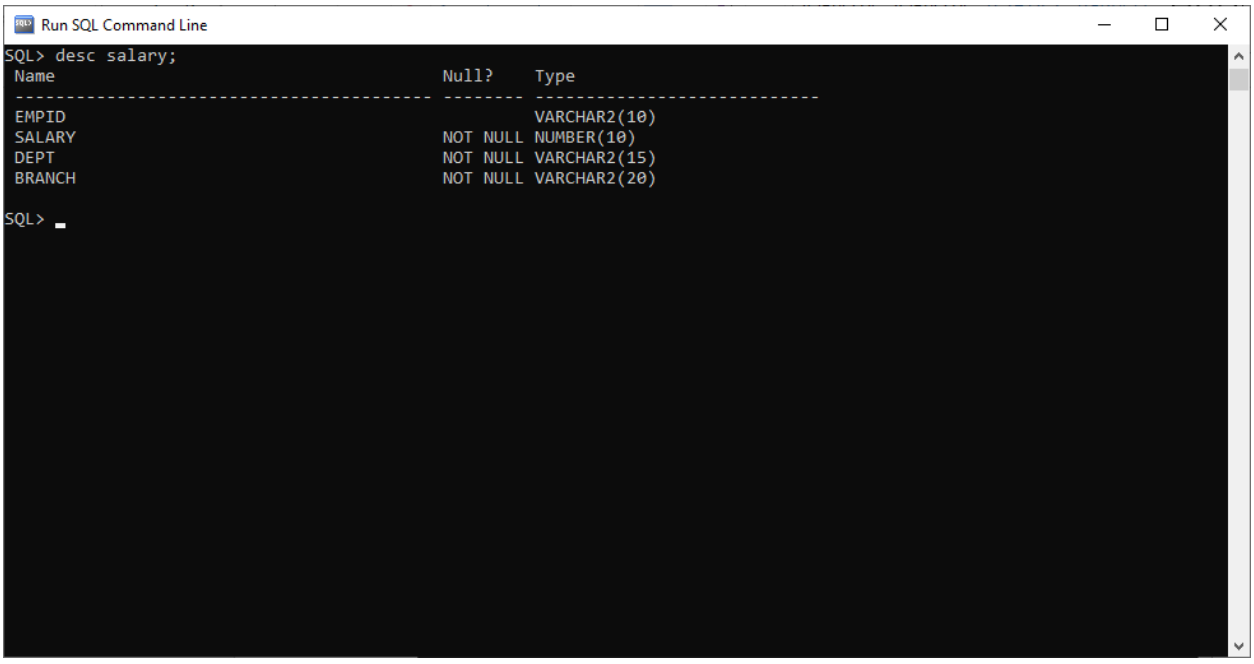



    - **SQL> create table branchtable (branch varchar(20) not null, city varchar(20) not null );**

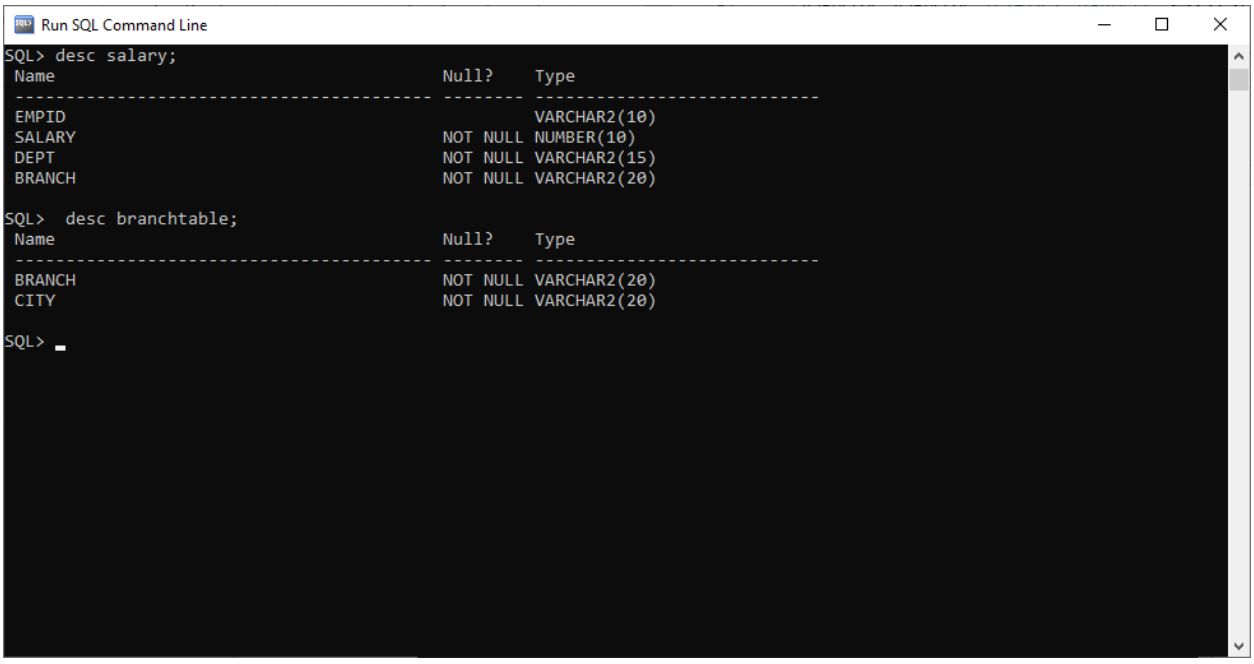


4. **Describe the newly created tables:**

- **SQL> desc salary;**



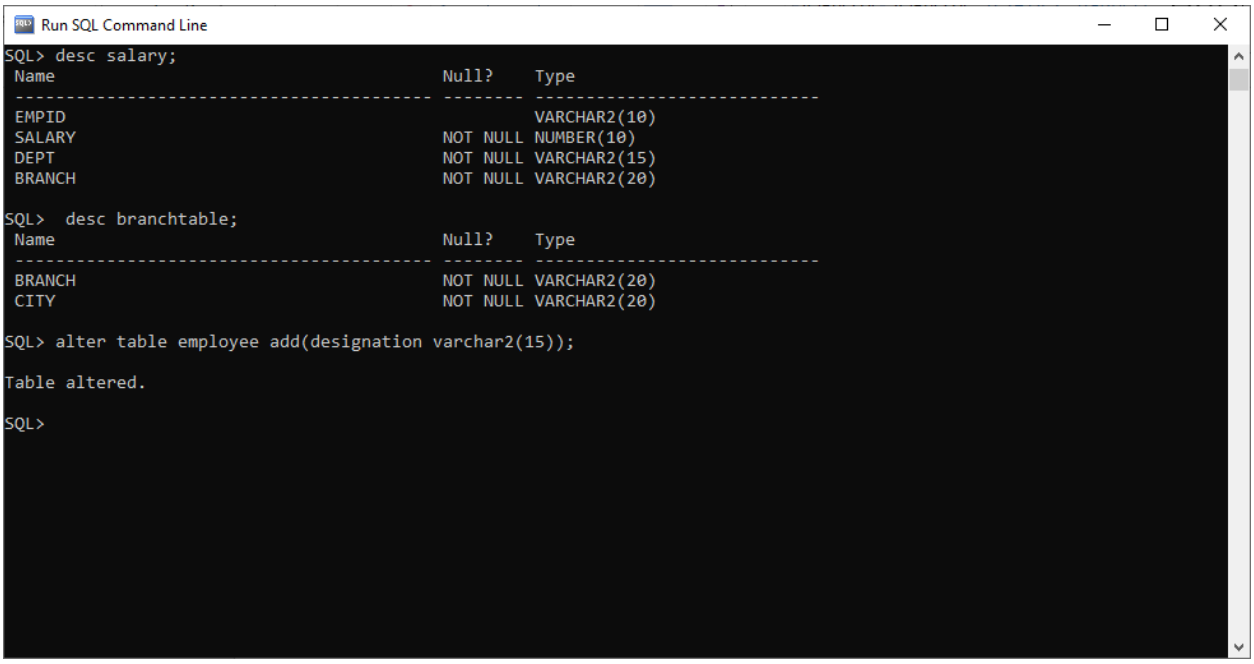

- **SQL> desc branchtable;**



5. **Alter Table:**

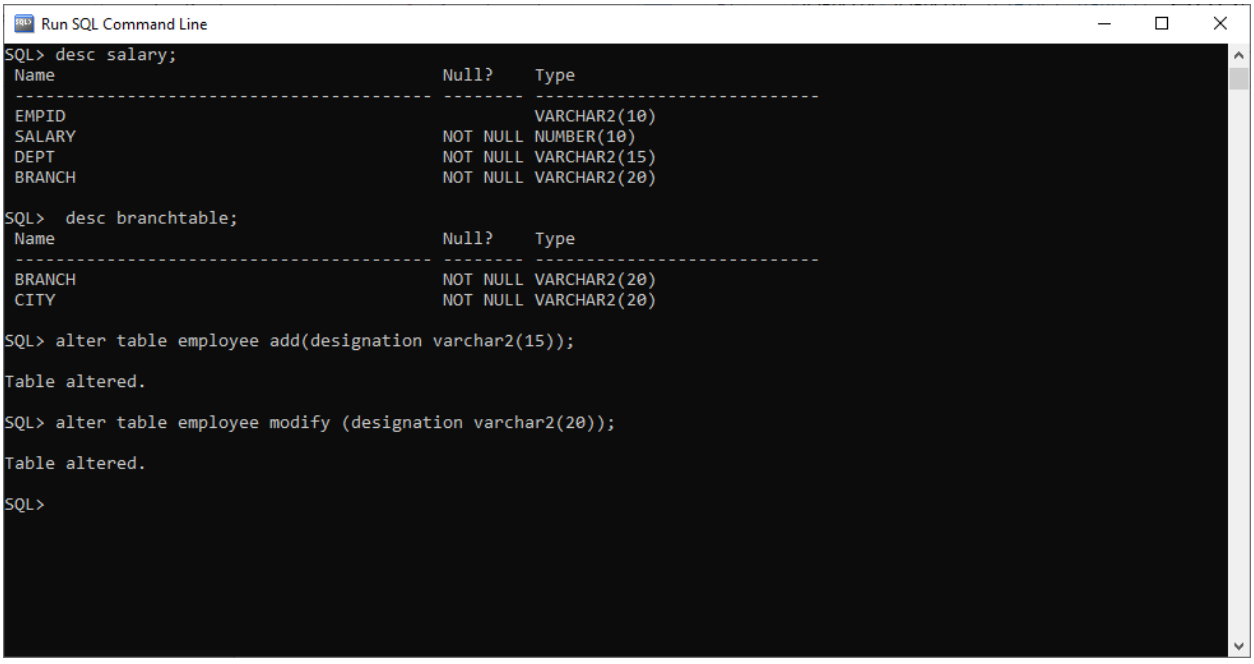
**I. ADD:**

**SQL> alter table employee add(designation varchar2(15));**




**II.MODIFY**

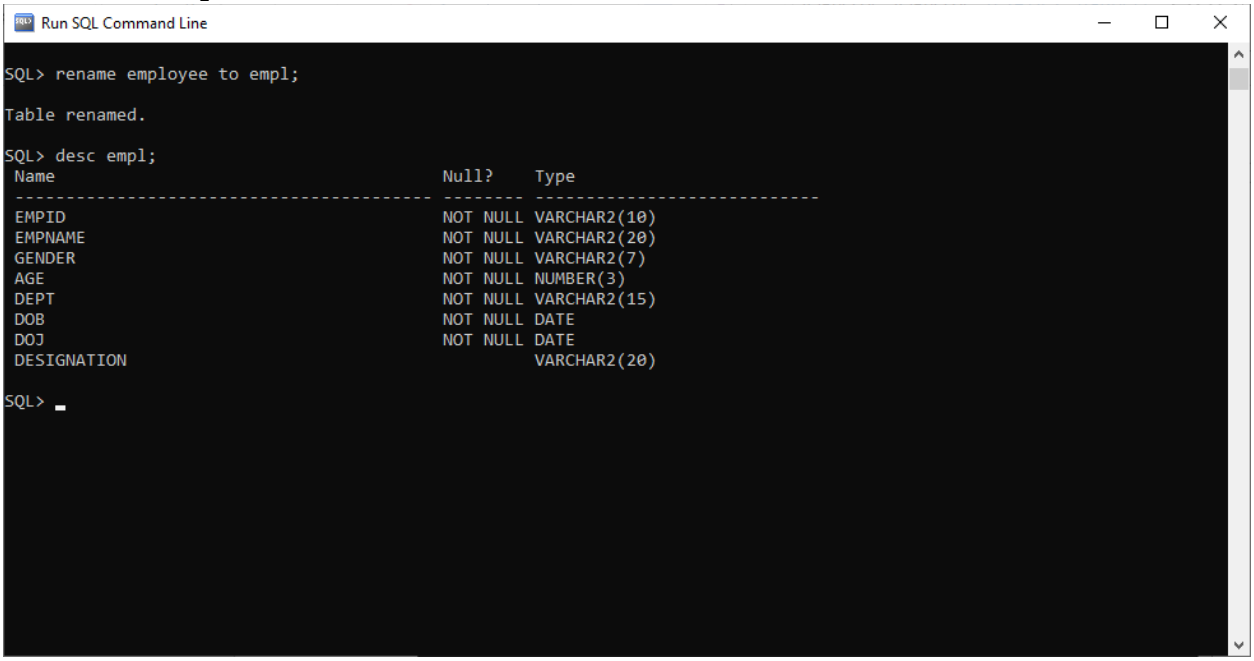
**SQL> alter table employee modify (designation varchar2(20));**



## 6. Rename table:

### SQL> rename emp to empl;


### SQL> desc empl;



7. **Truncate table data;**

Create a new table emp for this purpose.

**SQL> create table emp (empid varchar(10), empname varchar(20), dept varchar (20), age number(3), sex char);**

And then truncate, however, first you need to add some values.

**SQL> insert into emp values(&empid,'&empname','&dept',&age,'&sex');**

**Enter value for empid: 1**

**Enter value for empname: shiza**

**Enter value for dept: SE**

**Enter value for age: 25**

**Enter value for sex: f**

**old 1: insert into emp values(&empid,'&empname','&dept',&age,'&sex')**

**new 1: insert into emp values(1,'shiza','SE',25,'f')**

1 row created.

**SQL> insert into emp values(&empid,'&empname','&dept',&age,'&sex');**

**Enter value for empid: 2**

**Enter value for empname: maria**

**Enter value for dept: CS**

**Enter value for age: 26**

**Enter value for sex: f**

**old 1: insert into emp values(&empid,'&empname','&dept',&age,'&sex')**

**new 1: insert into emp values(2,'maria','CS',26,'f')**

1 row created.

**SQL> insert into emp values(&empid,'&empname','&dept',&age,'&sex');**

**Enter value for empid: 3**

**Enter value for empname: haris**

**Enter value for dept: SE**

**Enter value for age: 20**

**Enter value for sex: m**

**old 1: insert into emp values(&empid,'&empname','&dept',&age,'&sex')**

**new 1: insert into emp values(3,'haris','SE',20,'m')**

1 row created.

**SQL> insert into emp values(&empid,'&empname','&dept',&age,'&sex');**

**Enter value for empid: 4**

**Enter value for empname: faraz**

**Enter value for dept: CS**

**Enter value for age: 30**

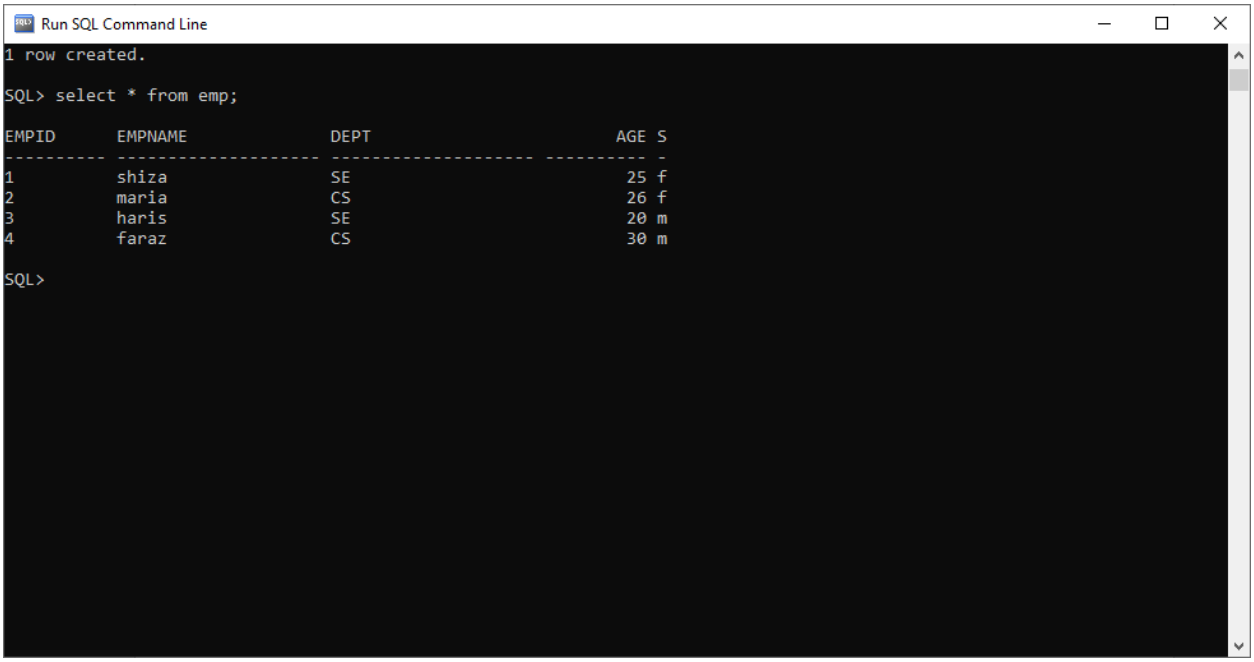
**Enter value for sex: m**

**old 1: insert into emp values(&empid,'&empname','&dept',&age,'&sex')**

**new 1: insert into emp values(4,'faraz','CS',30,'m')**

1 row created.

8. **SELECT Data from table.**
   **SQL> select * from emp;**

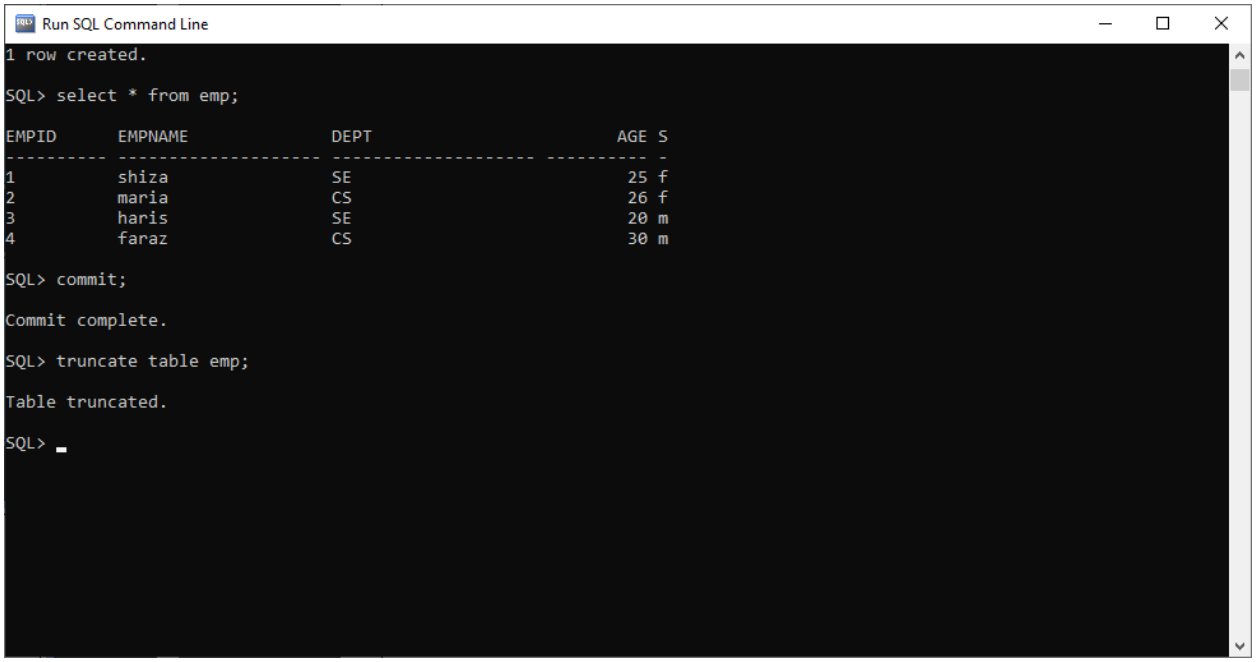


## 9. Commit database

The commit command in SQL lets a user permanently save all the changes made in the transaction of a database or table. Once you execute the COMMIT, the database cannot go back to its previous state in any case.

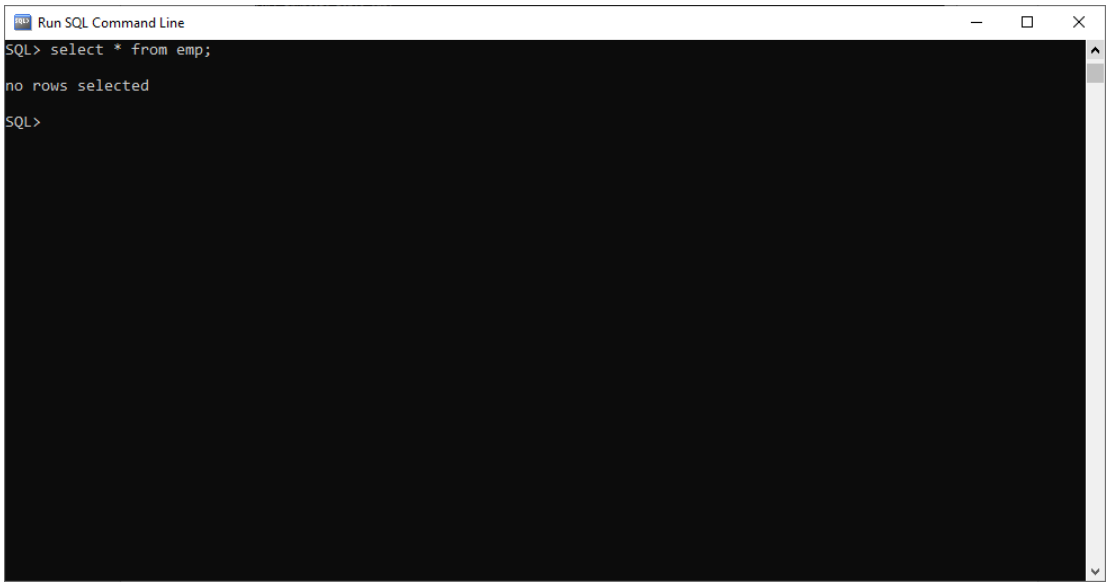
**SQL> commit;**

**SQL> truncate table emp;**




Checking whether table is truncated or not.

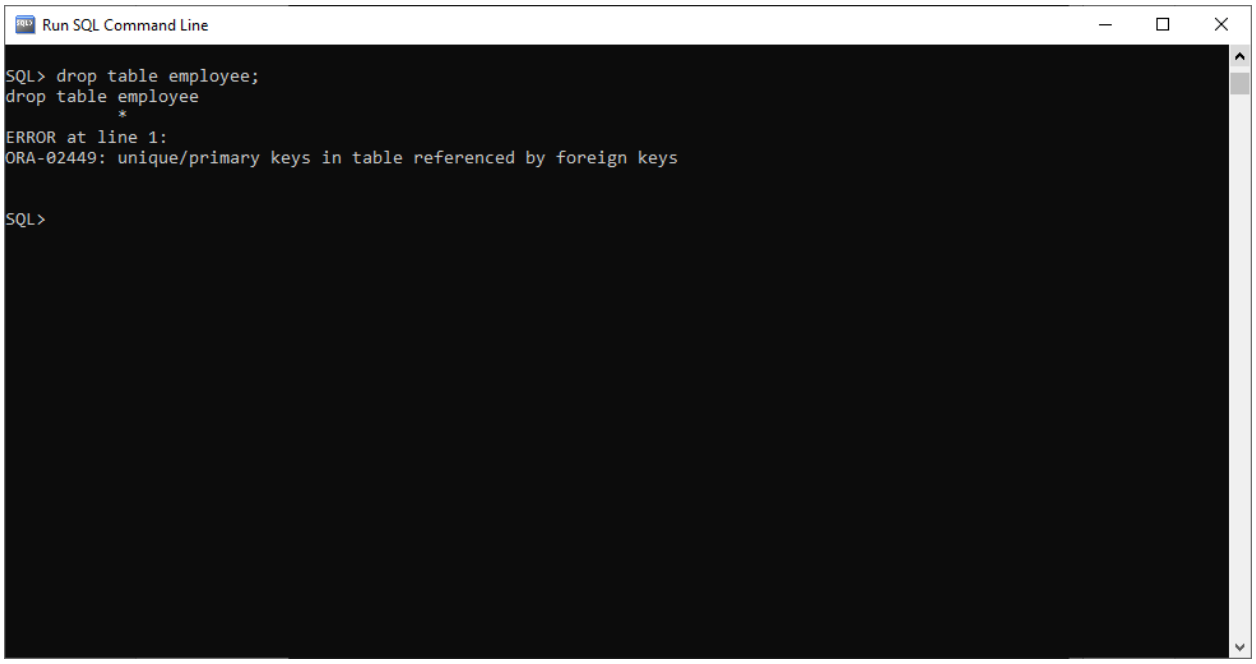
**SQL> select * from emp;**



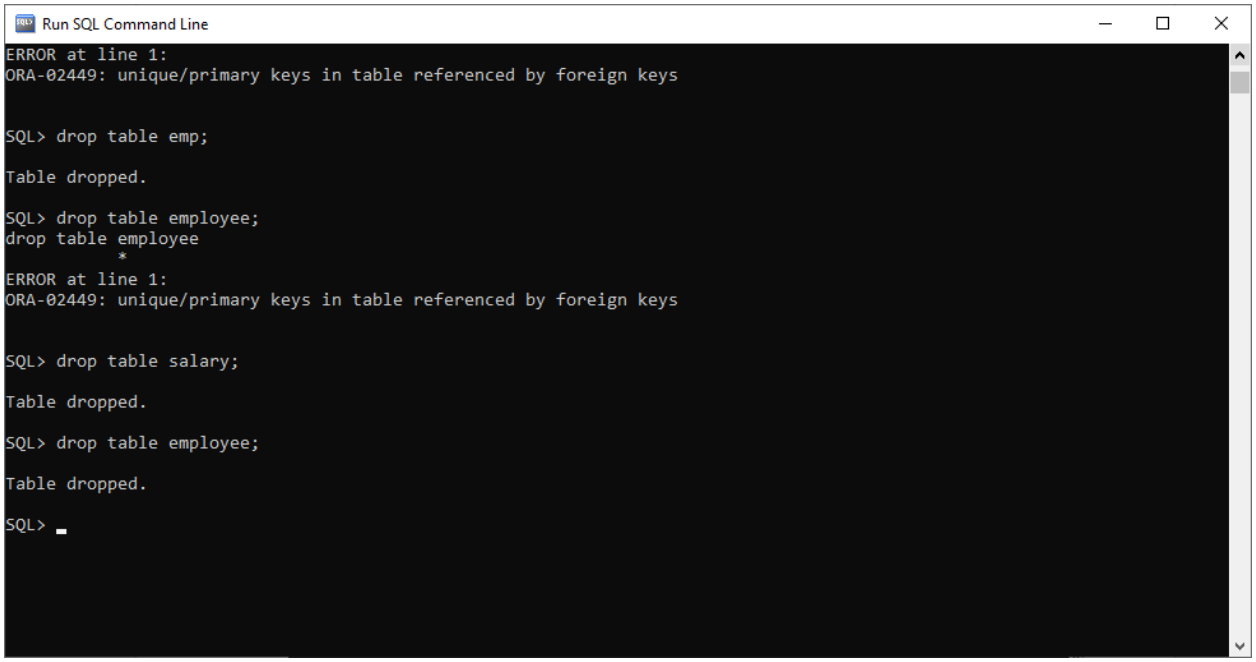
## 10. DROP TABLE

1. **Dropping table that has been referenced.**

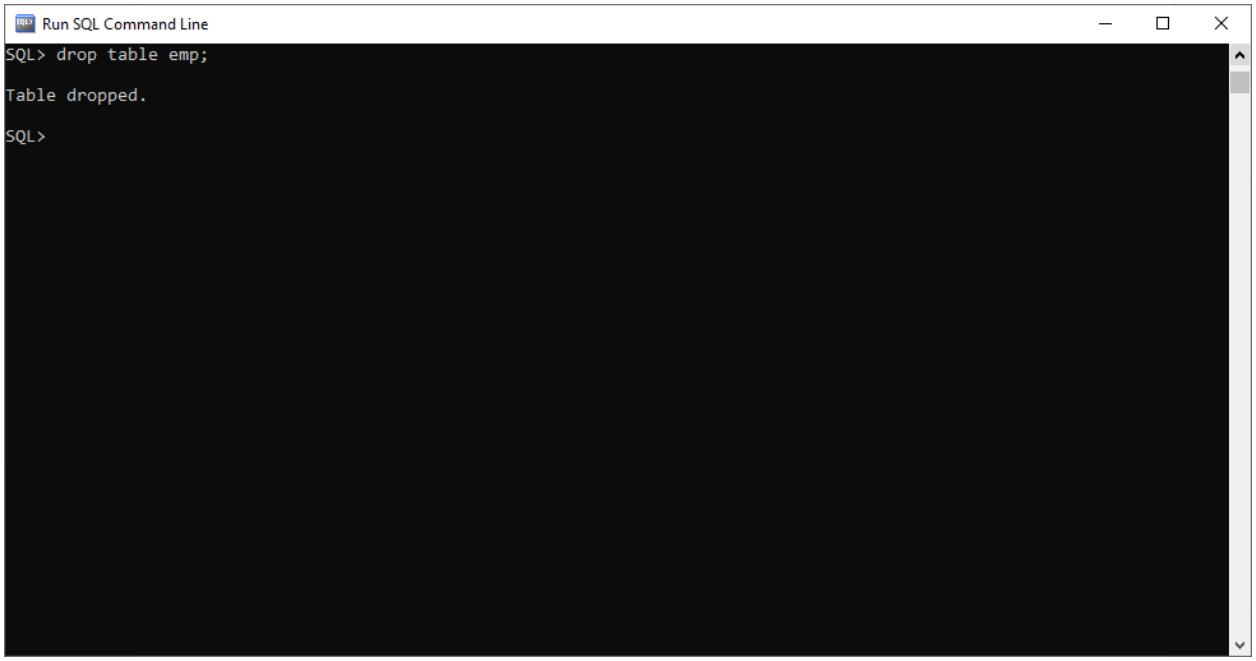
**SQL> drop table employee;**




It is required to first drop the referencing table and then the referenced table can be dropped.

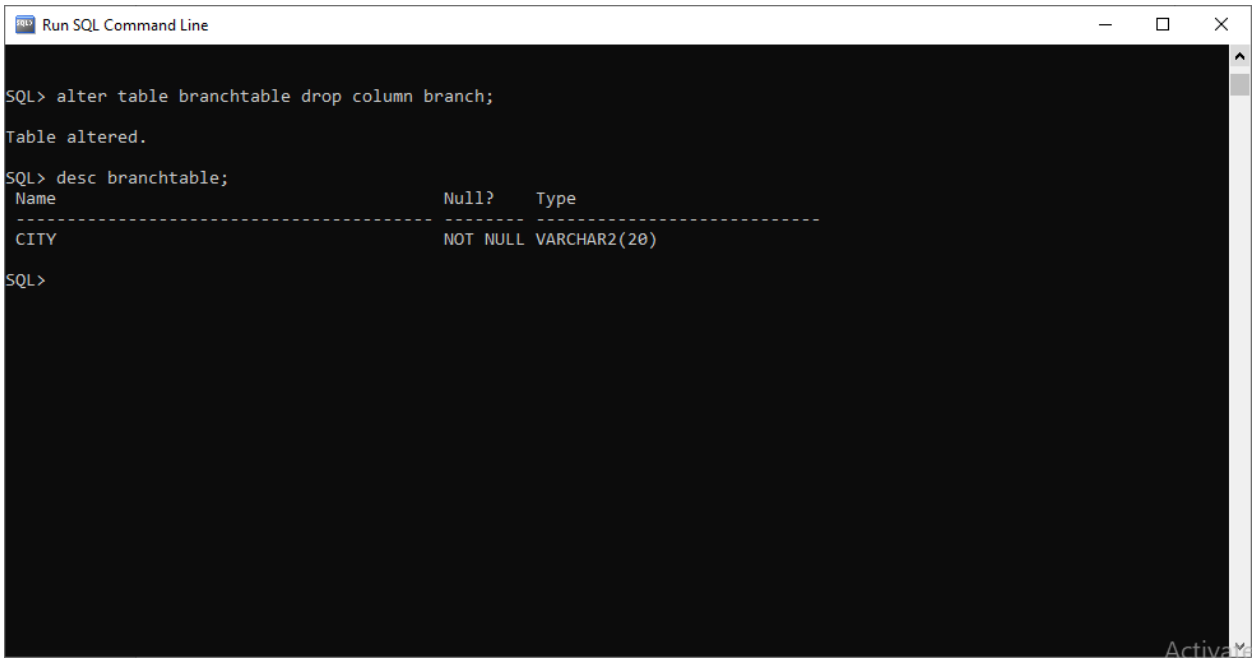


## 11. Does truncating the table, drops it as well?? NO

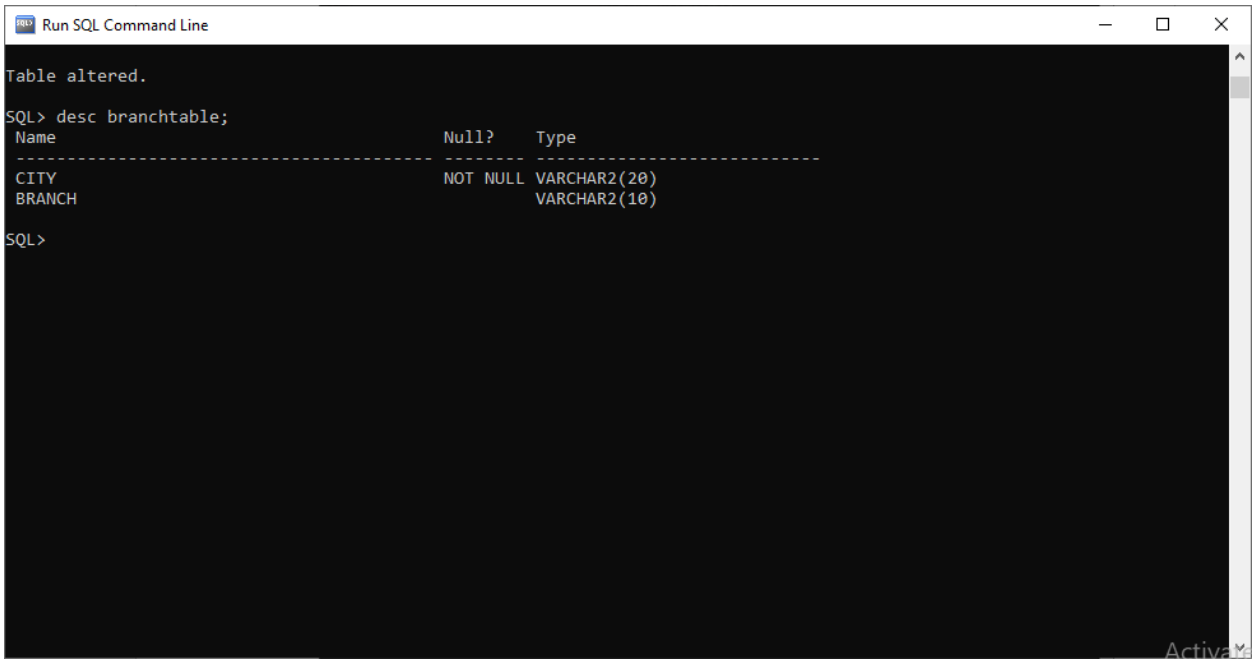



## 11. How to delete a column from table?

## SQL> alter table branchtable drop column branch;



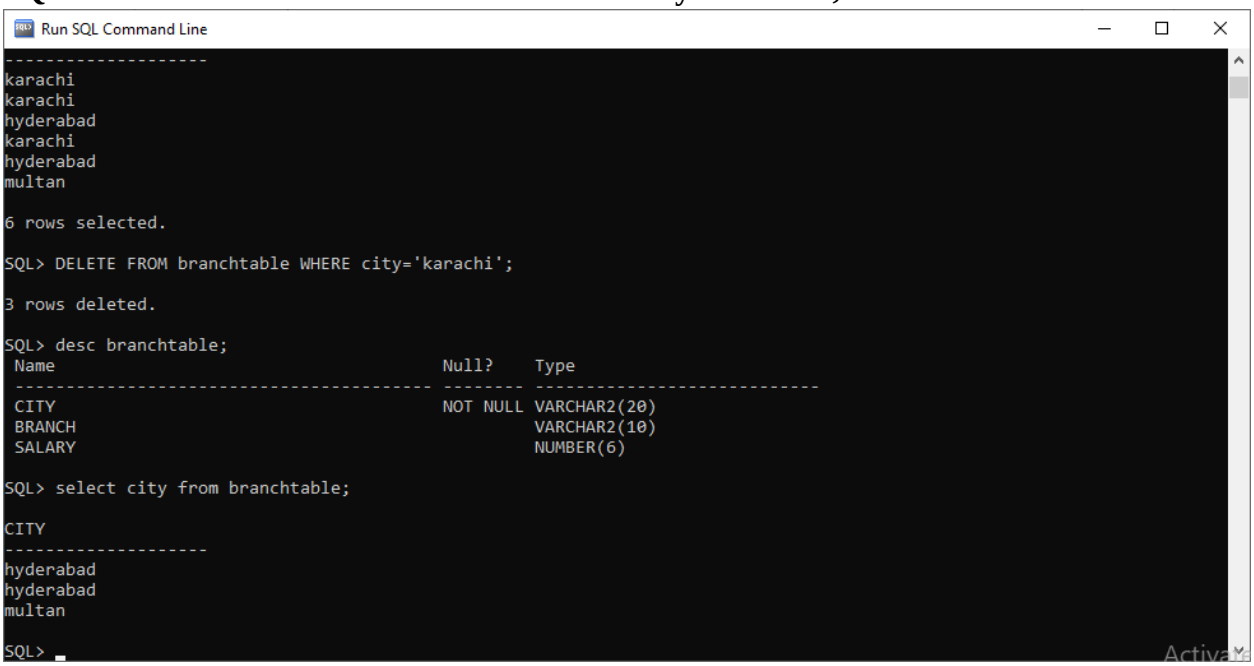
## 12. Selecting Specific Columns:




## SQL> select city FROM branchtable;

## 13. Deleting specific rows

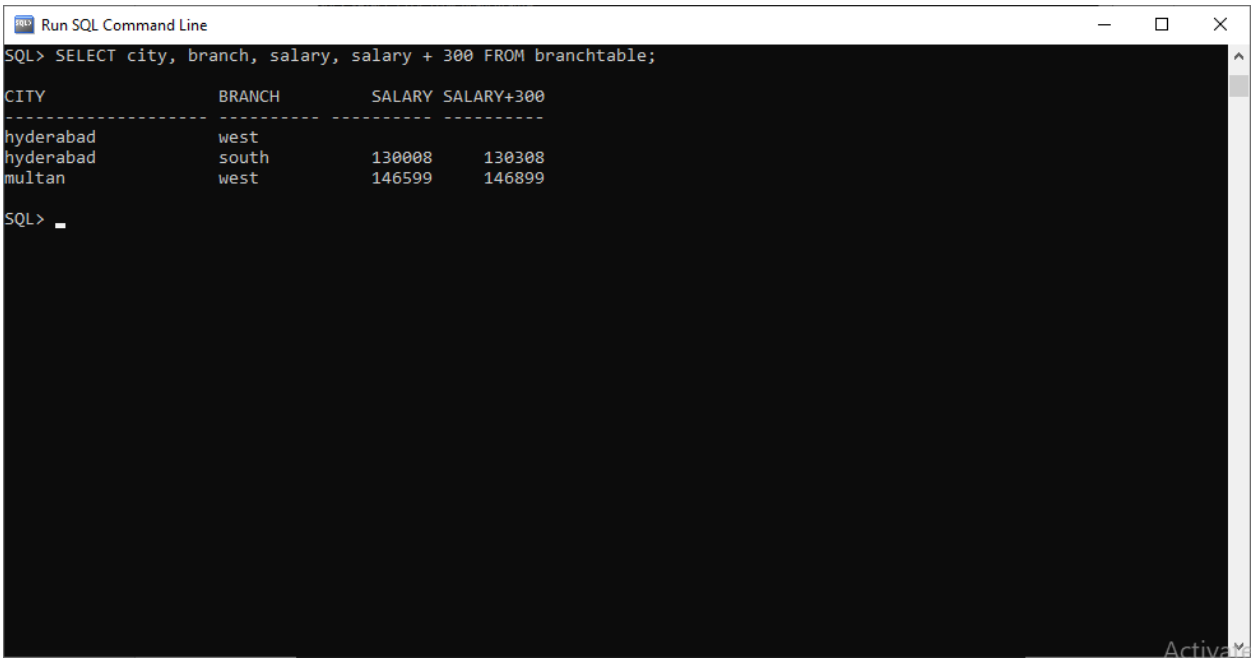
**SQL> DELETE FROM branchtable WHERE city='karachi';**




## 14. Create expressions with number and date data by using arithmetic operators.

Operators that can be used are :
Add, Subtract, Multiply, Divide

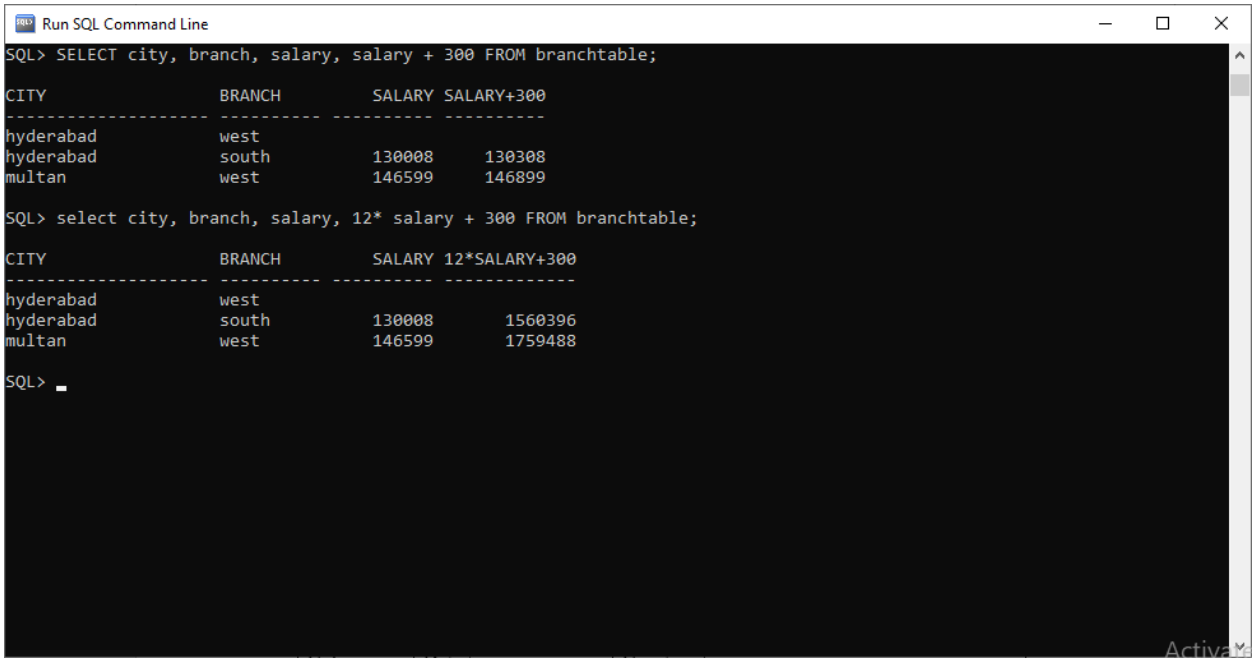
**SQL> select city, branch, salary, salary + 300 FROM branchtable;**



**15. Operator precedence:**

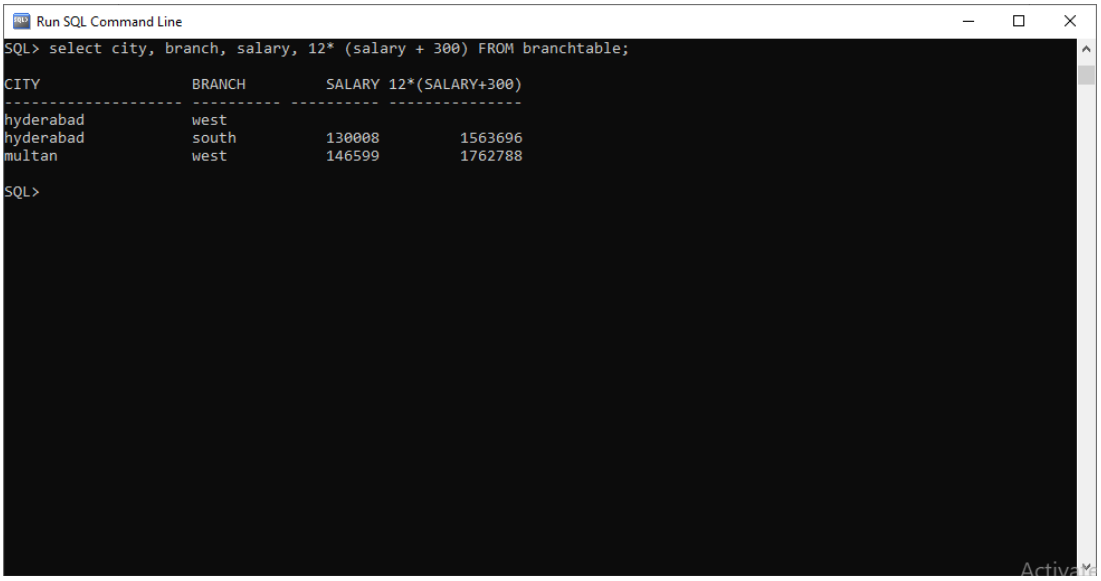
- / *+ _
- Multiplication and division take priority over addition and subtraction.
- Operators of the same priority are evaluated from left to right.
- Parentheses are used to force prioritized evaluation and to clarify statements.

**SQL>select city, branch, salary, 12* salary + 300 FROM branchtable;**




**16. Using Parentheses:**

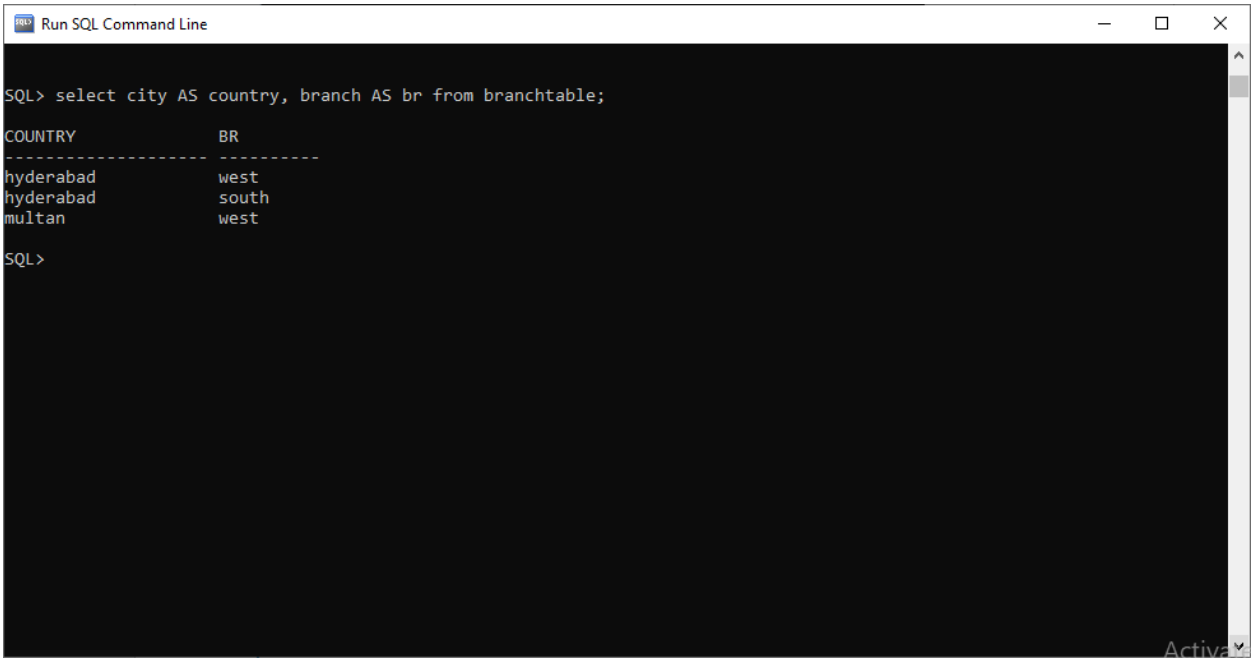
**SQL>select city, branch, salary, 12* (salary + 300) FROM branchtable;**



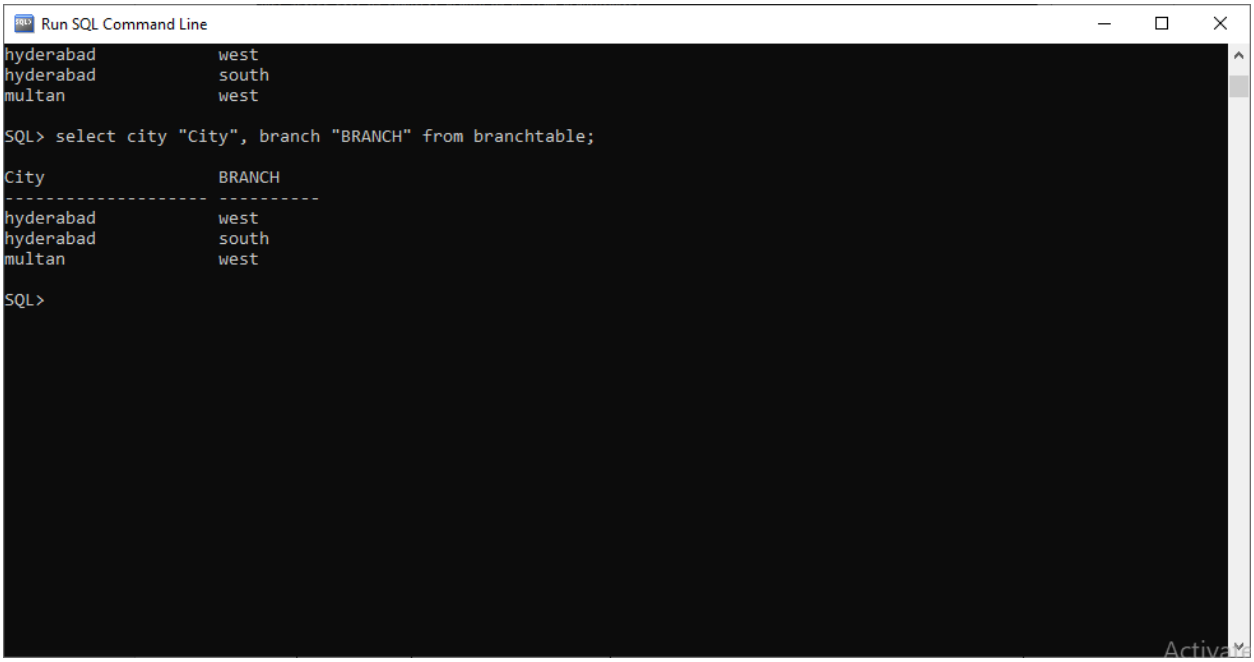
### 17. Defining a Column Alias

A column alias renames a column heading and is useful with calculations. It immediately follows the column nam. There can also be the optional AS keyword between the column name and alias The Alias requires double quotation marks if it contains spaces or special characters or is case sensitive

**SQL>select city AS country, branch AS br from branchtable;**



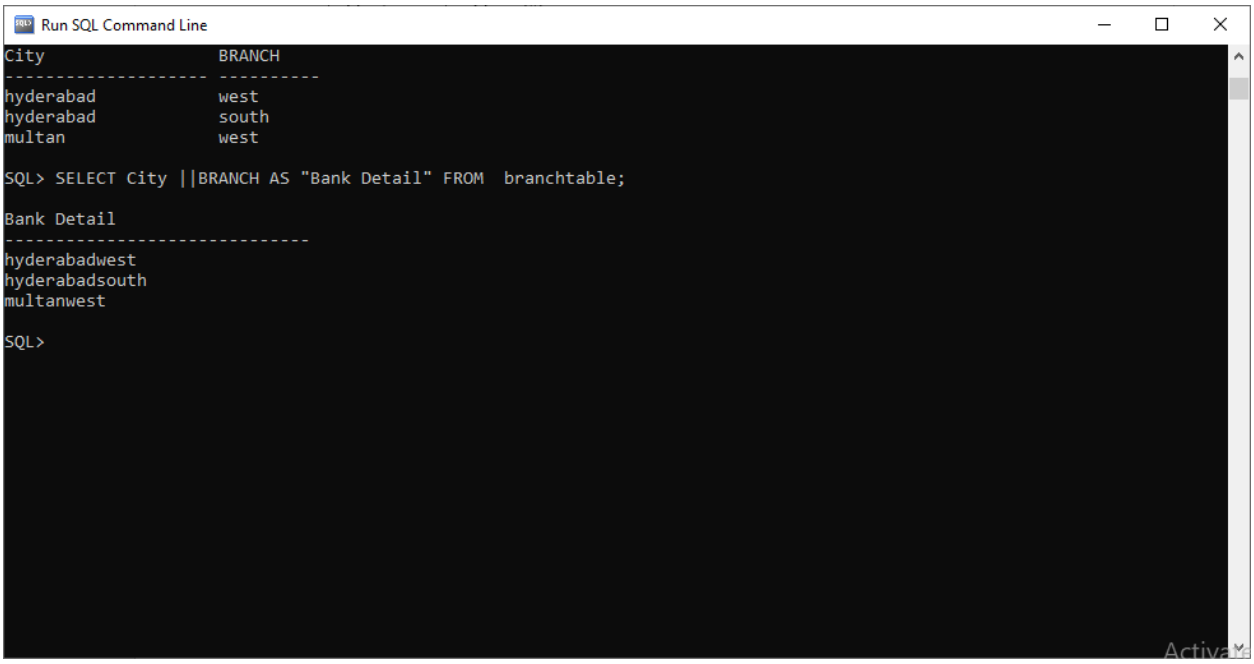

**SQL>select city “City”, branch “BRANCH” from branchtable;**



### 18. A concatenation operator:

A concatenation operator concatenates columns or character strings to other columns and is represented by two vertical bars (||)
It creates a resultant column that is a character expression

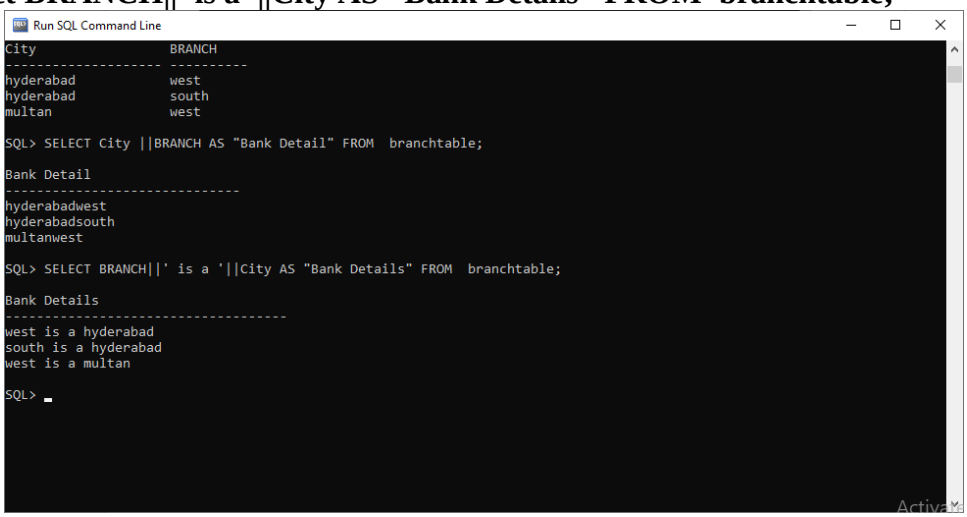
**SQL>select City ||BRANCH AS "Bank Detail" FROM  branchtable;**




### 19. Literal Character Strings

- A literal value is a character, a number, or a date included in the SELECT list.
- Date and character literal values must be enclosed within single quotation marks.
- Each character string is output once for each row returned.

**SQL>select BRANCH||' is a '||City AS "Bank Details" FROM  branchtable;**



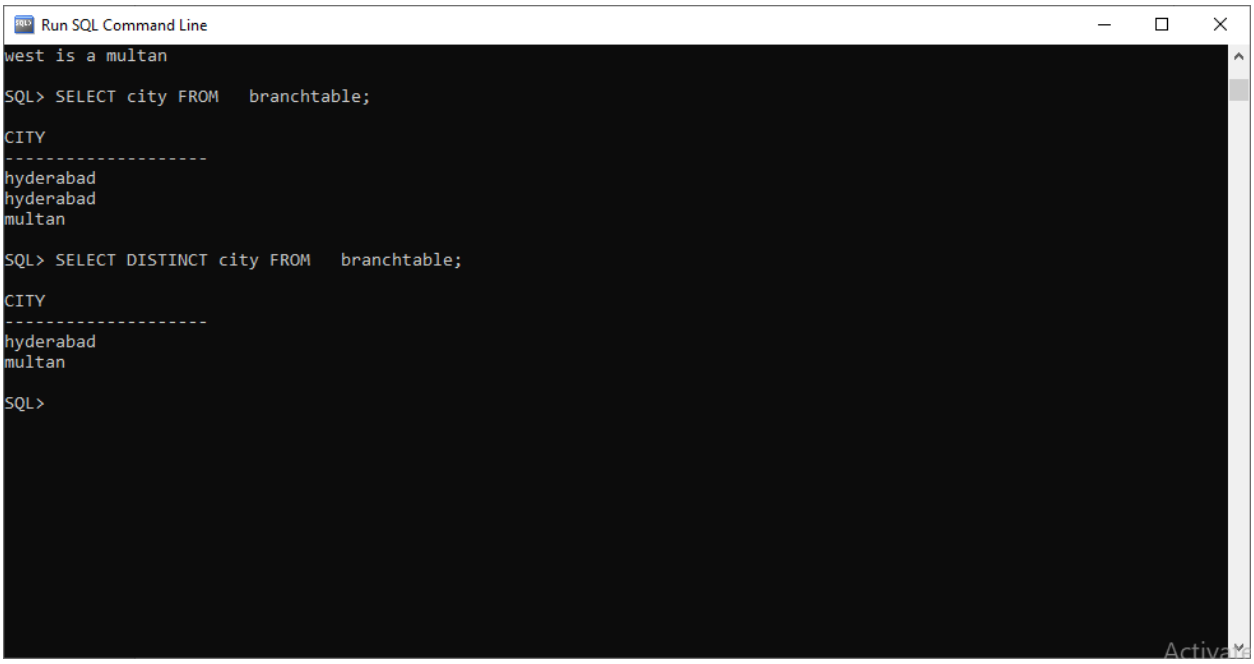
## 20. Duplicate Rows

The default display of queries is all rows, including duplicate rows.

SQL> select city FROM branchtable;

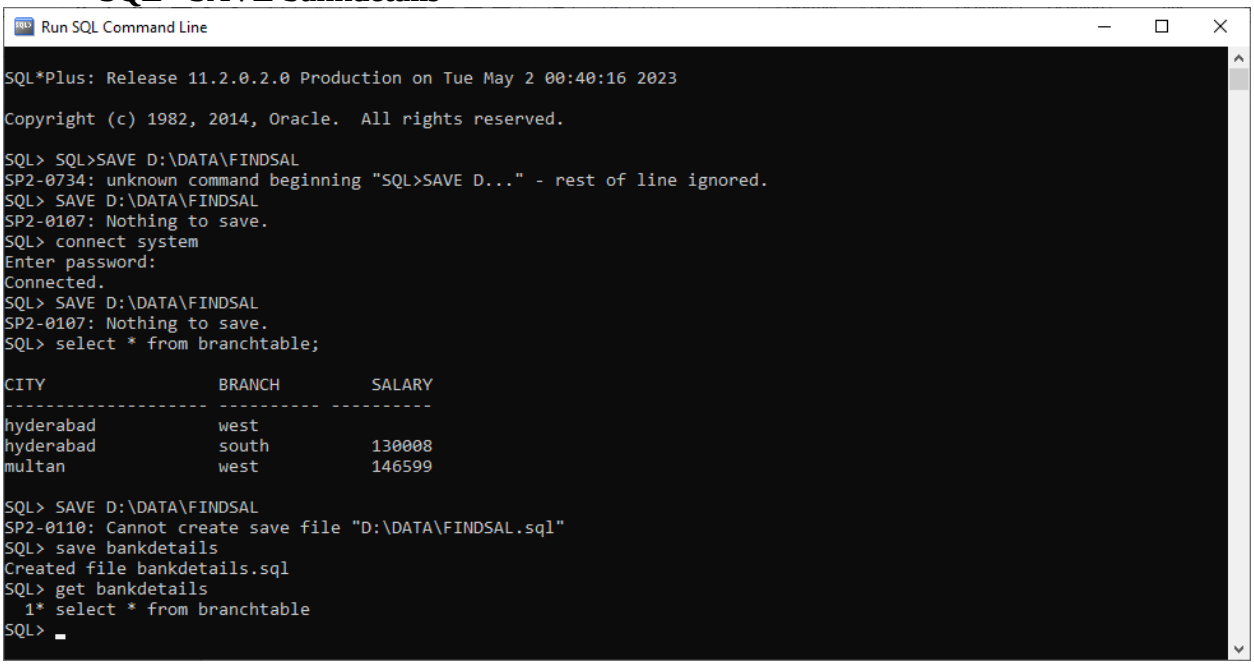
Eliminate duplicate rows by using the DISTINCT keyword in the SELECT clause.

**SQL>select DISTINCT city FROM branchtable;**

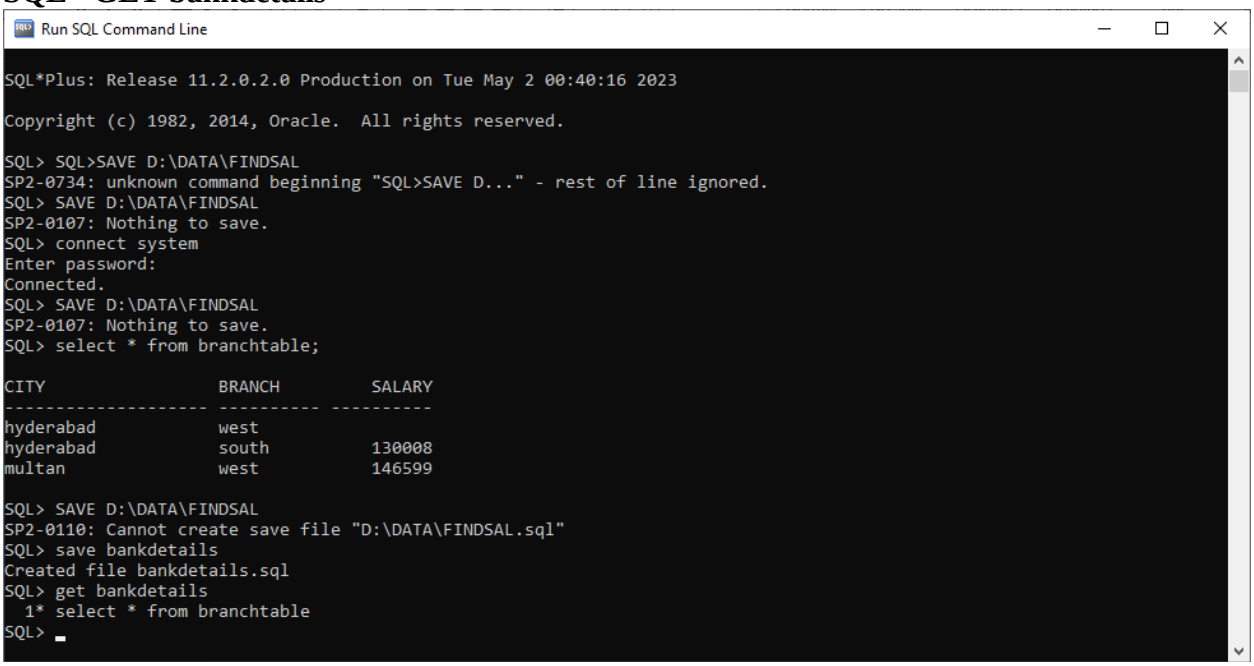



## 21. Saving the contents

**SQL> SAVE bankdetails**

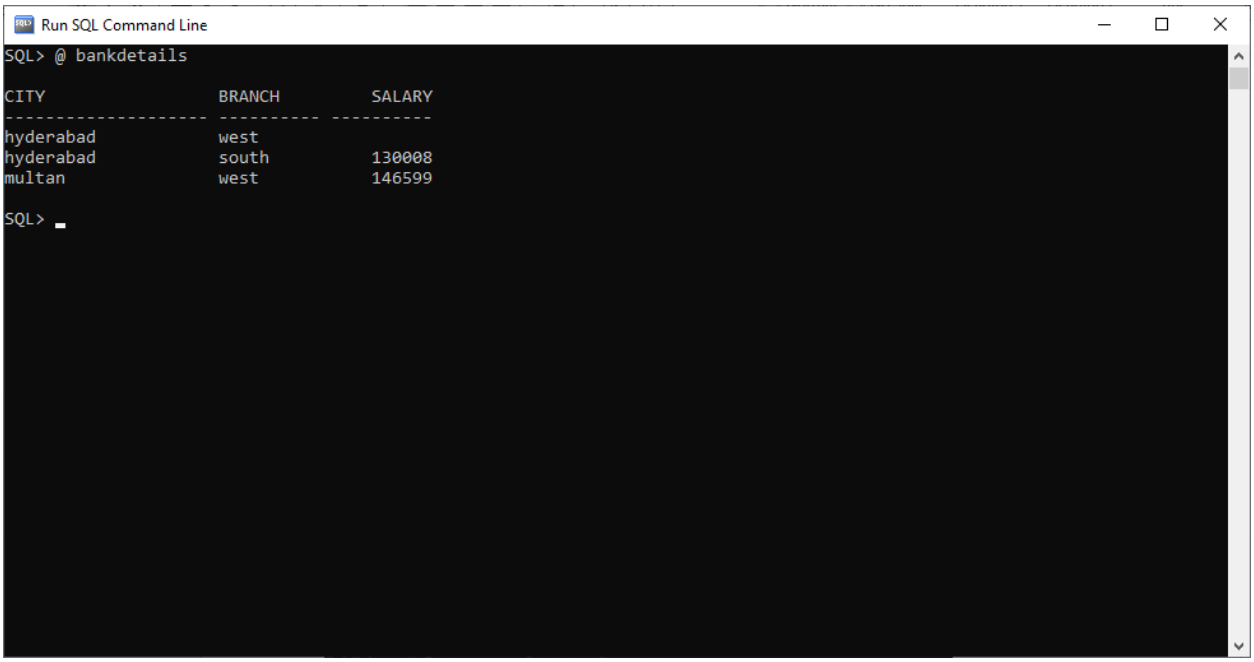


**23. Get the contents**
**SQL> GET bankdetails**



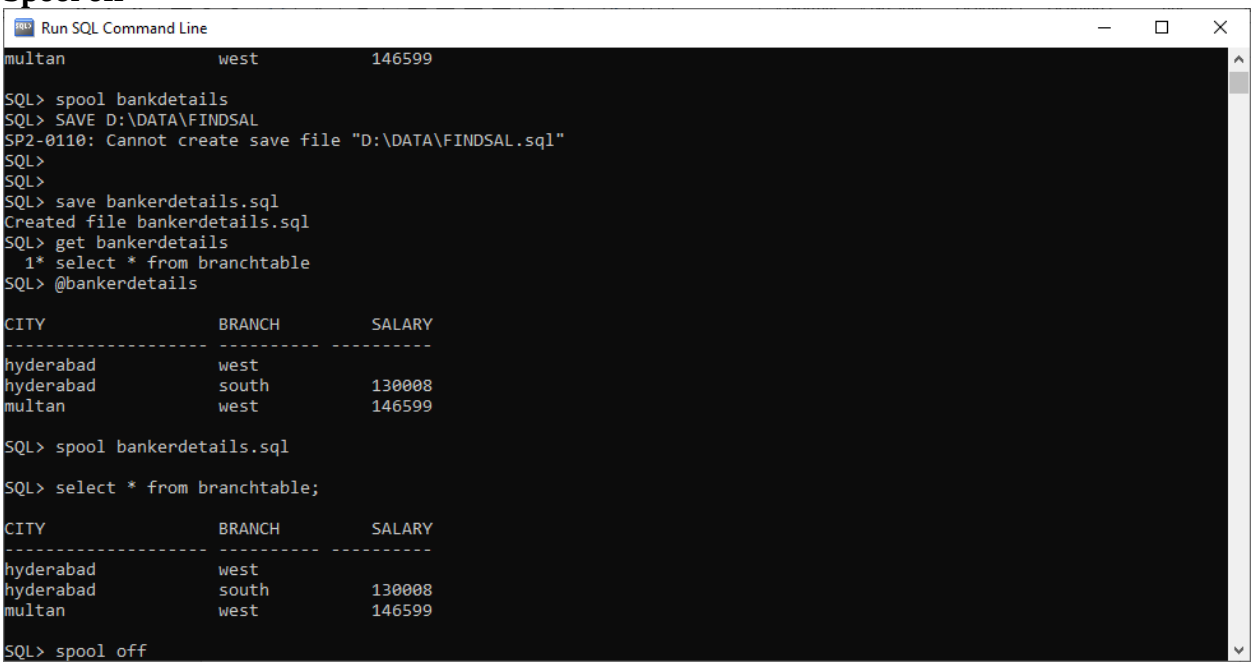

**24.“@” Runs a previously saved command file e.g. SQL>@ filename**

**SQL> @ bankdetails**

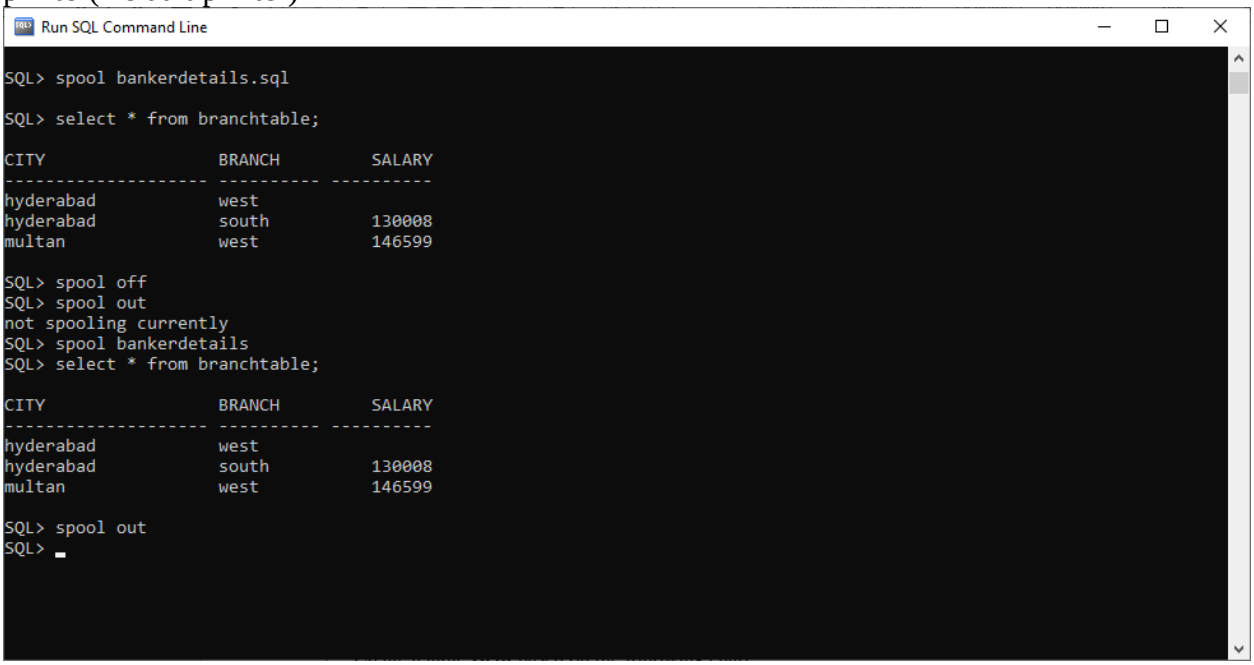


**25. SPOOL**:
Stores query results in a file e.g. SQL>SPOOL filename.ext

**SQL> spool bankerdetails.sql**
**SQL> select * from branchtable;**
**Spool off**



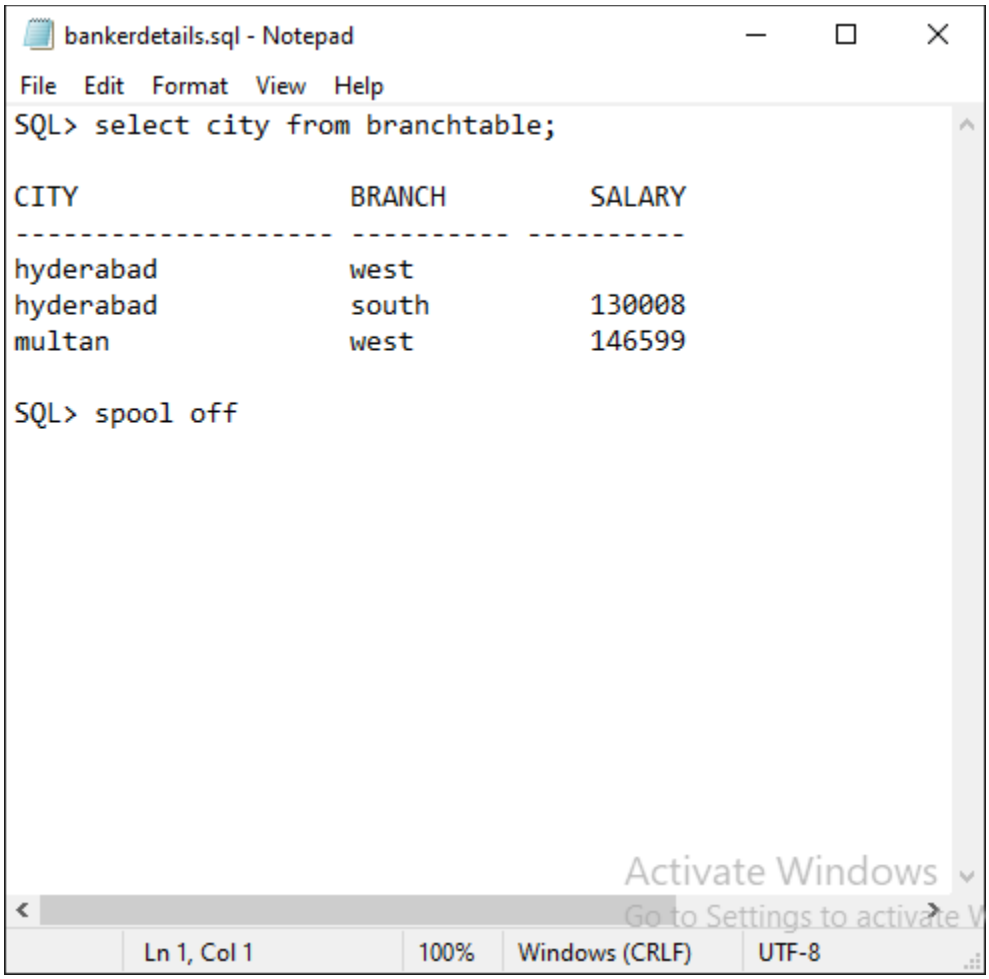

**SPOOL OUT**: Closes the spool file and sends the file results to the system printer(Default printer)



**Difference between spool and save:**

Save only saves last entered SQL command or the command that is in SQL buffer where as spool saves all commands and their outputs after writing spool command, unless spool off has been written

**26. EDIT [filename[.ext]]: Invokes editor to edit contents of a saved file**
**SQL> edit bankerdetails**




**27. Commenting in SQL Plus:**
For commenting any line in oracle SQL Plus, we use two dashes

Such as:

--This is a comment

**Lab Exercise:**

1. Create a table **Student** based on the following chart:

| Column | Data type | Constraints |
|---|---|---|
| Student_Id | Number (6) | Primary Key |
| Last_Name | Varchar2(15) | Not NULL |
| First_Name | Varchar2(15) | Not NULL |
| gender | Char(3) | |

   Confirm and validate the creation of the new table.

2. Create a table **Dept** based on the following chart:

| Column | Data type | Constraints |
|---|---|---|
| Dept_Code | Char (3) | Not NULL |
| Dept_Name | Varchar2(20) | Not NULL |

   Confirm and validate the creation of the new table.

3. Add a new column **Location** to **Dept** table which has data type Char(7). Confirm and validate the modification of the table.

4. Delete the column **Last_Name** from **Student**. Confirm and validate the modification of the table.

5. Increase **Last_Name** column to **25** characters long. Save the SQL statement as *ex5.sql*. Confirm and validate the modification of the table.
6. Create 2 more tables that you think are necessary for student data management.
7. Insert values in all the tables.
8. Select specific data from the tables
9. Select entire data from the tables
10. Truncate student table
11. Add 2 new columns to dept table
12. Delete a particular column from dept table.
13. Create arithmetic calculations
14. Select distinct values from the column
15. Concatenate columns and display
16. Drop all tables.

**LAB No 3.**

**Objective: Restricting and Sorting Data**

**Theory:**

**Syntax of Commands:**

1. **Limiting rows using selection**

   i. **Create a table**

   **SQL>create table employee ( empid varchar(10) primary key, ename varchar(20) not null, salary number (10) not null, deptno varchar(10) not null);**

```
SQL> create table employee (empid varchar(10) primary key, ename varchar (20) no
t null, salary number (10) not null, deptno varchar (10));

Table created.

SQL> desc employee;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------

 EMPID                                     NOT NULL VARCHAR2(10)
 ENAME                                     NOT NULL VARCHAR2(20)
 SALARY                                    NOT NULL NUMBER(10)
 DEPTNO                                             VARCHAR2(10)
```

   ii. **Then enter some values in this table**

```
SQL> insert into employee values ('&empid', '&ename', '&salary', '&deptno');
Enter value for empid: 1
Enter value for ename: shiza
Enter value for salary: 19000
Enter value for deptno: 10
old   1: insert into employee values ('&empid', '&ename', '&salary', '&deptno')
new   1: insert into employee values ('1', 'shiza', '19000', '10')

1 row created.
```

   iii. **Check all the data in the table using command**

   **SQL>select * from employee;**

```
SQL> select * from employee;

EMPID      ENAME                SALARY DEPTNO
---------- -------------------- ---------- ----------
1          shiza                 19000 10
2          maria                 16000 5
3          hina                  13000 4
4          amna                   8900 9
5          sadia                 16700 4
6          asma                    600 7

6 rows selected.

SQL>
```

iv. **Now limit the rows, using where**

**SQL>select empid, ename, salary, deptno FROM employee WHERE deptno= 10;**

```
SQL> select empid, ename, salary, deptno from employee where deptno=10;

EMPID      ENAME                SALARY DEPTNO
---------- -------------------- ---------- ----------
1          shiza                 19000 10

SQL>
```

**SQL>select ename from employee where salary=350;**

```
SQL> select ename from employee where salary=350;

no rows selected

SQL>
```

v. **Comparison**

Comparison between multiple values can be carried out through comparison operators. Some of the comparison operators are:

**= Equal to**

**>; Greater than**

**>;= Greater than or equal to**

**<; Less than**

**<= Less than or equal to**

**<>; Not equal to**

Note:The symbol != and ^= can also represent the not equal to condition.

**SQL>select ename, salary FROM employee WHERE salary <= 3000;**

```
SQL> select ename, salary from employee where salary <=3000;

ENAME                SALARY
-------------------- ----------
asma                    600

SQL>
```

### vi. Advance Comparison:

### i. Using BETWEEN AND Condition

Use the BETWEEN AND condition to display rows based on a range of values.

**SQL>select ename, salary FROM employee WHERE salary BETWEEN 2500 AND 3500;**

This condition has a Lower limit and an Upper limit

```
SQL> select ename, salary from employee where salary between 800 and 20000;

ENAME                    SALARY
-------------------- ----------
shiza                     19000
maria                     16000
hina                      13000
amna                       8900
sadia                     16700

SQL>
```

### ii. Using IN Condition

Use the IN membership condition to test for values in a list.

**SQL>select empid, ename, salary, deptno FROM employee WHERE deptno IN (10, 9,5);**

```
SQL> select empid, ename, salary, deptno from employee where deptno IN (10,9,5);

EMPID      ENAME                SALARY DEPTNO
---------- -------------------- ---------- ----------
1          shiza                 19000 10
2          maria                 16000 5
4          amna                   8900 9

SQL>
```

### iii. Using the LIKE Condition

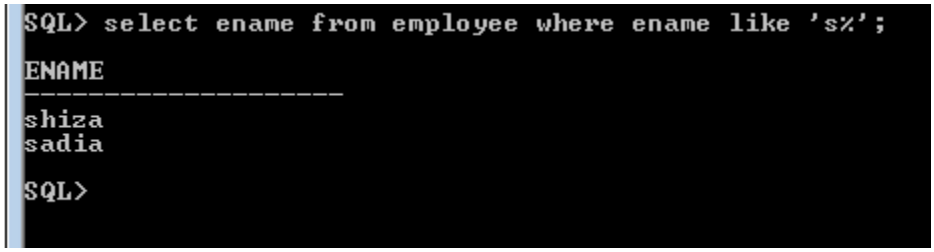
In SQL, the LIKE operator is used to perform pattern matching on text data. It is primarily used in the WHERE clause of a query to filter rows based on specific patterns or values.

The LIKE operator compares a column value to a pattern using wildcard characters. The two wildcard characters commonly used are:

1. % (percent sign): It represents any sequence of characters (including no characters).
2. '_' (underscore): It represents any single character.

Suppose you want to retrieve all the employees whose names start with the letter 's'. The SQL query would look like this

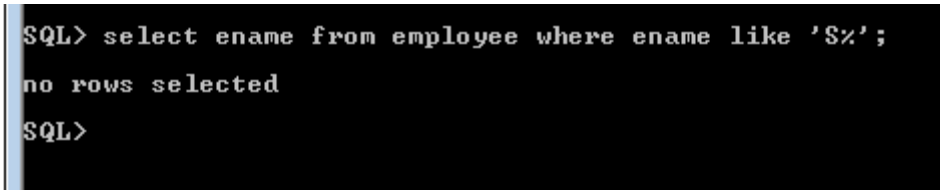
**SQL>select ename FROM employee WHERE ename LIKE ‘s%’;**




Note that the letters are case sensitive. You will not receive any output with the query

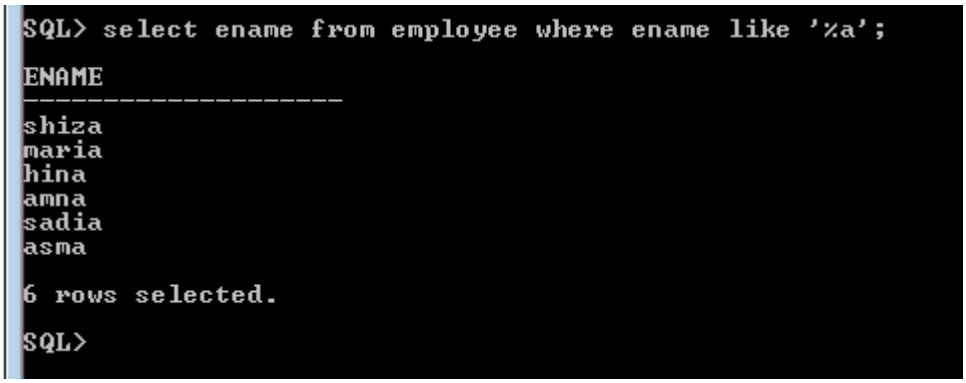
**SQL>select ename FROM employee WHERE ename LIKE ‘S%’;**

As there are no names that start with ‘S’



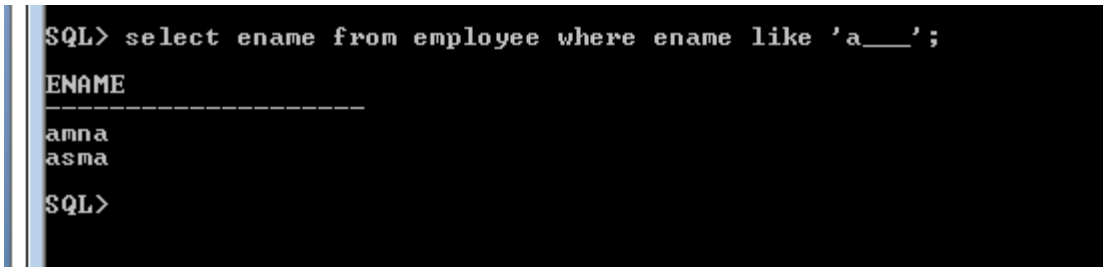

You can also use % in different positions to match different patterns. For example, if you want to retrieve all the employees whose names end with the letter 'a', you can use the following query:

**SQL>select ename FROM employee WHERE ename LIKE ‘%a’;**



Similarly, the underscore _ wildcard can be used to match a single character. For instance, to retrieve all employees whose names have exactly four characters and start with the letter 'A', you can use the following query:

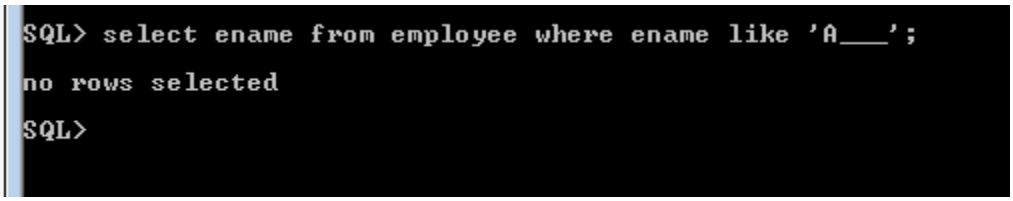
**SQL>select ename FROM employee WHERE ename LIKE ‘a___’;**




In this case, the underscore _ is used four times to represent four characters.

No rows will be selected if you use the query as

**SQL>select ename FROM employee WHERE ename LIKE ‘A___’;**

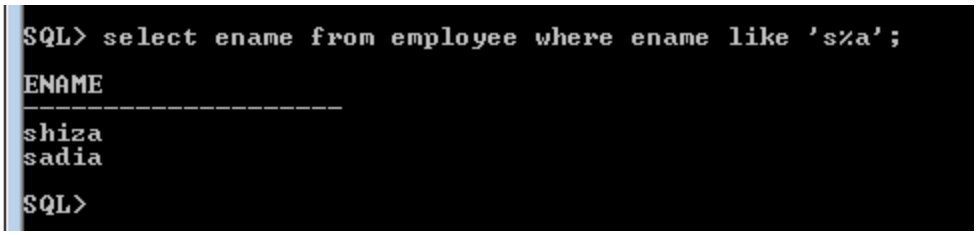



These examples demonstrate the basic usage of the LIKE operator in SQL queries for pattern matching. You can combine wildcards and regular characters to create more complex patterns as per your requirements.

If you want to to retrieve all employees whose names start with 's' and end with 'a'. You can use the following query:

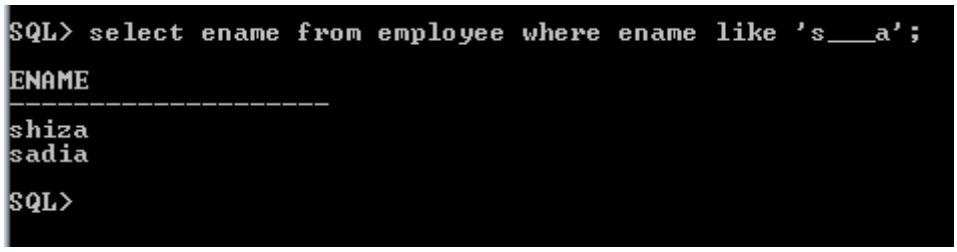
**SQL>select ename FROM employee WHERE ename LIKE ‘s%a’;**

The % wildcard is used to represent any sequence of characters between 'J' and 'n'.



**Using the underscore**

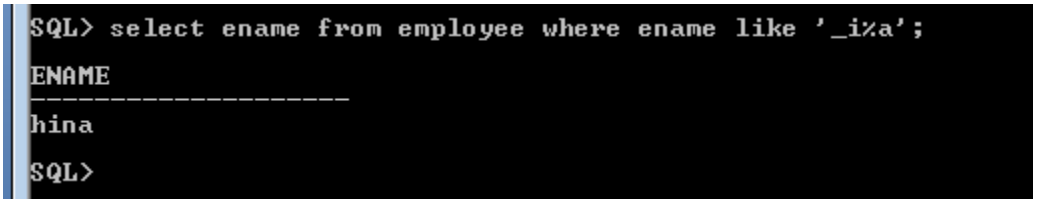
**SQL>select ename FROM employee WHERE ename LIKE ‘s___a’;**




You can combine the % and _ signs in SQL to create a more complex pattern using the LIKE operator in SQL*Plus:

Suppose you want to retrieve all employees whose names have at least three characters, where the second character is 'i' and the last character is 'a'. The remaining characters can be any sequence of characters. You can use the following SQL query in SQL*Plus:

SQL>select ename FROM employee WHERE ename LIKE ‘_i%a’;




**Comparing numeric values with queries:**

You can also compare between numeric values

**SQL>select ename, salary FROM employee WHERE ;**

| | |
|---|---|
| 1 | **WHERE SALARY LIKE '200%'**<br>Finds any values that start with 200. |
| 2 | **WHERE SALARY LIKE '%200%'**<br>Finds any values that have 200 in any position. |
| 3 | **WHERE SALARY LIKE '_00%'**<br>Finds any values that have 00 in the second and third positions. |
| 4 | **WHERE SALARY LIKE '2_%_%'**<br>Finds any values that start with 2 and are at least 3 characters in length. |
| 5 | **WHERE SALARY LIKE '%2'**<br>Finds any values that end with 2. |
| 6 | **WHERE SALARY LIKE '_2%3'**<br>Finds any values that have a 2 in the second position and end with a 3. |
| 7 | **WHERE SALARY LIKE '2___3'**<br>Finds any values in a five-digit number that start with 2 and end with 3. |

```
SQL> select ename, salary from employee where salary like '160%';

ENAME                SALARY
-------------------- ----------
maria                 16000

SQL>
```

**iii. Using NULL**

**SQL>select ename, salary FROM employee WHERE deptno IS NULL;**

```
SQL> select ename, salary from employee where deptno is NULL;

ENAME                SALARY
-------------------- ----------
haris                  9000

SQL>
```

**iv. Using the AND Operator. AND requires both conditions to be true.**

**SQL>select empid, ename, salary, deptno FROM employee WHERE salary>=1000 AND ename %adi%;**

```
SQL> select empid, ename, salary, deptno from employee where salary>=1000 AND en
ame LIKE '%adi%';

EMPID      ENAME                SALARY DEPTNO
---------- -------------------- ---------- ----------
5          sadia                 16700 4

SQL>
```

**v. Using the OR Operator. OR requires either condition to be true.**

**SQL>select empnid, ename, salary, deptno FROM employee WHERE salary>= 1000 OR ename LIKE %adi%;**

```
SQL> select empid, ename, salary, deptno from employee where salary>=1000 OR ena
me LIKE '%adi%';

EMPID      ENAME                SALARY DEPTNO
---------- -------------------- ---------- ----------
1          shiza                 19000 10
2          maria                 16000 5
3          hina                  13000 4
4          amna                   8900 9
5          sadia                 16700 4
7          haris                  9000

6 rows selected.
```

**vi. Using the NOT Operator**

**SQL>select ename, deptno FROM employee WHERE deptno NOT IN (10,9,5);**

```
SQL> select empid, ename, deptno from employee where deptno NOT IN (10,9,5);

EMPID      ENAME                DEPTNO
---------- -------------------- ----------
3          hina                 4
5          sadia                4
6          asma                 7

SQL>
```

**Rules of Precedence**

1 Arithmetic operators

2 Concatenation operator

3 Comparison conditions

4 IS [NOT] NULL , LIKE , [NOT] IN

5 [NOT] BETWEEN

6 NOT logical condition

7 AND logical condition

8 OR logical condition

**vii. Use parentheses to force priority.**

**SQL>select ename, salary, deptno FROM employee WHERE (deptno = 10 OR deptno=5)AND salary>= 1500;**

```
SQL> select ename, salary, deptno from employee where (deptno=10 OR deptno=5) AN
D salary>=1500;

ENAME                  SALARY DEPTNO
-------------------- ---------- ----------
shiza                   19000 10
maria                   16000 5

SQL>
```

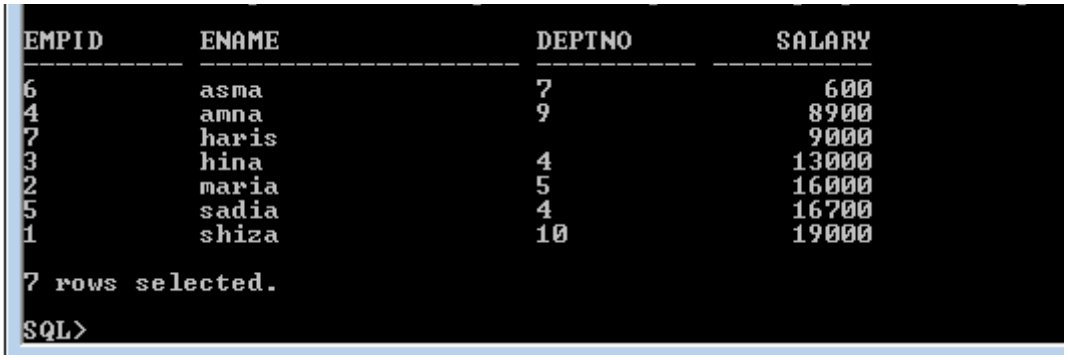
**viii. ORDER BY Clause**

Sort rows with the ORDER BY clause

ASC : ascending order (the default order) and DESC : descending order

The ORDER BY clause comes last in the SELECT statement.

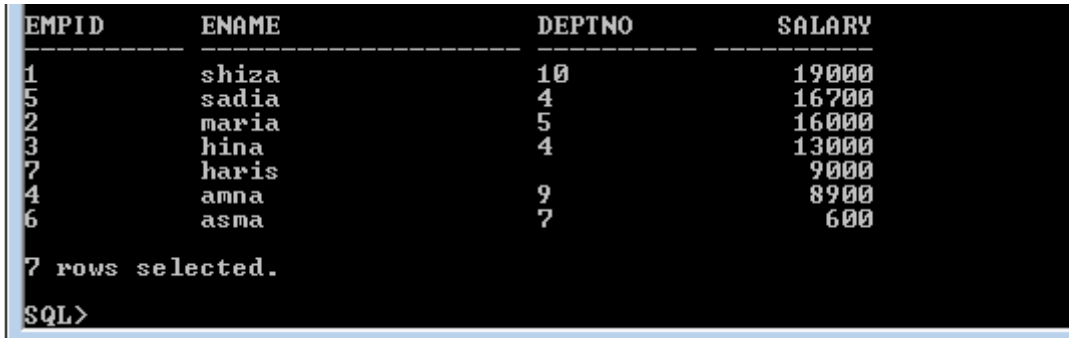
**SQL>select empid, ename, deptno, salary FROM employee ORDER BY salary;**



```
EMPID      ENAME                DEPTNO        SALARY
---------- -------------------- ---------- ----------
6          asma                 7                600
4          amna                 9               8900
7          haris                                9000
3          hina                 4               13000
2          maria                5               16000
5          sadia                4               16700
1          shiza                10              19000

7 rows selected.

SQL>
```

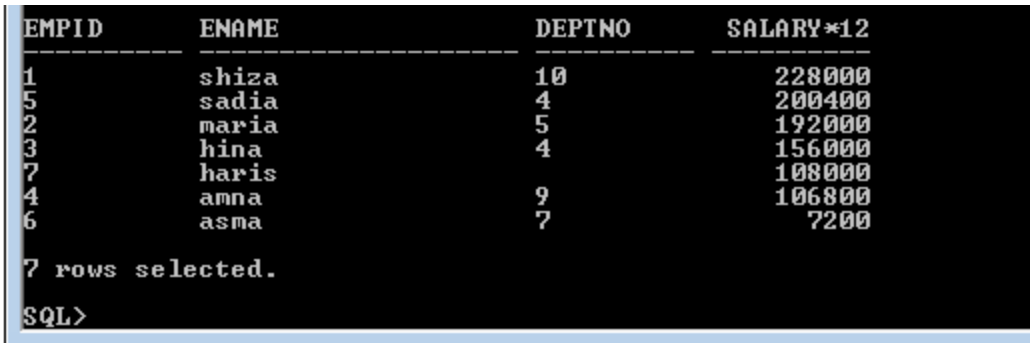
**For descending order**

**SQL>select empid, ename, deptno, salary FROM employee ORDER BY salary desc;**



```
EMPID      ENAME                DEPTNO        SALARY
---------- -------------------- ---------- ----------
1          shiza                10              19000
5          sadia                4               16700
2          maria                5               16000
3          hina                 4               13000
7          haris                                9000
4          amna                 9               8900
6          asma                 7                600

7 rows selected.

SQL>
```

**SQL>select empid, ename, deptno, salary*12 FROM employee ORDER BY salary desc;**



```
EMPID      ENAME                DEPTNO     SALARY*12
---------- -------------------- ---------- ----------
1          shiza                10             228000
5          sadia                4              200400
2          maria                5              192000
3          hina                 4              156000
7          haris                               108000
4          amna                 9              106800
6          asma                 7                7200

7 rows selected.

SQL>
```

### ix. Sorting by Multiple Columns

The order of ORDER BY list is the order of sort.

**SQL>select ename, deptno, salary FROM employee ORDER BY deptno, salary DESC;**

You can sort by a column that is not in the SELECT list.

**Lab Exercise:**

1. Create a query to display the name and salary of employees earning more than $4000.

2. Create a query to display the employee name and department number for employee number 7839.

3. Modify lab to display the name and salary for all employees whose salary is not in the range of $5,000 and $12,000.

4. Display the employee name, job , and hiredate of employees hired between February 20, 1998, and May 1, 1998. Order the query in ascending order by start date

.5. Display the name and department number of all employees in departments 20 and 30 in alphabetical order by name.

6. Modify lab to list the name and salary of employees who earn between $5,000 and $12,000, and are in department 20 or 50. Label the columns Employee and Monthly Salary , respectively.

7. Display the last name and hire date of every employee who was hired in 1994.

8. Display the last name and job title of all employees who do not have a manager.

9. Display the last name, salary, and commission for all employees who earn commissions. Sort data in descending order of salary and commissions.

10. Display the last names of all employees where the third letter of the name is an a.

11. Display the last name of all employees who have an a and an e in their name.

12. Display the last name, job, and salary for all employees whose job is salesman or clerk and whose salary is not equal to $2,500, $3,500, or $800.

**LAB No 4.**

**Objective: SQL Functions**

There are two types of SQL Functions.

i. Case Manipulation Functions
ii. Character Manipulation Functions

**A) CASE MANIPULATION FUNCTIONS**

1. **Upper Function :** If you want to view any data in upper case then ,

   **SQL> select upper(ename) from employee;**

```
SQL> select upper(ename) from employee;

UPPER(ENAME)
--------------------
SHIZA
HIRA
SANA

SQL>
```

2. **Lower Function:** If you want to view any data in lower case then,

   **SQL> select** lower(ename) from employee;

```
SQL> insert into employee values('&empid', '&ename', '&salary', '&deptno');
Enter value for empid: 4
Enter value for ename: HANIA
Enter value for salary: 80000
Enter value for deptno: csit
old   1: insert into employee values('&empid', '&ename', '&salary', '&deptno')
new   1: insert into employee values('4', 'HANIA', '80000', 'csit')

1 row created.
```

```
SQL> select lower(ename) from employee;

LOWER(ENAME)
--------------------
shiza
hira
sana
hania

SQL>
```

**B) CHARACTER MANIPULATION FUNCTION**

1. **SQL> select ename, lower(ename),length(ename), INSTR(ename,'s'),CONCAT(deptno,ename),RPAD(ename,7,'*') FROM employee WHERE upper(deptno) = 'se';**

```
SQL> select ename, lower(ename),length(ename), INSTR(ENAME,'s'), CONCAT(deptno,e
name),RPAD(ename,7,'*') from employee where lower(deptno)='se';

ENAME                LOWER(ENAME)         LENGTH(ENAME) INSTR(ENAME,'S')
-------------------- -------------------- ------------- ----------------
CONCAT(DEPTNO,ENAME)           RPAD(ENAME,7,'*')
------------------------------ ---------------------------
shiza                shiza                           5                1
seshiza                        shiza**

sana                 sana                            4                1
sesana                         sana***


SQL>
```

The columns being selected are:

**ename**: The original value of the "ename" column.

**lower(ename):** The lowercase version of the "ename" column.

**length(ename):** The length of the "ename" column.

**INSTR(ename, 's'):** The position of the letter 's' in the "ename" column.

**CONCAT(deptno,ename):** The concatenation of the "deptno" and "ename" columns.

**RPAD(ENAME, 7, '*'):** The "ename" column right-padded with '*' characters to a length of 7.

The table being queried is "employee".

The WHERE clause is filtering the results based on the condition LOWER(deptno) = 'se'. This condition checks if the lowercase version of the "deptno" column is equal to the string 'manager'. Only the rows that meet this condition will be returned.

In Oracle SQL*Plus, the INSTR function stands for "INSTRuction" and is used to find the position of a substring within a larger string. It returns the position of the first occurrence of the specified substring within the given string.

The syntax for instr is :

**INSTR(string, substring, [start_position], [nth_appearance])**

```
SQL> select ename, instr(deptno,'i', 3, 2) from employee where ename='sana';

ENAME                INSTR(DEPTNO,'I',3,2)
-------------------- ---------------------
sana                                    5

SQL> select ename, instr(deptno,'i',2, 1) from employee where ename='sana';

ENAME                INSTR(DEPTNO,'I',2,1)
-------------------- ---------------------
sana                                    3
```

However, the start position and nth appearance must be aligned otherwise the answers to query would be difficult to comprehend

```
SQL> select ename, instr(ename,'a',3, 2) from employee where deptno='csiti';

ENAME                INSTR(ENAME,'A',3,2)
-------------------- --------------------
sana                                    0

SQL> select ename, instr(ename,'a',3, 1) from employee where deptno='csiti';

ENAME                INSTR(ENAME,'A',3,1)
-------------------- --------------------
sana                                    4
```

Too many arguments may also exhaust the query such as

```
SQL> select ename, instr(ename,'a',1, 1,2) from employee where deptno='csiti';
select ename, instr(ename,'a',1, 1,2) from employee where deptno='csiti'
                                 *
ERROR at line 1:
ORA-00939: too many arguments for function
```

2. **Using LPAD and RPAD:**
   **SQL> select ename, lower(ename),length(ename), INSTR(ename,'s'),CONCAT(deptno,ename),LPAD(ename,7,'*') FROM employee WHERE upper(deptno) = 'se';**

```
SQL> select ename, lower(ename),length(ename), INSTR(ENAME,'s'), CONCAT(deptno,e
name), LPAD(ename,7,'*') from employee where lower(deptno)='se';

ENAME                LOWER(ENAME)         LENGTH(ENAME) INSTR(ENAME,'S')
-------------------- -------------------- ------------- -----------------
CONCAT(DEPTNO,ENAME)           LPAD(ENAME,7,'*')
------------------------------ ---------------------------
shiza                shiza                             5                 1
seshiza                        **shiza

sana                 sana                              4                 1
sesana                         ***sana


SQL>
```

3. **SUBSTR:**

The SUBSTR function is used to extract a substring from a larger string. It allows you to retrieve a portion of a string based on a specified starting position and length.

The syntax is

**SUBSTR(string, start_position, [length])**

**string** is the original string or column from which you want to extract the substring.

**start_position** is the position within the string where the extraction should begin. The first character is at position 1.

**length (optional)** specifies the number of characters to extract from the string. If not specified, SUBSTR will return all characters from the starting position to the end of the string.

```
SQL> select ename, deptno, substr(deptno,1,2) from employee;

ENAME                DEPTNO     SUBSTR(D
-------------------- ---------- --------
shiza                se         se
hira                 csit       cs
sana                 se         se
HANIA                csit       cs

SQL>
```

4. **Any string can also be subtracted:**

**SQL> select substr(deptno,1,2) from employee**;

```
SQL> select ename, substr(deptno,1,2) from employee;

ENAME                SUBSTR(D
-------------------- --------
shiza                se
sana                 cs

SQL> select ename, substr(deptno,2,1) from employee;

ENAME                SUBS
-------------------- ----
shiza                e
sana                 s
```

5. **Number Functions:**

- ROUND : Rounds value to specified decimal
- ROUND(45.926, 2) 45.93
- TRUNC : Truncates value to specified decimal
- TRUNC(45.926, 2) 45.92
- MOD : Returns remainder of division

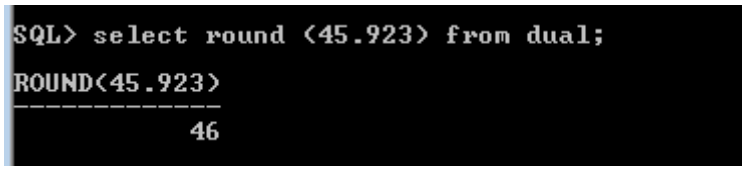
- MOD(1600, 300) 100

| ROUND(45.923,2) | ROUND(45.923,0) | ROUND(45.923,-1) |
|---|---|---|
| 45.92 | 46 | 50 |

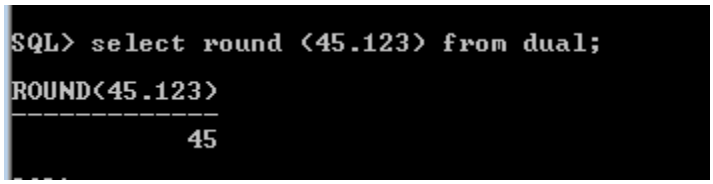
**Using the ROUND Function**

In Oracle SQL, the ROUND function is used to round a number to a specified decimal place or precision. It follows the standard rounding rules: if the decimal portion is 5 or greater, the number is rounded up; if it is less than 5, the number is rounded down.

**SQL> select round(45.923) from dual;**

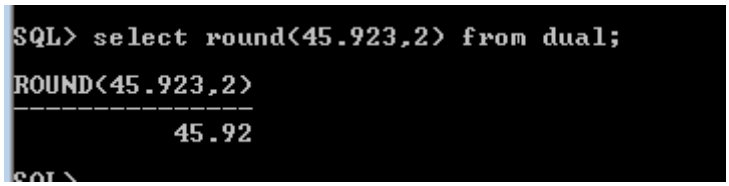



**SQL> select round(45.123) from dual;**

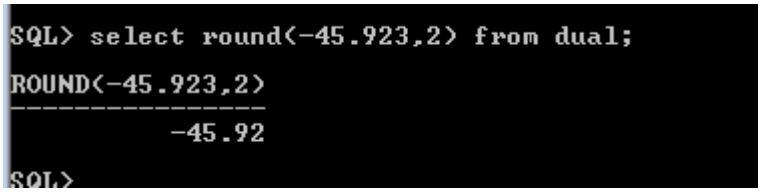



**You can also round off to a specified decimal number**

**SQL> select round(45.923,2) from dual;**

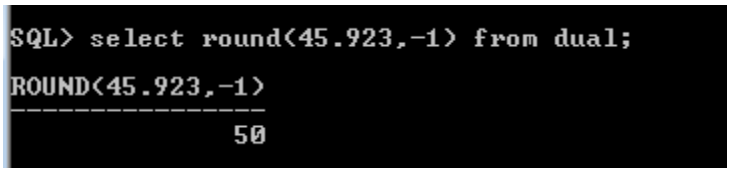



**We can also round off negative numbers**



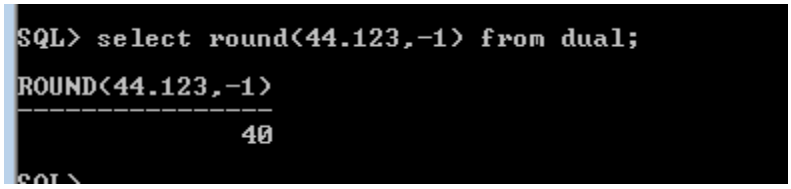
**Rounding off values with negative parameters**

**SQL> select round(45.923,-1) from dual;**

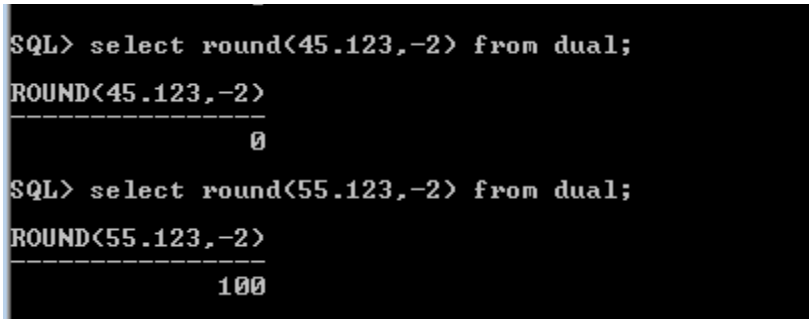



The output of the query SELECT ROUND(45.923, -1) FROM dual; is 50 because the -1 specified as the precision parameter in the ROUND function rounds the number to the nearest 10.

When the precision parameter is negative, the ROUND function rounds the number to the left of the decimal point. In this case, the number 45.923 is rounded to the nearest 10, resulting in 50.




**Rounding off to nearest 100**




**What is dual?**

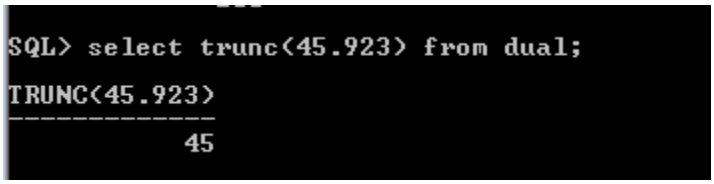
In Oracle, the DUAL table is a special one-row, one-column table that is automatically created with the Oracle database. It is commonly used in SQL statements where a table reference is required but the actual table doesn't matter.

The DUAL table is primarily used for performing calculations, testing expressions, or retrieving system information without referencing any specific table. It is often used as a placeholder table when you need to execute a query that doesn't involve any actual table data.

| TRUNC(45.923,2) | TRUNC(45.923) | TRUNC(45.923, 2) |
|---|---|---|
| 45.92 | 45 | 0 |

**Using the TRUNC Function**

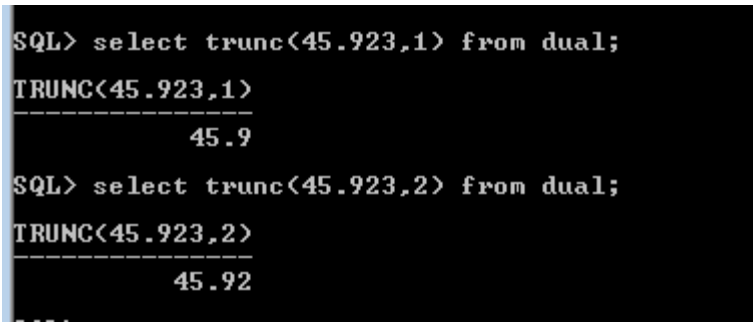
In Oracle SQL, you can use the TRUNC function to truncate values, which means removing the decimal portion of a number and returning the integer part. The TRUNC function can also be used to truncate dates to a specified level of precision.




Truncating a number to a specified decimal place;

**SQL> select trunc(45.923,1) from dual;**

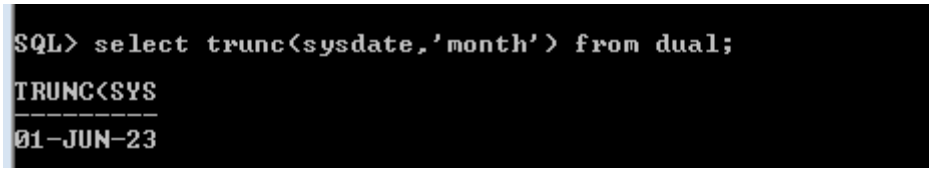
**SQL> select trunc(45.923,2) from dual;**




**Truncating a date to a specific precision:**

**SQL> select TRUNC(SYSDATE, 'MONTH') FROM dual;**

This will return the date with the time portion set to 00:00:00 and truncate it to the first day of the month.




You can use various precision values such as 'YEAR', 'MONTH', 'DAY', , etc., to truncate dates to different levels of precision.

**SQL> select TRUNC(SYSDATE, 'YEAR') FROM dual;**

**SQL> select TRUNC(SYSDATE, 'DAY') FROM dual;**

```
SQL> select trunc(sysdate,'year') from dual;

TRUNC(SYS
---------
01-JAN-23

SQL> select trunc(sysdate,'day') from dual;

TRUNC(SYS
---------
11-JUN-23
```

In Oracle SQL*Plus, the TRUNC function is primarily used to truncate the date portion of a value, rather than the time portion. It does not support truncating time components such as hours, minutes, or seconds directly.

**Using the MOD Function**

In Oracle SQL*Plus, you can use the MOD function to calculate the remainder of a division operation between two numbers. The MOD function takes two arguments: the dividend and the divisor.

**SQL> select MOD(10, 3) FROM dual;**

```
SQL> select mod(10,3) from dual;

 MOD(10,3)
----------
         1
```

You can also use variables or column values as inputs to the MOD function. Here's an example with a column value:

Calculate the remainder of a salary after it is divided by 500 for all employees whose job title is Manager.

**SQL> select MOD(salary, 500) FROM  employee;**

```
SQL> select mod(salary, 500) from employee;

MOD(SALARY,500)
---------------
              0
            400
```

**LAB No 5.**

**Objective: Working with DATE in SQL Plus**

**Theory:**

**Syntax of commands**

**SQL> select sysdate from dual;**

```
SQL> select sysdate from dual;

SYSDATE
---------
13-JUN-23
```

If you want to display both the date and time components using Oracle SQL*Plus, you can adjust the format of the output using the TO_CHAR function

This query uses the **TO_CHAR** function to convert the SYSDATE to a string format with the desired date and time representation.

In this case, the format mask 'YYYY-MM-DD HH24:MI:SS' is used to specify the format.

The output will include both the date and time components.

```
SQL> SELECT TO_CHAR(SYSDATE, 'YYYY-MM-DD HH24:MI:SS') FROM DUAL;

TO_CHAR(SYSDATE,'YY
-------------------
2023-06-13 02:32:33
```

Assume SYSDATE = '13-JUN-23':

**• ROUND(SYSDATE,'MONTH')**

This function rounds the current system date (SYSDATE) to the nearest month.

The result will be the first day of the following month.

For example, if the current date is '2023-06-13', the rounded result will be '2023-06-01'.

**• ROUND(SYSDATE ,'YEAR')**

This function rounds the current system date (SYSDATE) to the nearest year.

The result will be the first day of the following year.

For example, if the current date is '2023-06-13', the rounded result will be '2023-01-01'.

**• TRUNC(SYSDATE ,'MONTH')**

This function truncates the current system date (SYSDATE) to the beginning of the current month.

The result will be the first day of the current month.

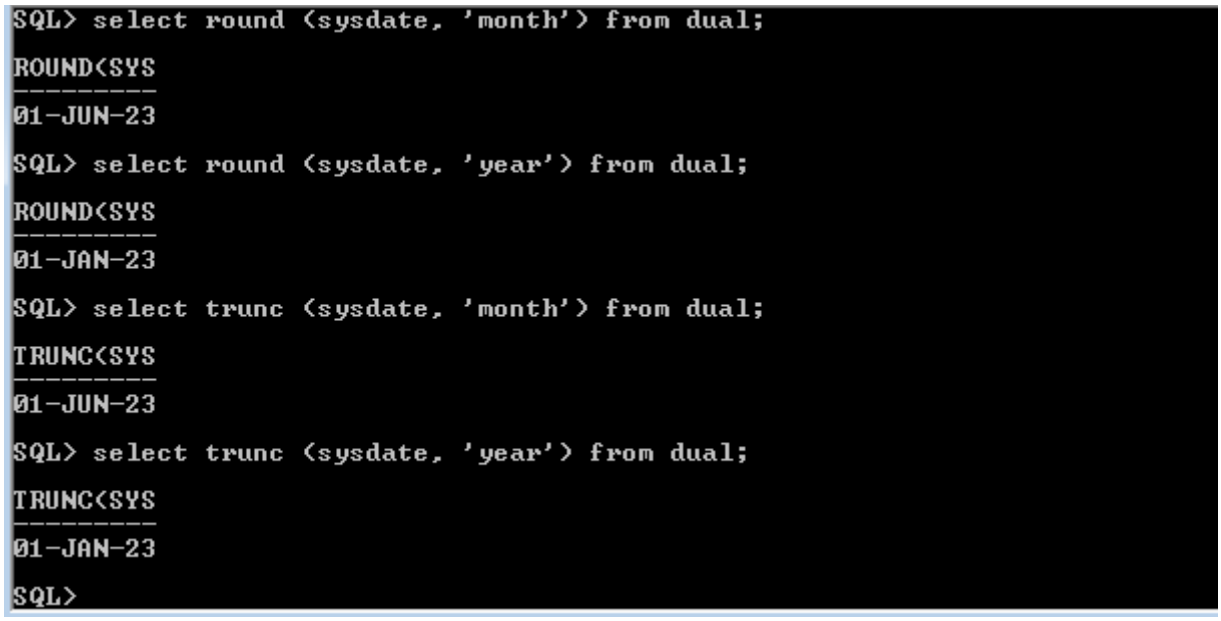
For example, if the current date is '2023-06-13', the truncated result will be '2023-06-01'.

**• TRUNC(SYSDATE ,'YEAR')**

This function truncates the current system date (SYSDATE) to the beginning of the current year.

The result will be the first day of the current year.

For example, if the current date is '2023-06-13', the truncated result will be '2023-01-01'.




- **Working with the dates in table employee**

We add another column of hire_date in table employee and add values to it;

```
SQL> alter table employee add hire_date date;

Table altered.

SQL> desc employee;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------

 EMPID                                     NOT NULL VARCHAR2(10)
 ENAME                                     NOT NULL VARCHAR2(20)
 SALARY                                    NOT NULL NUMBER(10)
 DEPTNO                                    NOT NULL VARCHAR2(10)
 HIRE_DATE                                          DATE
```

```
SQL> update employee set hire_date=TO_DATE('2023-06-13', 'YYYY-MM-DD') where emp
id=1;

1 row updated.

SQL> select * from employee;

EMPID      ENAME                SALARY DEPTNO     HIRE_DATE
---------- -------------------- ---------- ---------- ---------
1          shiza                  5000 se         13-JUN-23
2          sana                  70900 csiti

SQL> update employee set hire_date=TO_DATE('2023-06-13', 'YYYY-MM-DD') where emp
id=2;

1 row updated.
```

**Rounding and truncating the hire_date;**

```
SQL> select round(hire_date, 'month') from employee;

ROUND(HIR
---------
01-JUN-23
01-JUN-23

SQL> select round(hire_date, 'year') from employee;

ROUND(HIR
---------
01-JAN-23
01-JAN-23
```

**Arithmetic Operations with dates:**

**SQL> select ename, (SYSDATE-hire_date)/7 AS WEEKS FROM employee;**

```
SQL> select ename, (sysdate-hire_date)/7 as weeks from employee;

ENAME                    WEEKS
-------------------- ----------
shiza                .017291667
sana                 1.73157738

SQL>
```

The calculation (SYSDATE-hiredate)/7 computes the number of days between the hire date and the current date and then divides it by 7 to get the corresponding number of weeks. The result is displayed in the result set as the WEEKS column.

**Adding a number of days:**

**SQL> select hire_date+ 7 FROM employee;**

```
SQL> select hire_date+7 from employee;

HIRE_DATE
---------
20-JUN-23
08-JUN-23
```

**Subtracting a number of days:**

**SQL> select hire_date-6 FROM employee;**

```
SQL> select hire_date-6 from employee;

HIRE_DATE
---------
07-JUN-23
26-MAY-23
```

**Adding months to date;**

**SQL> select add_months(hire_date,6) FROM employee;**

```
SQL> select add_months(hire_date, 6) from employee;

ADD_MONTH
---------
13-DEC-23
01-DEC-23
```

**Subtracting a number of months:**

**SQL> select add_months(hire_date,-6) FROM employee;**

```
SQL> select add_months(hire_date,-6) from employee;

ADD_MONTH
---------
13-DEC-22
01-DEC-22
```

**Adding a number of years:**

**SQL> select add_months(hire_date,6*12) FROM employee;**

```
SQL> select add_months(hire_date,6*12) from employee;

ADD_MONTH
---------
13-JUN-29
01-JUN-29
```

**Subtracting a number of years:**

**SQL> select add_months(hire_date,-6*12) FROM employee;**

```
SQL> select add_months(hire_date,-6*12) from employee;

ADD_MONTH
---------
13-JUN-17
01-JUN-17
```

# LAB No 6.

**Objective: Conversion Functions**

**Theory:**

**What are Conversion Functions?**

Conversion functions are used in Oracle SQL*Plus to perform data type conversions. They allow you to convert data from one data type to another, facilitating compatibility between different data types in expressions, comparisons, and other operations.

A few reasons why conversion functions are commonly used:

**Data Type Compatibility**: Sometimes, data stored in the database is of a different data type than what you need for a particular operation. Conversion functions help you convert the data to the appropriate data type for performing calculations, comparisons, or formatting.

**Data Presentation**: Conversion functions are useful for presenting data in a specific format. For example, converting a date value to a specific date format or converting a numeric value to a string format for display purposes.

**Data Aggregation**: When performing calculations or aggregating data, it may be necessary to convert data types to ensure that the operations are performed correctly. For example, converting a string to a number before performing mathematical calculations or aggregating numeric values.

**Data Filtering and Searching**: Conversion functions can be used to convert data to a specific format for filtering or searching purposes. For example, converting a string to uppercase or lowercase before comparing it with other strings.

**Data Import/Export**: Conversion functions are often used when importing or exporting data to/from different systems or formats. They help ensure that the data is correctly transformed and mapped to the desired data types.

Overall, conversion functions provide flexibility in handling data of different types, enabling you to manipulate, present, and use the data effectively in your SQL queries and operations.

**Types of conversions:**

In Oracle SQL Plus, there are two types of data type conversions: implicit and explicit.

**Implicit Data Type Conversion**: Implicit data type conversion occurs automatically by Oracle without the need for explicit conversion functions or operators. It is based on predefined rules that

Oracle applies when encountering different data types in an expression or operation. Oracle performs implicit conversions to make the data types compatible for the operation. For example:

**Arithmetic Operations**: If you perform an arithmetic operation (such as addition, subtraction, multiplication, or division) between different data types, Oracle may implicitly convert one or both operands to a common data type before executing the operation.

**Comparison Operations**: When comparing values of different data types, Oracle may implicitly convert one or both values to a common data type to perform the comparison.

Implicit data type conversion can be convenient as it saves you from having to explicitly convert data types in many cases. However, it's important to be aware of potential issues related to precision, data loss, or unexpected behavior that can occur with implicit conversions.

**Example 1:**

CREATE TABLE employees ( emp_id NUMBER,emp_name VARCHAR2(50), emp_salary NUMBER);

INSERT INTO employees VALUES (1, 'John', 1000);

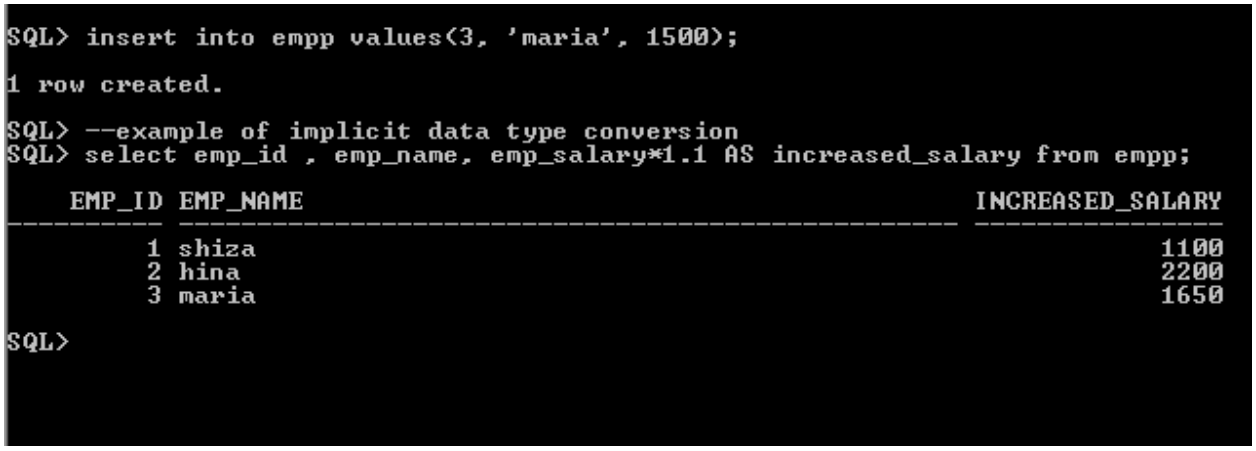
INSERT INTO employees VALUES (2, 'Jane', 2000);

INSERT INTO employees VALUES (3, 'Alice', 1500);

**Example of implicit data type conversion**

**SQL> select emp_id, emp_name, emp_salary * 1.1 AS increased_salary FROM employees;**

```
SQL> connect system
Enter password:
Connected.
SQL> create table empp(emp_id number, emp_name varchar2(50), emp_salary number)


Table created.

SQL> insert into empp values(1, 'shiza', 1000);

1 row created.

SQL> insert into empp values(2, 'hina', 2000);
```




In this example, we have a table called "empp" with three columns: emp_id (NUMBER), emp_name (VARCHAR2), and emp_salary (NUMBER). We insert some sample data into the table.

The SELECT statement then retrieves data from the table. Here, we perform an arithmetic operation by multiplying the emp_salary column by 1.1. Since the multiplication involves a NUMBER data type (emp_salary) and a decimal literal (1.1), Oracle implicitly converts the literal to the NUMBER data type to perform the calculation.

The result of the implicit conversion is a new column called "increased_salary" that shows the original emp_salary multiplied by 1.1 for each employee.

When executing the SQL statement in Oracle SQL*Plus, you will see the output with the implicit conversion applied to calculate the increased salary.

In the given example, the emp_salary column would not undergo any implicit conversion since it is already defined as the NUMBER data type. Only the literal value would undergo implicit conversion if necessary.

**Another Example**

CREATE TABLE employees ( emp_id NUMBER, emp_name VARCHAR2(50), emp_salary VARCHAR2(10));

INSERT INTO employees VALUES (1, 'John', '1000');

INSERT INTO employees VALUES (2, 'Jane', '2000');

INSERT INTO employees VALUES (3, 'Alice', '1500');

**Example of implicit data type conversion**

**SQL> select emp_id, emp_name, emp_salary * 1.1 AS increased_salaryFROM employees;**

**You can also alter the existing emp_salary table**

**SQL> alter table empp modify emp_salary varchar2(10);**

However to modify, you must delete all entries of this column.

```
SQL> update empp set emp_salary= NULL where emp_name= 'shiza';
1 row updated.
SQL> update empp set emp_salary= NULL where emp_name= 'hina';
1 row updated.
SQL> update empp set emp_salary= NULL where emp_name= 'maria';
1 row updated.
SQL> alter table empp modify emp_salary varchar2(10);
Table altered.
SQL>
```

**Now add values to this column**

```
1 row updated.
SQL> update empp set emp_salary= NULL where emp_name= 'maria';
1 row updated.
SQL> alter table empp modify emp_salary varchar2(10);
Table altered.
SQL> update empp set emp_salary= '1000' where emp_name='shiza';
1 row updated.
SQL> update empp set emp_salary= '2000' where emp_name='hina';
1 row updated.
SQL> update empp set emp_salary= '1500' where emp_name='maria';
1 row updated.
SQL> select * from empp;
```

```
SQL> select emp_id, emp_name, emp_salary * 1.1 AS increased_salary from empp;

    EMP_ID EMP_NAME                                           INCREASED_SALARY
---------- -------------------------------------------------- ----------------
         1 shiza                                                          1100
         2 hina                                                           2200
         3 maria                                                          1650

SQL>
```

In this revised example, the emp_salary column is defined as VARCHAR2(10) instead of NUMBER. We insert the emp_salary values as strings.

When performing the arithmetic operation (emp_salary * 1.1), Oracle will implicitly convert the emp_salary values from VARCHAR2 to NUMBER to execute the calculation.

**Explicit Data Type Conversion:**

Explicit data type conversion is performed using conversion functions or operators explicitly defined in the SQL statement. It allows you to explicitly convert a value from one data type to another. Examples of explicit conversion functions include TO_CHAR, TO_NUMBER, TO_DATE, and CAST, as mentioned in the previous response. By using these functions, you have control over how the conversion is performed and can specify the desired data type and format.

Explicit conversions are useful when you want to ensure a specific data type for an operation, apply a specific formatting, or handle data in a customized way that differs from the default behavior of implicit conversions.

It's generally recommended to use explicit data type conversions when you want to ensure clarity, precision, and consistency in your SQL statements, as they make the conversions explicit and easier to understand for other developers and maintainers of the code.

Here are some commonly used conversion functions in Oracle SQL*Plus:

**TO_CHAR**: This function is used to convert a value of any datatype (such as a number or date) to a string in a specified format.

The syntax is **TO_CHAR(value, format)**

The value parameter represents the value you want to convert to a string, and the format parameter specifies the format for the output string.

**Usage**: You can use the TO_CHAR function in Oracle SQL*Plus to convert various data types to strings with specific formats. Some common use cases include:

- Converting dates to strings with desired date formats.
- Formatting numbers as strings with specific decimal places or thousands separators.

**Example:** Here's an example of using the TO_CHAR function in Oracle SQL*Plus:

**Converting a date to a string with a specific format**

**SQL> select TO_CHAR(sysdate, 'DD-MON-YYYY HH:MI:SS') FROM dual;**

```
SQL> select TO_CHAR(sysdate, 'DD-MON-YYYY HH:MI:SS') from dual;

TO_CHAR(SYSDATE,'DD-MON-YYYYH
-----------------------------
04-JUL-2023 02:02:03

SQL> select TO_CHAR(sysdate, 'DD-MON-YYYY HH:MI') from dual;

TO_CHAR(SYSDATE,'DD-MON-YY
--------------------------
04-JUL-2023 02:02

SQL> select TO_CHAR(sysdate, 'DD-MON-YY HH:MI') from dual;

TO_CHAR(SYSDATE,'DD-MON-
------------------------
04-JUL-23 02:02
```

You can use different formats and visualize the output.

```
SQL> select TO_CHAR(sysdate, 'DD-MO-YY HH:MI') from dual;
select TO_CHAR(sysdate, 'DD-MO-YY HH:MI') from dual
                        *
ERROR at line 1:
ORA-01821: date format not recognized


SQL> select TO_CHAR(sysdate, 'D-MO-YY HH:MI') from dual;
select TO_CHAR(sysdate, 'D-MO-YY HH:MI') from dual
                        *
ERROR at line 1:
ORA-01821: date format not recognized


SQL> select TO_CHAR(sysdate, 'D-MO-YYY HH:MI') from dual;
select TO_CHAR(sysdate, 'D-MO-YYY HH:MI') from dual
                        *
ERROR at line 1:
ORA-01821: date format not recognized
```

In Oracle SQL*Plus, you can use the TO_CHAR function to format numbers as strings with specific decimal places or thousands separators. The formatting is achieved by using number format models in the format parameter of the TO_CHAR function.

**Formatting a number with decimal places and thousands separator**

SQL> select TO_CHAR(12345.6789, '9,999.99') FROM dual;

The output will be

```
12,345.68
```

In this example, the number 12345.6789 is formatted as a string with two decimal places and a comma as the thousands separator. The result would be '12,345.68'.

The format model used in the TO_CHAR function can be elaborated as:

**9: Represents a digit position**. If the corresponding digit exists, it will be displayed. If not, it will be replaced with a space.

**,: Represents the thousands separator**. It inserts a comma at the appropriate place to separate thousands.

**.: Represents the decimal point.**

**99: Represents two decimal places.** If there are more decimal places in the number, they will be rounded.

You can modify the format model to suit your specific formatting requirements. For example, if you want to display four decimal places, you can use '9,999.9999'.

Note that the TO_CHAR function can be used with various numeric data types, such as NUMBER, FLOAT, and INTEGER. The function will convert the numeric value to a formatted string according to the specified format model.

By utilizing different format models and customizing the format parameter, you can achieve specific formatting for numbers in Oracle SQL*Plus.

**TO_NUMBER:**

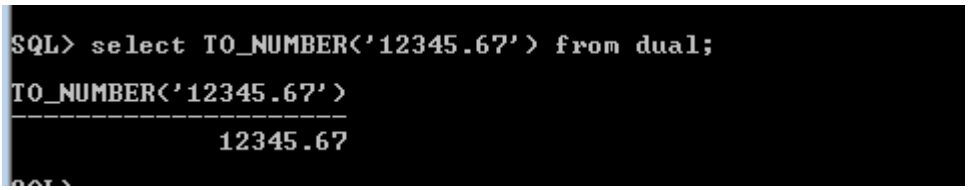
In Oracle SQL*Plus, the TO_NUMBER function is used to convert a character string or an expression to a numeric value. It is typically used when you want to perform calculations or comparisons involving numeric values stored as strings.

The TO_NUMBER function has the following syntax:

TO_NUMBER(string, [format])

**Converting a string to a number**

**SQL> select TO_NUMBER('12345.67') FROM dual;**




In this example, the string '12345.67' is converted to a numeric value using the TO_NUMBER function. The result will be the number 12345.67.

**TO_DATE:**

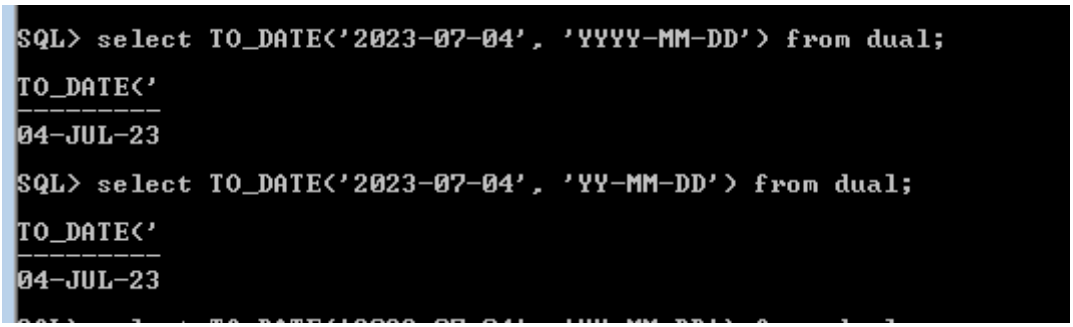
In Oracle SQL*Plus, the TO_DATE function is used to convert a character string to a date data type. It is particularly useful when you need to work with date values stored as strings or when you want to perform date-related operations and comparisons.

The TO_DATE function has the following syntax:

TO_DATE(string, [format])

**Converting a string to a date**

**SQL> select TO_DATE('2023-07-04', 'YYYY-MM-DD') FROM dual;**




In this example, the string '2023-07-04' is converted to a date value using the TO_DATE function. The result will be the date value representing July 4, 2023.

**TTO_TIMESTAMP:**

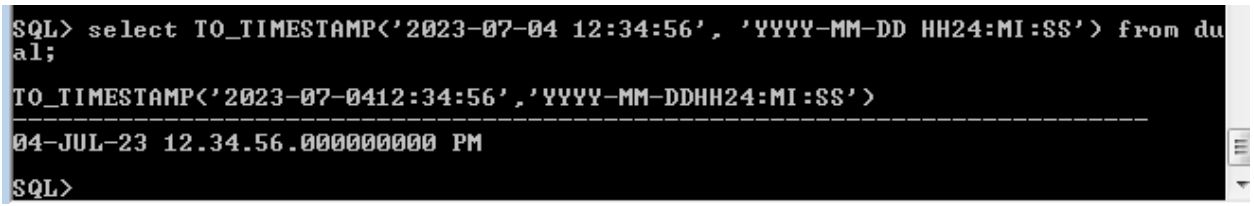
In Oracle SQL*Plus, the TO_TIMESTAMP function is used to convert a character string to a timestamp data type. It allows you to convert date and time information stored as strings into the more precise timestamp data type.

The TO_TIMESTAMP function has the following syntax:

**TO_TIMESTAMP(string, [format])**

**Converting a string to a timestamp**

**SQL> select TO_TIMESTAMP('2023-07-04 12:34:56', 'YYYY-MM-DD HH24:MI:SS') FROM dual;**



In this example, the string '2023-07-04 12:34:56' is converted to a timestamp value using the TO_TIMESTAMP function. The result will be the timestamp value representing July 4, 2023, at 12:34:56.

**CAST:**

In Oracle SQL*Plus, the CAST function is used to convert a value from one data type to another. It allows you to explicitly specify the desired target data type for a value.

The CAST function has the following syntax:

CAST(value AS data_type)

**Casting a value to a different data type**

**SQL> select CAST('123' AS NUMBER) FROM dual;**

```
SQL> select CAST ('123' AS NUMBER) FROM DUAL;

CAST('123'ASNUMBER)
-------------------
                123
```

**LAB No 7.**

**Objective: Nesting Functions**

**Theory:**

**What are Nesting Functions?**

• Single-row functions can be nested to any level.

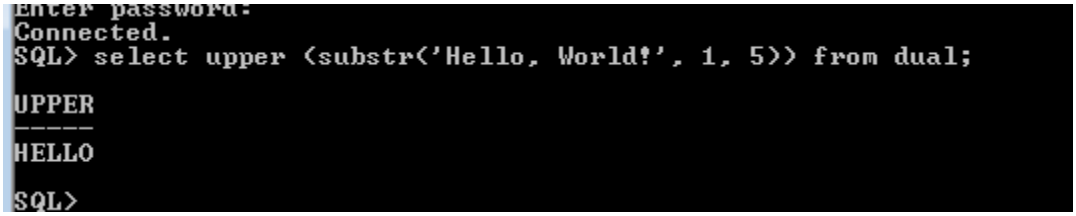
• Nested functions are evaluated from deepest level to the least deep level.

Function3(Function2(Function1(column,argument1),argument2),argument3)

In Oracle SQL*Plus, you can use nested functions to perform complex operations by combining multiple functions within a single query. Nesting functions allows you to manipulate and transform data in a more concise and efficient manner.

**Example of nesting functions**

**SQL> select UPPER(SUBSTR('Hello, World!', 1, 5)) FROM dual;**

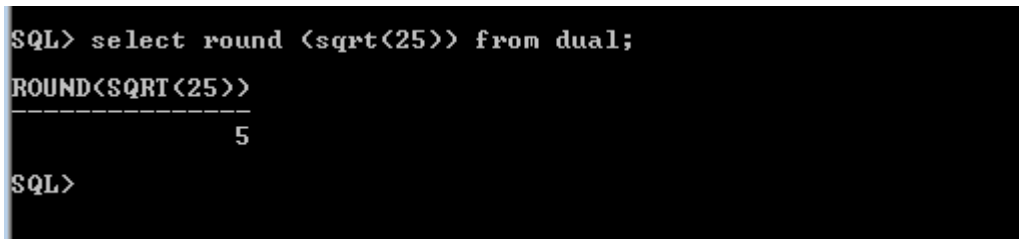



In this example, the UPPER and SUBSTR functions are nested. The SUBSTR function is used to extract the substring 'Hello' from the string 'Hello, World!', and then the UPPER function is applied to convert that substring to uppercase. The result will be 'HELLO'.

You can nest functions as needed to achieve the desired data manipulation or transformation. Here's another example:

**Nested functions to calculate the square of a number and round it to the nearest integer**

**SQL> select ROUND(SQRT(25)) FROM dual;**



In this example, the SQRT function is used to calculate the square root of 25, and then the ROUND function is applied to round that result to the nearest integer. The final result will be the number 5.

You can nest functions to multiple levels based on your requirements. However, it's important to ensure that the nested functions are used correctly and that the output of one function matches the input requirements of the next function.

**NVL Functions:**

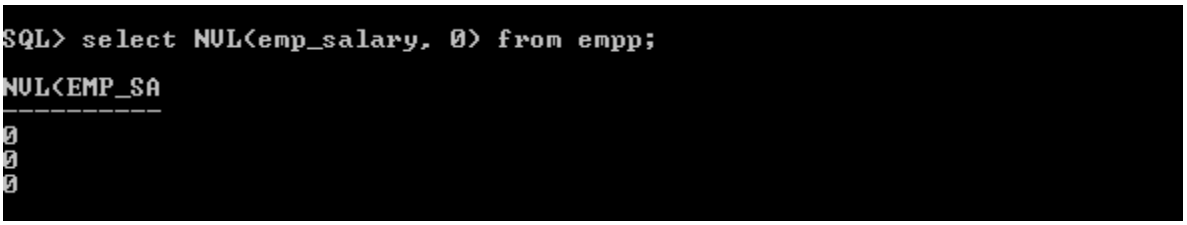
In Oracle SQL*Plus, the NVL function is used to handle NULL values in expressions or columns. It allows you to replace a NULL value with an alternate value of your choice. The NVL function takes two arguments: the expression or column to evaluate and the substitute value to use if the expression or column is NULL.

The syntax of the NVL function is as follows:

**NVL(expression, substitute_value**)

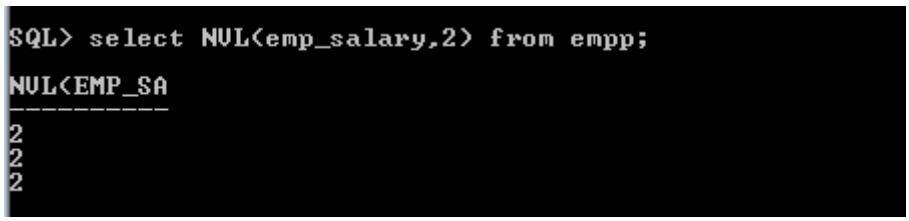
**Using NVL to handle NULL values**

**SQL> select NVL(emp_salary, 0) FROM employees;**




In this example, the NVL function is applied to the salary column in the employees table. If the salary value is NULL, it will be replaced with 0. This ensures that the result of the query always contains a non-NULL value for the salary.

The substitute_value can be any valid expression or constant value. It can even be another column or the result of another function. The data types of the expression and the substitute value should be compatible. If they are not compatible, an error may occur.




**Using NVL with a substitute value from another column**

**SQL> select NVL(emp_salary, emp_name) FROM employees;**

```
SQL> select NVL(emp_salary, emp_name) from empp;

NVL(EMP_SALARY,EMP_NAME)
--------------------------------------------------
shiza
hina
maria

SQL>
```

In this case, if the salary value is NULL, it will be replaced with the value from the emp_name column. This allows you to use a different value from the same row as the substitute.

The NVL function is useful in scenarios where you need to handle NULL values and ensure consistent and meaningful results in your queries. It provides a way to handle the absence of a value and replace it with a suitable substitute in your SQL expressions.

**NVL2 Function:**

In Oracle SQL*Plus, the NVL2 function is similar to the NVL function, but it provides an additional feature. The NVL2 function allows you to specify two different substitute values based on whether an expression or column is NULL or not.

The syntax of the NVL2 function is as follows:

**NVL2(expression, value_if_not_null, value_if_null)**

**Using NVL2 to handle NULL values with different substitute values**

**SQL> select NVL2(emp_salary, 'Salary is not null', 'Salary is null') FROM empp;**

```
SQL> select NVL2(emp_salary, 'salary is not null', 'salary is null') from empp;

NVL2(EMP_SALARY,'S
------------------
salary is null
salary is null
salary is null

SQL>
```

In this example, the NVL2 function is applied to the salary column in the empp table. If the salary value is not NULL, it will return the string 'Salary is not null'. If the salary value is NULL, it will return the string 'Salary is null'.

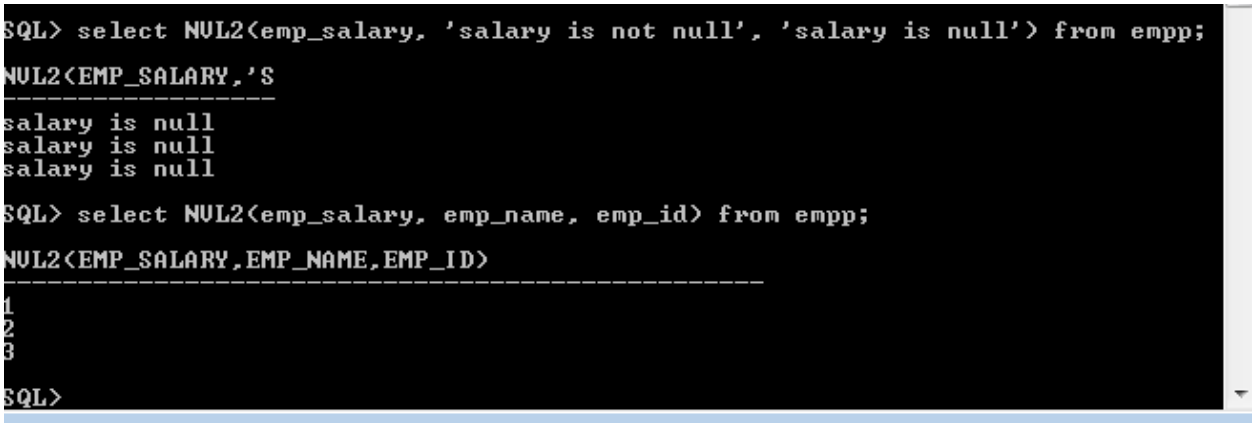
The first argument, expression, is the column or expression you want to evaluate for NULL. The second argument, value_if_not_null, is the substitute value to use if the expression is not NULL. The third argument, value_if_null, is the substitute value to use if the expression is NULL.

**Using NVL2 with different substitute values from other columns**

**SQL> select NVL2(salary, bonus, commission) FROM employees;**

In this case, if the salary value is not NULL, it will return the value from the emp_name column. If the salary value is NULL, it will return the value from the emp_id column.

The NVL2 function is useful when you need to handle NULL values and provide different substitute values based on whether the expression is NULL or not. It gives you more flexibility in handling NULL scenarios compared to the NVL function, where you can only provide one substitute value.




**DECODE FUNCTION:**

In Oracle SQL*Plus, the DECODE function is used to conditionally return a value based on the comparison of an expression with a series of values. It is commonly used as a shorthand for writing complex CASE statements.

The syntax of the DECODE function is as follows:

**DECODE(expression, search_value1, result_value1, search_value2, result_value2, ..., default_value)**

**Using DECODE to conditionally return values**

**SQL> select emp_name, DECODE(job_id, 'IT_PROG', 'IT Department', 'SA_MAN', 'Sales Department', 'Other Department') AS department FROM employees;**

In this example, the DECODE function is applied to the job_id column in the employees table. It compares the job_id value with a series of search values and returns the corresponding result value when a match is found. If none of the search values match, the default value (which is not mandatory) is returned. The result is aliased as "department" in the query.

The DECODE function evaluates the expression and compares it with the search_value arguments sequentially. When a match is found, the corresponding result_value is returned. If there is no match, the default value (optional) is returned.

**Using DECODE with a default value**

**SQL> select employee_name, DECODE(department_id, 10, 'Finance', 20, 'Marketing', 'Other Department') AS department FROM employees;**

In this case, if the department_id value is 10, it returns 'Finance'. If the department_id value is 20, it returns 'Marketing'. For any other department_id value, it returns 'Other Department'.

The DECODE function can be a convenient way to simplify conditional logic in SQL queries, especially when there are multiple comparisons and result values involved. However, it's important to note that the DECODE function is specific to Oracle and may not be supported in other database systems.

**LAB No 8.**

**Objective: SQL Joins**

**Theory:**

**What are Joins?**

In SQL Plus, a "join" is an operation that combines rows from two or more tables based on a related column between them. It allows you to retrieve data from multiple tables in a single query, creating a new result set that contains columns from both tables. Joins are essential for retrieving and presenting data when the required information is distributed across different tables in a relational database.

**Why do we use joins?**

Conditions to use join

1. Minimum 2 tables
2. Both tables should have one similar column with name and records same

**Types of Joins:**

There are 7 types of join

1. **Equijoin:**

In Oracle SQL, you can perform an equijoin by using the JOIN keyword and specifying the join condition in the ON clause

**SQL> select column1, column2 FROM table1 JOIN table2, ON table1.column_name = table2.column_name;**

Column1, column2, represents the columns you want to select from the tables.
Table1 and table2 are the names of the tables you want to join.
Column_name represents the common column(s) between table1 and table2 that you want to use for the equijoin.

```
SQL> CREATE TABLE customer (cId number(5), cName varchar(15) not null, city varchar(20) not null);

Table created.

SQL> CREATE TABLE orders (oId number(5), pName varchar(10), price number(10), cId number(5));

Table created.
```

```
SQL> desc customer;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 CID                                                NUMBER(5)
 CNAME                                     NOT NULL VARCHAR2(15)
 CITY                                      NOT NULL VARCHAR2(20)

SQL> desc orders;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 OID                                                NUMBER(5)
 PNAME                                              VARCHAR2(10)
 PRICE                                              NUMBER(10)
 CID                                                NUMBER(5)

SQL> SELECT * FROM customer;

       CID CNAME           CITY
---------- --------------- --------------------
         1 Ahmed1          Karachi
         2 Ahmed2          Karachi
         3 Ahmed3          Lahore
         4 Ahmed4          Islamabad
         5 Ahmed5          Faisalabad

SQL> SELECT * FROM orders;

       OID PNAME           PRICE        CID
---------- ---------- ---------- ----------
         1 Shirt            1025          1
         2 Shirt            1225          1
         3 Jacket           2000          1
         4 Jacket2          4200          2
         5 Shoes             700          3
         6 Hoodie           7330          4
         7 Cap              9030          5
         7 Sweater          9000          6
```

**Joining Tables with WHERE Clause**

```
SQL> SELECT * FROM customer, orders WHERE customer.cid = orders.cid;

       CID CNAME           CITY                      OID PNAME           PRICE
---------- --------------- ------------------- ---------- ---------- ----------
       CID
----------
         1 Ahmed1          Karachi                     1 Shirt            1025
         1

         1 Ahmed1          Karachi                     2 Shirt            1225
         1

         1 Ahmed1          Karachi                     3 Jacket           2000
         1


       CID CNAME           CITY                      OID PNAME           PRICE
---------- --------------- ------------------- ---------- ---------- ----------
       CID
----------
         2 Ahmed2          Karachi                     4 Jacket2          4200
         2

         3 Ahmed3          Lahore                      5 Shoes             700
         3

         4 Ahmed4          Islamabad                   6 Hoodie           7330
         4


       CID CNAME           CITY                      OID PNAME           PRICE
---------- --------------- ------------------- ---------- ---------- ----------
       CID
----------
         5 Ahmed5          Faisalabad                  7 Cap              9030
         5

7 rows selected.
```

**Joining tables using JOIN Keyword**

```
SQL> SELECT customer.cId, cName FROM customer JOIN orders ON customer.cID = orders.cId;

       CID CNAME
---------- ---------------
         1 Ahmed1
         1 Ahmed1
         1 Ahmed1
         2 Ahmed2
         3 Ahmed3
         4 Ahmed4
         5 Ahmed5

7 rows selected.
```

There are other types of joins available in Oracle SQL Plus, such as inner join, outer join, and cross join, which have different syntax and purposes. The equijoin is a specific type of inner join where the join condition is based on equality between columns

2. **Non equi join:**
   In SQLPlus, you can perform a non-equijoin by using the traditional SQL syntax with the JOIN keyword and specifying the non-equijoin condition in the ON clause or the WHERE clause.

   **SQL> select column1, column2, FROM table1 JOIN table2 ON table1.column_name < table2.column_name;**

The non-equijoin condition (table1.column_name < table2.column_name) is specified in the ON clause. You can modify the non-equijoin condition in the ON clause as per your requirements.

```
SQL> SELECT customer.cId, cName FROM customer JOIN orders ON customer.cId < orders.cId;

       CID CNAME
---------- ---------------
         1 Ahmed1
         1 Ahmed1
         1 Ahmed1
         1 Ahmed1
         1 Ahmed1
         2 Ahmed2
         2 Ahmed2
         2 Ahmed2
         2 Ahmed2
         3 Ahmed3
         3 Ahmed3

       CID CNAME
---------- ---------------
         3 Ahmed3
         4 Ahmed4
         4 Ahmed4
         5 Ahmed5

15 rows selected.
```

```
SQL> SELECT customer.cId, cName FROM customer JOIN orders ON customer.cId > orders.cId;

       CID CNAME
---------- ---------------
         2 Ahmed2
         2 Ahmed2
         2 Ahmed2
         3 Ahmed3
         3 Ahmed3
         3 Ahmed3
         3 Ahmed3
         4 Ahmed4
         4 Ahmed4
         4 Ahmed4
         4 Ahmed4

       CID CNAME
---------- ---------------
         4 Ahmed4
         5 Ahmed5
         5 Ahmed5
         5 Ahmed5
         5 Ahmed5
         5 Ahmed5
         5 Ahmed5

18 rows selected.
```

3. **Cross join**

In SQLPlus, a cross join, also known as a Cartesian join, is a join operation that produces the combination of every row from two or more tables. It does not require any join condition.

**SQL> select * FROM table1 CROSS JOIN table2;**

In the above syntax, table1 and table2 represent the names of the tables you want to cross join. The * in the SELECT statement retrieves all columns from both tables.

The cross join simply executes a Cartesian product between table1 and table2, combining every row from table1 with every row from table2. The result set will contain all possible combinations of rows from both tables.

4. **Natural join**

In SQLPlus, a natural join is a join operation that combines two or more tables based on the columns that have the same name and the same data type. The join condition is implicitly determined by the matching column names.

**SQL> select * FROM table1 NATURAL JOIN table2;**

In the above syntax, table1 and table2 represent the names of the tables you want to perform the natural join on. The * in the SELECT statement retrieves all columns from the joined tables.

The query performs a natural join between table1 and table2 based on the columns with the same name and data type. The result set will include only the rows where the values in the matching columns of both tables are equal

```
SQL> SELECT cId, cName FROM customer NATURAL JOIN orders;

       CID CNAME
---------- ---------------
         1 Ahmed1
         1 Ahmed1
         1 Ahmed1
         2 Ahmed2
         3 Ahmed3
         4 Ahmed4
         5 Ahmed5

7 rows selected.
```

5. **Inner join**

In SQL Plus, an inner join is a type of join operation that combines rows from two or more tables based on a specified join condition. It returns only the matching rows between the tables involved in the join. Here's an example of how an inner join works in SQLPlus:

**SQL> select * FROM table1 INNER JOIN table2 ON table1.column_name = table2.column_name**;

In the above query, table1 and table2 represent the names of the tables you want to join. The * in the SELECT statement retrieves all columns from both tables.
This query simply executes an inner join between table1 and table2, combining every row from table1 with every row from table2. The result set will contain all possible combinations of rows from both tables. The equijoin and inner join can be used simultaneously.

```
SQL> SELECT customer.cId, cName FROM customer INNER JOIN orders ON customer.cID = orders.cId;

       CID CNAME
---------- ---------------
         1 Ahmed1
         1 Ahmed1
         1 Ahmed1
         2 Ahmed2
         3 Ahmed3
         4 Ahmed4
         5 Ahmed5

7 rows selected.
```

6. **Outer join**

In SQL*Plus, an outer join is a type of join operation that combines rows from two or more tables based on a specified join condition, while also including unmatched rows from one or both of the tables. It is useful when you want to retrieve all the rows from one table and matching rows from the other table(s), including null values for unmatched rows. There are three types of outer joins: left outer join, right outer join, and full outer join.

- **Left outer join**

A left outer join is a type of join operation that combines rows from two or more tables based on a specified join condition, while also including all the unmatched rows from the left (or "leftmost") table in the result set. It allows you to retrieve all rows from the left table and matching rows from the right table(s), and if there is no match in the right table(s), the corresponding columns will contain NULL values in the result set.
In an SQL query, a left outer join is typically expressed using the LEFT OUTER JOIN or LEFT JOIN keywords.

**SQL> select * FROM table1 LEFT OUTER JOIN table2 ON table1.column_name = table2.column_name;**

In the above query, table1 and table2 represent the names of the tables you want to perform the left outer join on. Column_name represents the common column between the two tables that you want to use for the join.
The LEFT OUTER JOIN keyword specifies the type of join operation, where all rows from table1 will be included in the result set, regardless of whether there is a match in table2. The join condition is specified using the ON keyword.

The result set will contain all rows from table1, with the corresponding matching rows from table2. If there is no match in table2 for a row in table1, the columns from table2 will contain NULL values in the result set.
LEFT OUTER JOIN and LEFT JOIN are interchangeable and can be used interchangeably in most database systems.

```
SQL> SELECT customer.cId, cName FROM customer LEFT OUTER JOIN orders ON customer.cID = orders.cId;

       CID CNAME
---------- ---------------
         1 Ahmed1
         1 Ahmed1
         1 Ahmed1
         2 Ahmed2
         3 Ahmed3
         4 Ahmed4
         5 Ahmed5

7 rows selected.
```

- **Right outer join**

A right outer join is a type of join operation that combines rows from two or more tables based on a specified join condition, while also including all the unmatched rows from the right (or ”rightmost”) table in the result set. It allows you to retrieve all rows from the right table and matching rows from the left table(s), and if there is no match in the left table(s), the corresponding columns will contain NULL values in the result set.

In an SQL query, a right outer join is typically expressed using the RIGHT OUTER JOIN or RIGHT JOIN keywords. Here's an example of how a right outer join works:

**SQL> select * FROM table1 RIGHT OUTER JOIN table2 ON table1.column_name = table2.column_name;**

In the above query, table1 and table2 represent the names of the tables you want to perform the right outer join on. Column_name represents the common column between the two tables that you want to use for the join.
The RIGHT OUTER JOIN keyword specifies the type of join operation, where all rows from table2 will be included in the result set, regardless of whether there is a match in table1. The join condition is specified using the ON keyword.
The result set will contain all rows from table2, with the corresponding matching rows from table1. If there is no match in table1 for a row in table2, the columns from table1 will contain NULL values in the result set.
RIGHT OUTER JOIN and RIGHT JOIN are interchangeable and can be used interchangeably in most database systems.

```
SQL> SELECT customer.cId, cName FROM customer RIGHT OUTER JOIN orders ON customer.cID = orders.cId;

       CID CNAME
---------- --------------
         1 Ahmed1
         1 Ahmed1
         1 Ahmed1
         2 Ahmed2
         3 Ahmed3
         4 Ahmed4
         5 Ahmed5


8 rows selected.
```

- **Full outer join:**

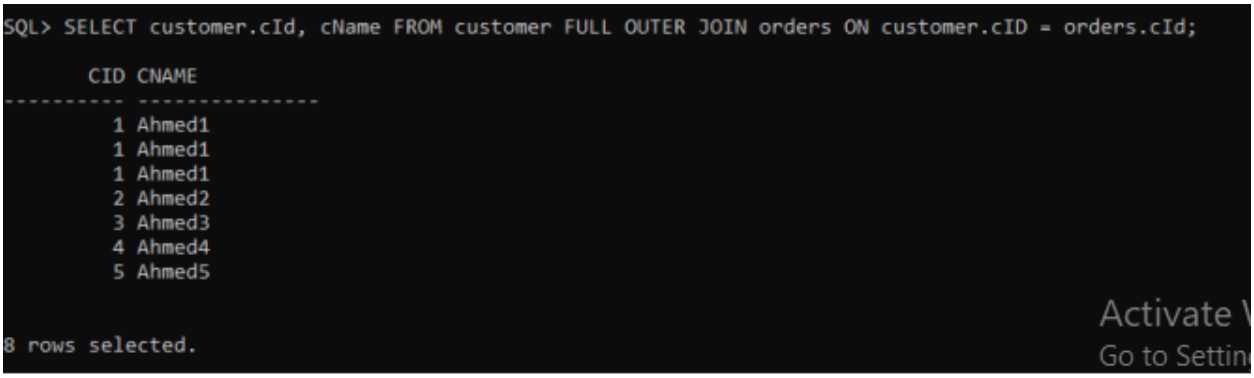
A full outer join is a type of join operation that combines rows from two or more tables based on a specified join condition, while also including all the unmatched rows from both the left and right tables in the result set. It allows you to retrieve all rows from both tables, and if there is no match in either table, the corresponding columns will contain NULL values in the result set.

In an SQL query, a full outer join is typically expressed using the FULL OUTER JOIN keyword. Here's an example of how a full outer join works

**SQL> select * FROM table1 FULL OUTER JOIN table2 ON table1.column_name = table2.column_name;**

In the above query, table1 and table2 represent the names of the tables you want to perform the full outer join on. Column_name represents the common column between the two tables that you want to use for the join.

The FULL OUTER JOIN keyword specifies the type of join operation, where all rows from both table1 and table2 will be included in the result set, regardless of whether there is a match. The join condition is specified using the ON keyword.
The result set will contain all rows from table1 and table2, with the corresponding matching rows. If there is no match in table1 for a row in table2, the columns from table1 will contain NULL values, and vice versa.
Not all database systems support the FULL OUTER JOIN syntax. In some cases, you can achieve the same result using a combination of a left outer join and a right outer join, along with a UNION or UNION ALL operator.




7. **Self join**

A self join is a type of join operation that occurs when a table is joined with itself. It is used to combine rows from a single table based on a specified join condition between two or more columns within that table.
To perform a self join, you would typically use table aliases to differentiate between the different instances of the same table within the join

SELECT t1.column_name, t2.column_name FROM table_name t1 JOIN table_name t2 ON t1.join_column = t2.join_column;

In the above query, table_name represents the name of the table you want to perform the self join on. Column_name represents the columns you want to select from the table for the result set. Join_column represents the column(s) used as the join condition.

By using table aliases t1 and t2, you create two separate instances of the same table within the join. The join condition specifies how the rows from these instances should be matched.

The result set will contain the selected columns from the self join, combining the rows where the join condition is satisfied within the same table.

Self joins can be useful when you have a table that contains hierarchical or recursive data, such as an employee table where you want to find relationships between employees based on their manager or reporting structure.

## LAB No 9.

**Objective: Introduction to NoSQL Databases**

**Theory:**

**What are NoSQL Databases?**

NoSQL databases, also known as "not only SQL" databases, are a type of database management system that provides a non-relational approach to data storage and retrieval. Unlike traditional relational databases, which use structured query language (SQL) and organized data into tables with predefined schemas, NoSQL databases are designed to handle large volumes of unstructured or semi-structured data.

NoSQL databases are often used in scenarios where the flexibility to handle varying data types, high scalability, and high availability are crucial. They are particularly well-suited for handling big data, real-time web applications, and scenarios where rapid development and iteration are priorities.

**How is data stored in NoSQL Databases?**

Data storage in NoSQL databases varies depending on the type of NoSQL database being used. Here are the typical ways data is stored in different types of NoSQL databases:

**Key-value stores**: In key-value stores, data is stored as a collection of key-value pairs. Each key is unique and is used to retrieve the corresponding value. The values can be simple data types like strings or binary data, or they can be more complex structures like JSON objects. Key-value stores offer fast retrieval of values based on keys and are highly scalable.

**Document databases:** Document databases store data as JSON-like documents. A document is a self-contained unit that contains the data and its associated metadata. Documents are usually organized into collections or buckets. Each document can have a unique identifier and can have a flexible schema, meaning that different documents in the same collection can have different sets of fields. The documents can be queried using various criteria, and indexes are often used to optimize the retrieval of data.

**Column-family databases:** In column-family databases, data is organized into column families, which are containers for related data. Each column family can have a different schema, allowing flexibility in data structure. Within each column family, data is stored as columns and rows. Columns contain individual data values, and rows group related columns together. Column-family databases are optimized for read and write scalability and are designed to handle large amounts of distributed data.

**Graph databases**: Graph databases store data as nodes, edges, and properties. Nodes represent entities, edges represent relationships between entities, and properties store additional information about nodes and edges. Graph databases use index-free adjacency, which means that relationships between nodes are stored directly, allowing for efficient traversal of the graph and querying of complex relationships.

In all types of NoSQL databases, data can be distributed across multiple nodes to achieve scalability and fault tolerance. NoSQL databases often provide mechanisms for horizontal scaling, allowing them to handle large amounts of data and high levels of traffic.

**What are some advantages of NoSQL Databases?**

Some key benefits of NoSQL Databases are:

**Data Model Flexibility:** NoSQL databases support various data models, including key-value pairs, document-based, column-family, and graph databases. This flexibility allows developers to choose the most appropriate data model based on their specific use case and data requirements.

**Schema-less Design:** Unlike traditional SQL databases, NoSQL databases typically have a dynamic or schema-less design. This means that each record or document in the database can have a different structure, and new fields can be added on-the-fly without affecting other records. This flexibility is beneficial in scenarios where data evolves over time or when dealing with semi-structured or unstructured data.

**Horizontal Scalability:** NoSQL databases are generally designed to scale horizontally, which means they can handle increased data volumes by adding more machines to the database cluster. This approach is particularly useful for web applications and big data scenarios where data demands can grow rapidly.

**High Performance:** NoSQL databases often prioritize performance and can achieve high throughput and low latency. Since they are optimized for specific data models, they can provide faster data access and retrieval in certain scenarios compared to traditional SQL databases.

**Distributed Architecture:** Many NoSQL databases are designed to operate in distributed environments, where data is spread across multiple nodes or servers. This architecture enhances fault tolerance and availability, ensuring that the database remains operational even if some nodes fail.

**Use Cases:** NoSQL databases are well-suited for various use cases, including real-time analytics, social media platforms, content management systems, Internet of Things (IoT) applications, recommendation systems, and more. They excel in scenarios with large-scale data ingestion, storage, and retrieval needs.

**What are some of the most popular No SQL Databases?**

Some of the most popular NoSQL Databases are:

**MongoDB:** MongoDB is a document-oriented NoSQL database that stores data in flexible JSON-like documents. It is widely used for web applications, content management systems, and other projects that require dynamic and schema-less data structures.

**Apache Cassandra**: Cassandra is a distributed column-family NoSQL database known for its high availability and horizontal scalability. It is designed to handle large amounts of data and is used in various applications, including real-time analytics and IoT.

**Redis:** Redis is an in-memory data structure store that can function as a key-value database, a cache, and a message broker. It is highly performant and commonly used for caching, real-time analytics, session management, and more.

**Couchbase:** Couchbase is a NoSQL database that supports both key-value and document data models. It offers high performance, scalability, and mobile synchronization, making it suitable for various use cases.

**Neo4j:** Neo4j is a graph database that specializes in managing and processing highly interconnected data. It is used in applications that require complex relationships, such as social networks, recommendation engines, and fraud detection systems.

**Amazon DynamoDB:** DynamoDB is a fully managed NoSQL database service provided by Amazon Web Services (AWS). It offers seamless scalability, high availability, and low latency, making it popular for cloud-based applications.

**Apache HBase:** HBase is a distributed column-family database built on top of the Hadoop Distributed File System (HDFS). It is often used in big data environments and is capable of handling massive amounts of data.

**CouchDB:** CouchDB is a document-oriented NoSQL database that emphasizes ease of use and replication. It is suitable for applications that require offline access and data synchronization.

**RavenDB:** RavenDB is a document database that provides ACID transactions and flexible data modeling. It is often used in .NET-based applications.

**LAB No 10.**

**Objective: Installing MongoDB and Mogosh**

**Theory:**

**What is MongoDB?**

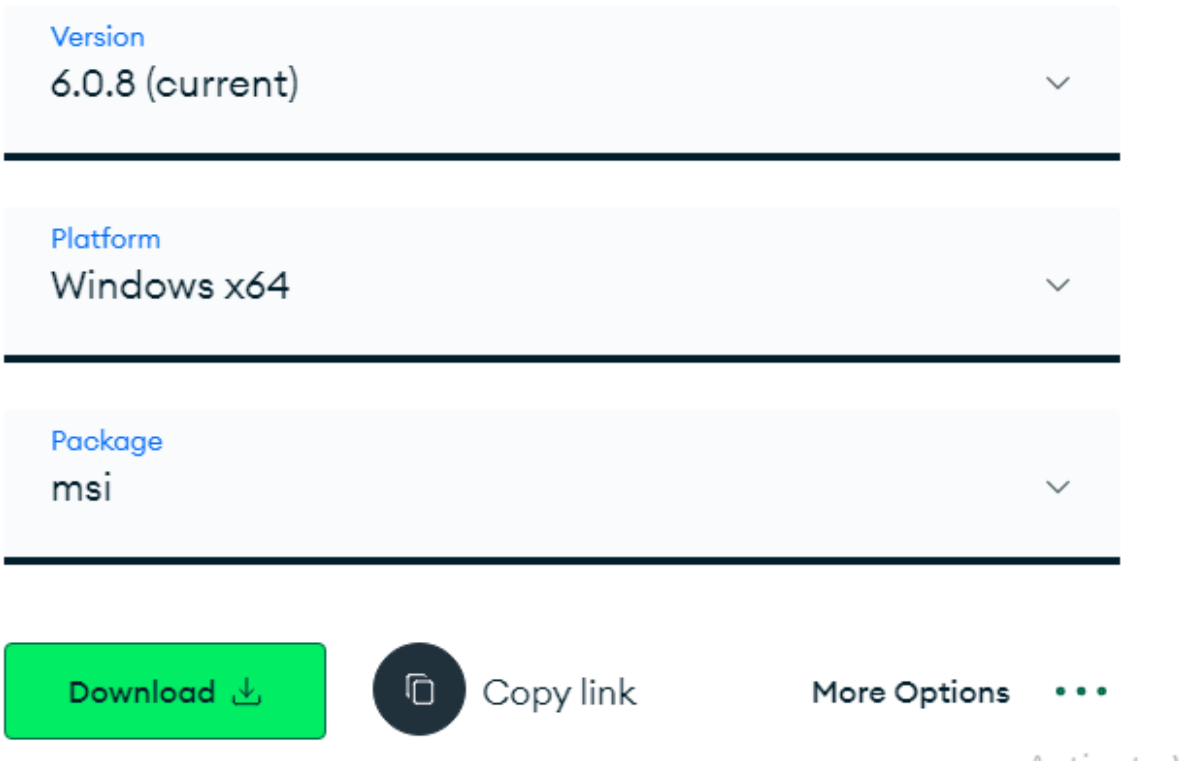
MongoDB is a popular open-source document-oriented NoSQL database that falls under the category of document databases. It is designed to store, retrieve, and manage semi-structured data in the form of JSON-like documents.

**What is Mongosh?**

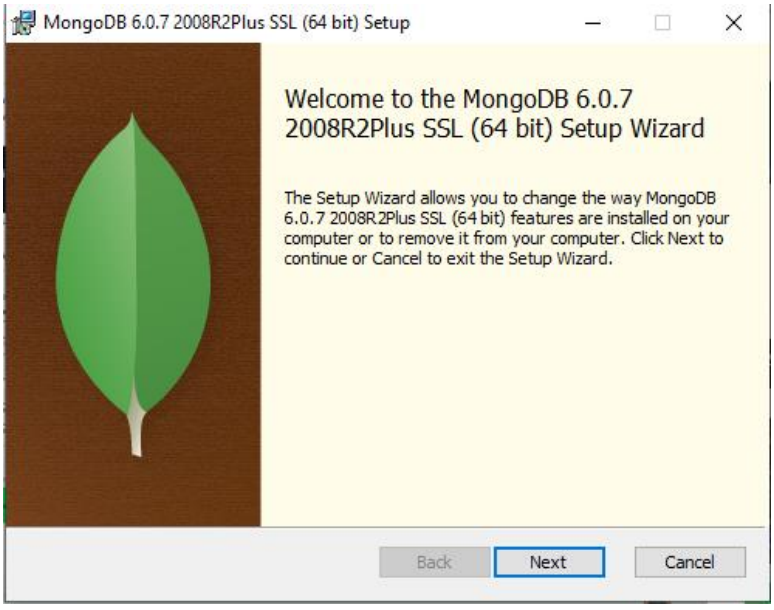
Mongosh is a new command-line shell and interactive JavaScript-based MongoDB shell. It is developed and maintained by MongoDB, Inc. and is intended to replace the legacy MongoDB shell (mongo) as the default shell for interacting with MongoDB databases.

**Installing MongoDB:**

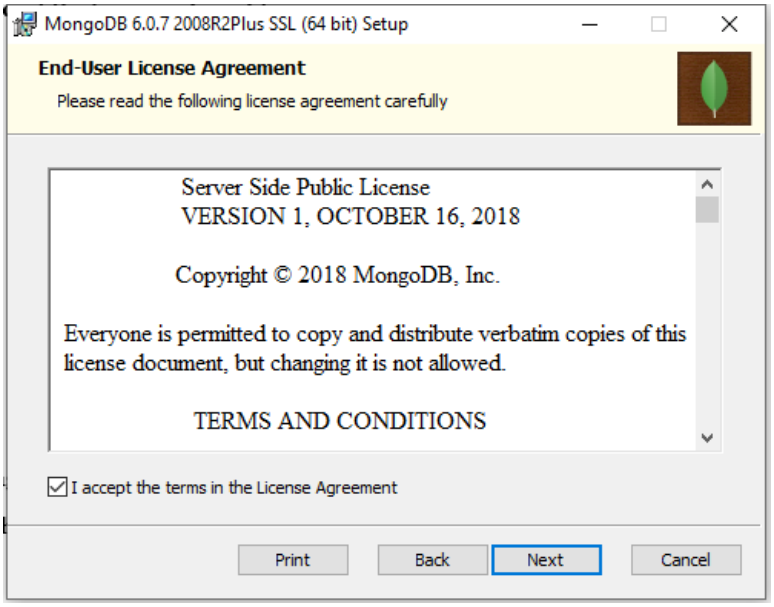
**Step1: Download the Community Edition setup from Official website of MongoDB.**



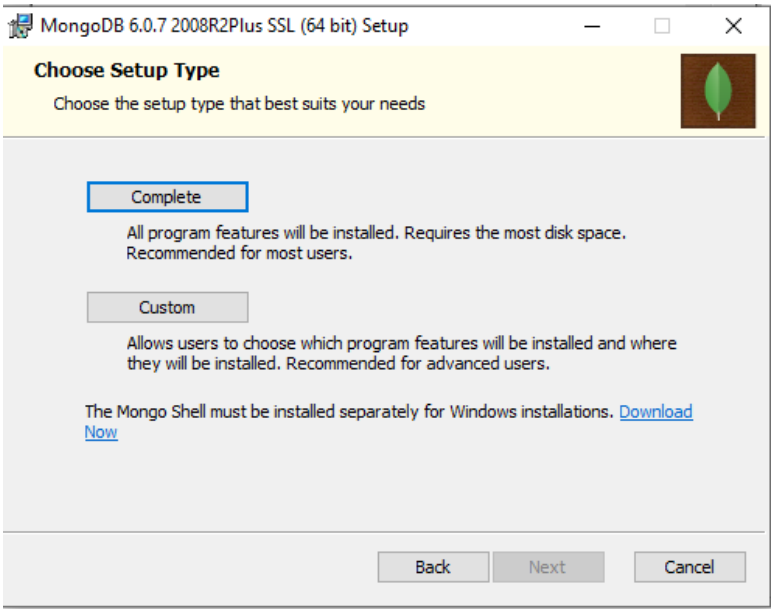
**Step2: Once the setup is downloaded, you can install it by clicking onto the downloaded installer.**



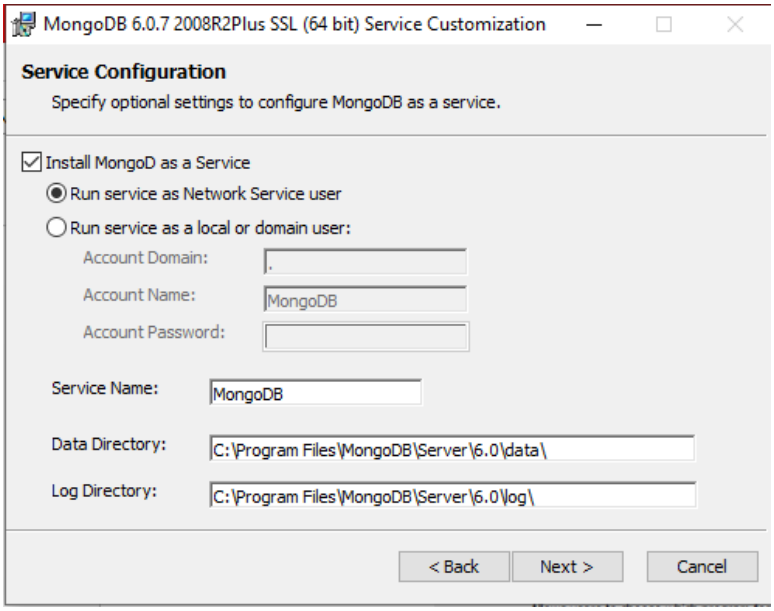

**Step 3: Accept the terms and conditions**



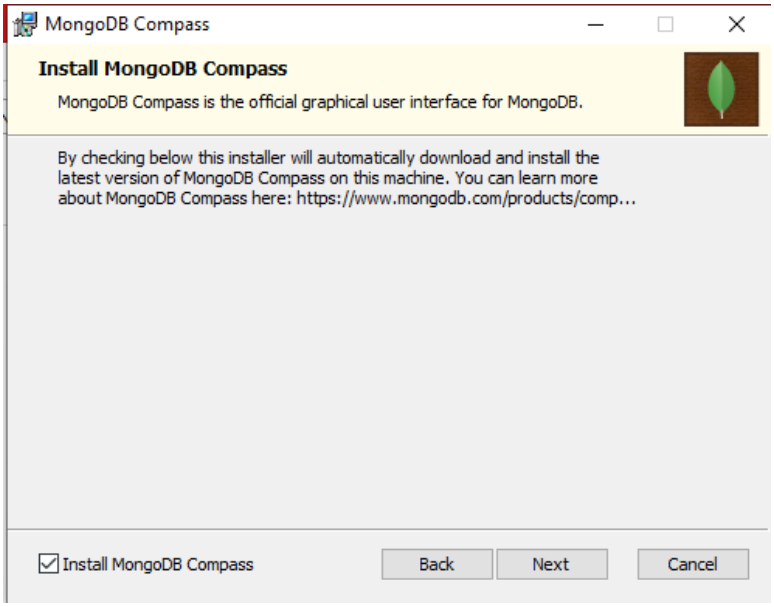
**Step 4: Choose the setup type as ‘Complete’**



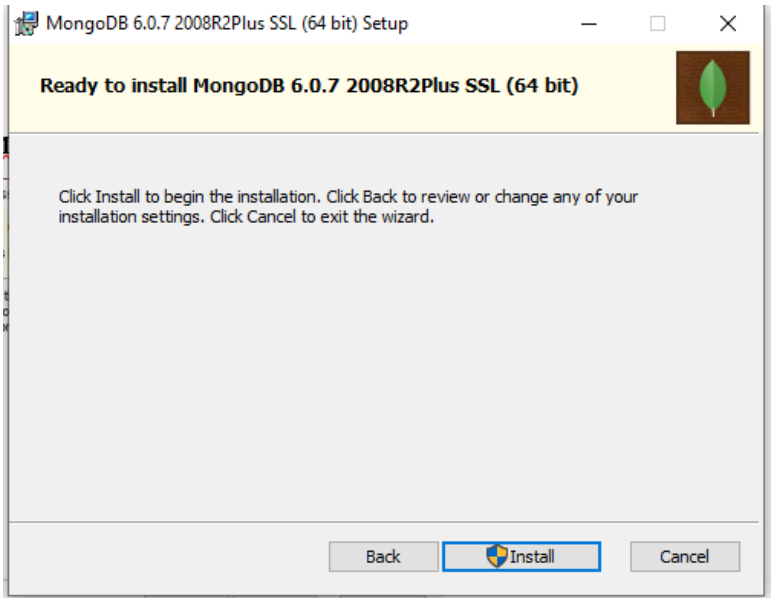

**Step 5: Check Install MongoDB as a service in service configuration and select Run Service as Network service user**



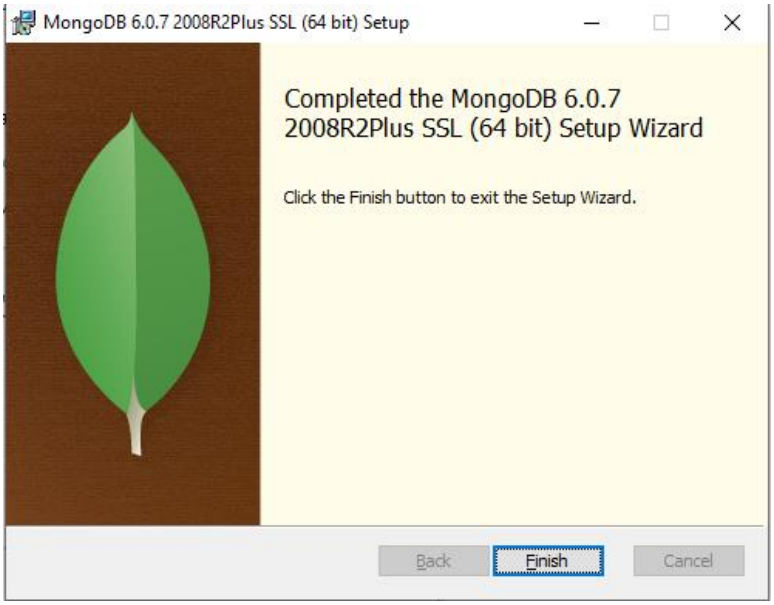
**Step 6: Make sure that Install MongoDB Compass is selected.**



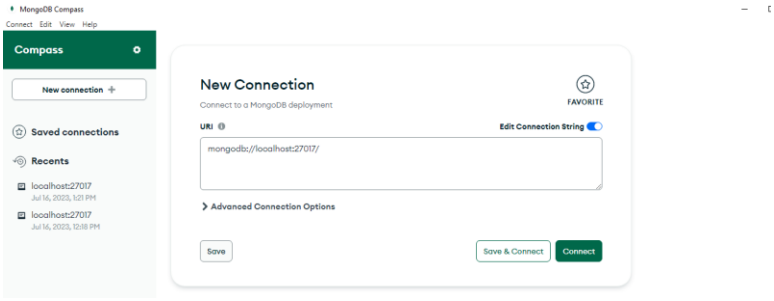

**Step 7: Once done, click on Install**



**Step 8: Click Finish after installation.**



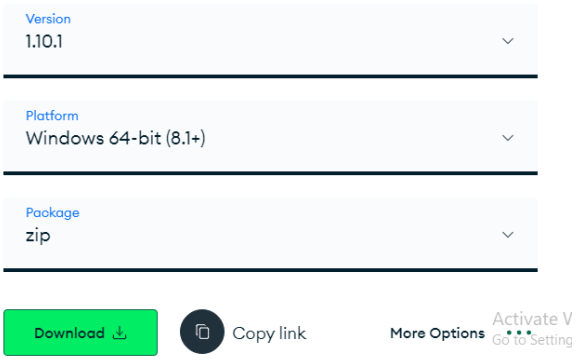

**Step 9: A new Application Compass must open after installation. This is the GUI of MongoDB**



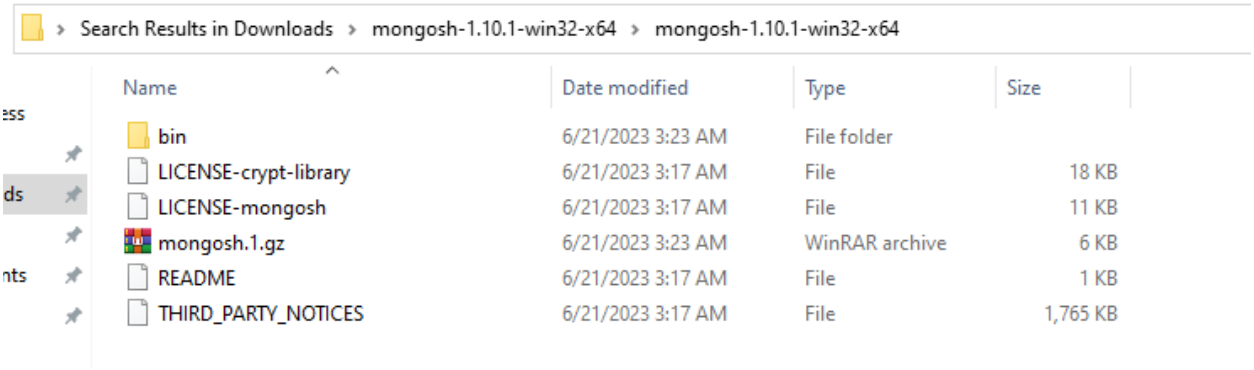

**Installation of Mongosh**

Once MongoDB is installed, you must now install Mongosh which is a new command-line shell. The following steps must be carried out:

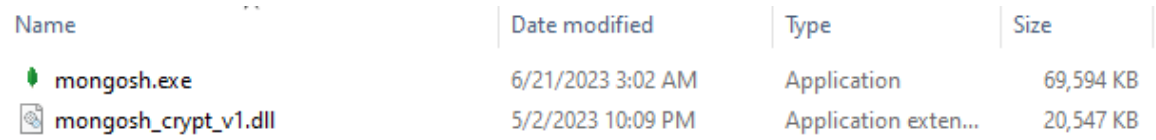
**Step 1: Download the setup of Mongosh from the official website of MongoDB.**



**Step 2: Once the setup in downloaded, unzip and go to the bin folder .**

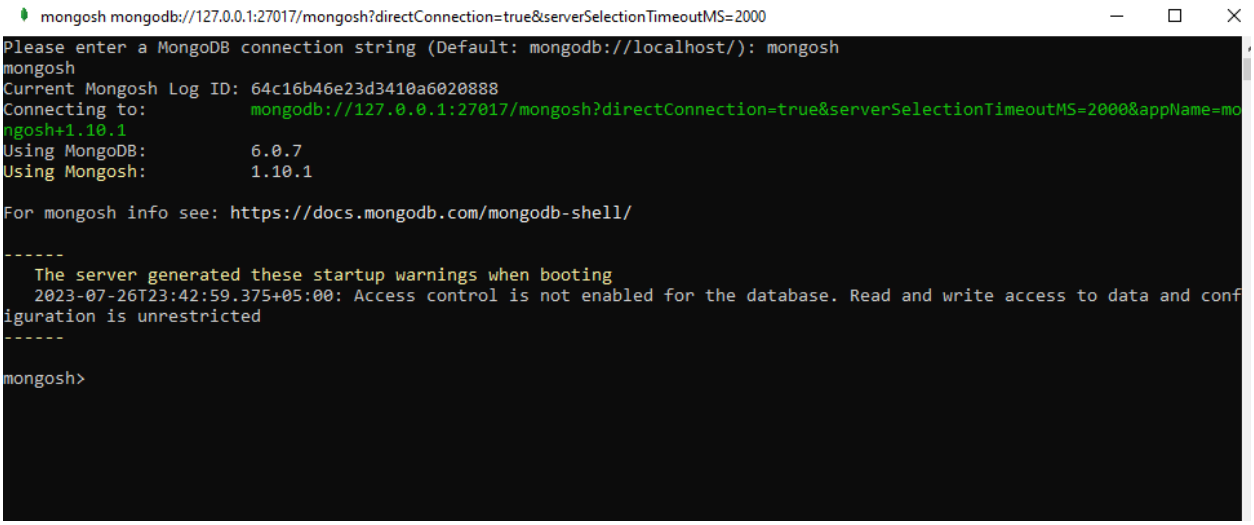



**Step 3: Once inside the bin folder, you may now be able to view the Mongosh Application. Run the executable file**




**Step 4: After running the executable file, you may view a command line prompt.**

## Step 5: Establish connection by typing mongosh in the command line interface



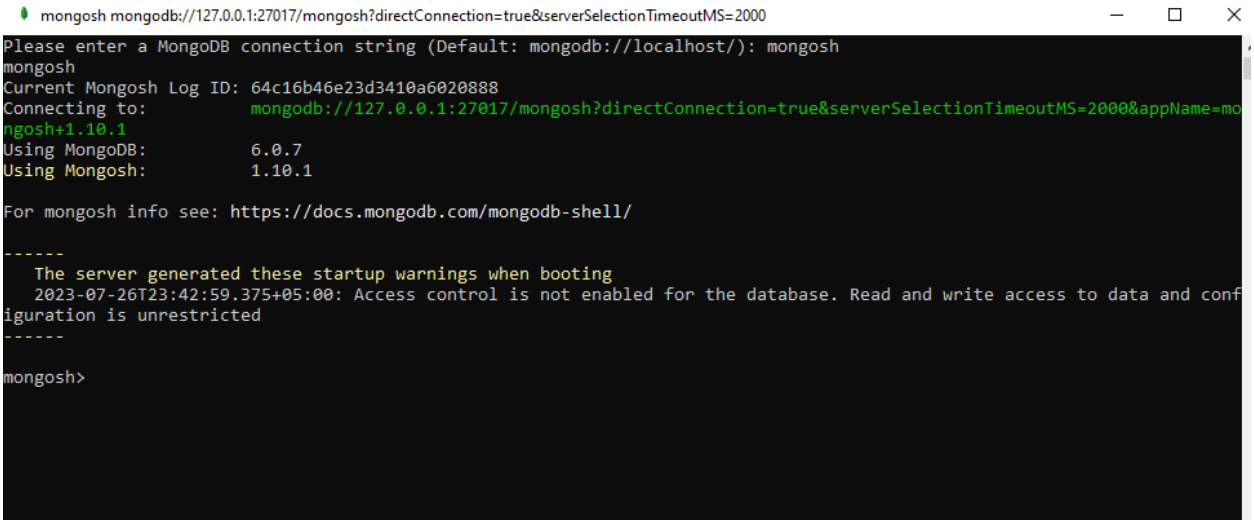
## LAB No 11.

**Objective: Working with MongoDB Shell "Mongosh"**

**Theory:**

**Syntax of Commands**

To establish connection with the MongoDb shell, type mongosh in the command line interface

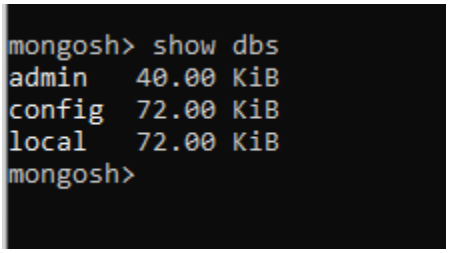



To clear screen; **mongosh> cls**

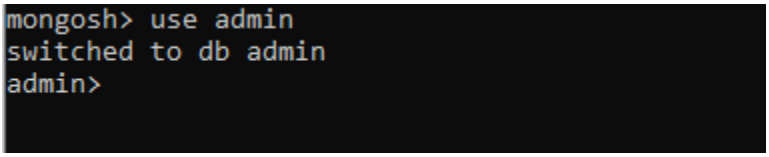
To exit; **mongosh> exit**

To show all databases; **mongosh> show dbs**

This gives a list of all databases available




To use any of the existing database; **mongosh> use admin**



To create a new database; **admin> use school**

```
admin> use school
switched to db school
school>
```

To show school database, you need to enter some data here, otherwise it wont show.

```
school> show dbs
admin   40.00 KiB
config  72.00 KiB
local   72.00 KiB
school>
```

To add data to the database **school> db.createCollection("student")**

```
school> db.createCollection("student")
{ ok: 1 }
school>
```

Student is the name of collection.
Now if you want to see school database, you will be able to see it by using the command show dbs.

```
school> show dbs
admin    40.00 KiB
config  108.00 KiB
local    72.00 KiB
school    8.00 KiB
school>
```

To see the collections in the database **school> show collections**

To drop any database; **school>db.dropDatabase()**

```
school> db.dropDatabase()
{ ok: 1, dropped: 'school' }
school>
```

To insert document in MongoDb: **school>db.student.insertOne({name:"shiza", age:30, gpa:3.2})**

```
school> db.student.insertOne({name: "shiza",age:30, gpa:3.2})
{
  acknowledged: true,
  insertedId: ObjectId("64c16dc2fdd332ea482f9bbe")
}
school>
```

To return all documents in the collection: **school>db.student.find()**

```
school> db.student.find()
[
  {
    _id: ObjectId("64c16dc2fdd332ea482f9bbe"),
    name: 'shiza',
    age: 30,
    gpa: 3.2
  }
]
school>
```

To insert more than one document at a time: **school>db.student.insertMany([{name:"amna", age: 38, gpa:1.5},{name: "hina", age: 27, gpa: 4.0},{name: "simra", age: 18, gpa:2.5}])**

```
school> db.student.insertMany([{name:"amna", age:38, gpa:1.5}, {name: "hina", age:27, gpa:4.0}, {name: "simra", age: 18,
 gpa: 2.5}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId("64c16e50fdd332ea482f9bbf"),
    '1': ObjectId("64c16e50fdd332ea482f9bc0"),
    '2': ObjectId("64c16e50fdd332ea482f9bc1")
  }
}
school>
```

You can add as many documents and as many field values inside the documents as you like. They do not have to be necessarily consistent.

To display all of them: **school>db.student.find()**

```
school> db.student.find()
[
  {
    _id: ObjectId("64c16dc2fdd332ea482f9bbe"),
    name: 'shiza',
    age: 30,
    gpa: 3.2
  },
  {
    _id: ObjectId("64c16e50fdd332ea482f9bbf"),
    name: 'amna',
    age: 38,
    gpa: 1.5
  },
  {
    _id: ObjectId("64c16e50fdd332ea482f9bc0"),
    name: 'hina',
    age: 27,
    gpa: 4
  },
  {
    _id: ObjectId("64c16e50fdd332ea482f9bc1"),
    name: 'simra',
    age: 18,
    gpa: 2.5
  }
]
school>
```

**Data types in MongoDB**

1. **String:**

**school>db.student.insertOne({name: “ayesha”})**

The above query has string as a data type. Strings are series of text in single or double quotes, they can have spaces in between and can also have numbers.

2. **Integer:**

**school>db.student.insertOne({age: 32})**

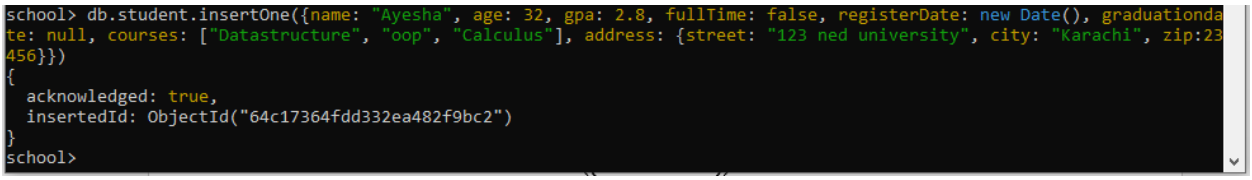
The above query has integer as a data type. Integers are whole numbers.

3. Double:

**school>db.student.insertOne({gpa: 3.2})**

The above query has double as a data type Doubles are numbers that contain decimals. It’s a double because it contains a decimal portion

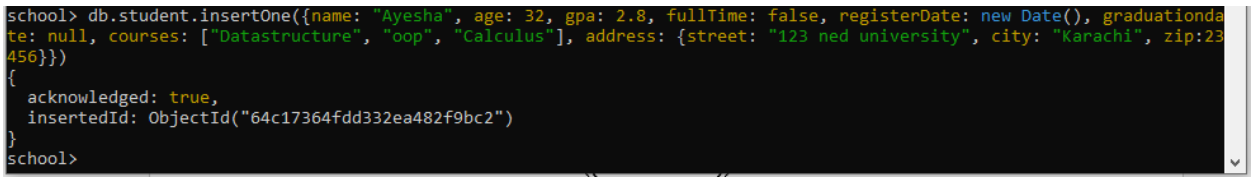
4. **Booleans** are either true or off **({fulltime: false})**
5. **Date objects**: **({registerDate: new Date(“2023-01-07”) or just ({registerDate: new Date()**
6. **Null value**: No Value **({graduationDate: null})**
7. **Field**: That can contain multiple values. **courses: [“Datastructures”, “oop”, “Calculus”]**




**Nested Documents:**

We can have multiple documents inside a document. This can be done by adding another document inside already existing document.

**school> Address: { street: “123 ned university” , city: “ Karachi”, zip: 2345}6});**




**Sorting documents:**

We can sort the values inside a document in an ascending or descending order. This can be done by using the following queries.

**For Ascending Sort:**

**school>db.student.find(). sort({name:1})**

```
school> db.student.find().sort({name:1})
[
  {
    _id: ObjectId("64c17364fdd332ea482f9bc2"),
    name: 'Ayesha',
    age: 32,
    gpa: 2.8,
    fullTime: false,
    registerDate: ISODate("2023-07-26T19:26:28.369Z"),
    graduationdate: null,
    courses: [ 'Datastructure', 'oop', 'Calculus' ],
    address: { street: '123 ned university', city: 'Karachi', zip: 23456 }
  },
  {
    _id: ObjectId("64c16e50fdd332ea482f9bbf"),
    name: 'amna',
    age: 38,
    gpa: 1.5
  },
  {
    _id: ObjectId("64c16e50fdd332ea482f9bc0"),
    name: 'hina',
    age: 27,
    gpa: 4
  },
  {
    _id: ObjectId("64c16dc2fdd332ea482f9bbe"),
```

```
  {
    _id: ObjectId("64c16e50fdd332ea482f9bc0"),
    name: 'hina',
    age: 27,
    gpa: 4
  },
  {
    _id: ObjectId("64c16dc2fdd332ea482f9bbe"),
    name: 'shiza',
    age: 30,
    gpa: 3.2
  },
  {
    _id: ObjectId("64c16e50fdd332ea482f9bc1"),
    name: 'simra',
    age: 18,
    gpa: 2.5
  }
]
school>
```

**school>db.student.find(). sort({age:1})**

```
school> db.student.find().sort({age:1})
[
  {
    _id: ObjectId("64c16e50fdd332ea482f9bc1"),
    name: 'simra',
    age: 18,
    gpa: 2.5
  },
  {
    _id: ObjectId("64c16e50fdd332ea482f9bc0"),
    name: 'hina',
    age: 27,
    gpa: 4
  },
  {
    _id: ObjectId("64c16dc2fdd332ea482f9bbe"),
    name: 'shiza',
    age: 30,
    gpa: 3.2
  },
  {
    _id: ObjectId("64c17364fdd332ea482f9bc2"),
    name: 'Ayesha',
    age: 32,
    gpa: 2.8,
    fullTime: false,
    registerDate: ISODate("2023-07-26T19:26:28.369Z"),
    graduationdate: null,
    courses: [ 'Datastructure', 'oop', 'Calculus' ],
    address: { street: '123 ned university', city: 'Karachi', zip: 23456 }
  },
  {
    _id: ObjectId("64c16e50fdd332ea482f9bbf"),
    name: 'amna',
    age: 38,
    gpa: 1.5
  }
]
school>
```

**For Descending Sort:**

```
school> db.student.find().sort({name:-1})
[
  {
    _id: ObjectId("64c16e50fdd332ea482f9bc1"),
    name: 'simra',
    age: 18,
    gpa: 2.5
  },
  {
    _id: ObjectId("64c16dc2fdd332ea482f9bbe"),
    name: 'shiza',
    age: 30,
    gpa: 3.2
  },
  {
    _id: ObjectId("64c16e50fdd332ea482f9bc0"),
    name: 'hina',
    age: 27,
    gpa: 4
  },
  {
    _id: ObjectId("64c16e50fdd332ea482f9bbf"),
    name: 'amna',
    age: 38,
    gpa: 1.5
  },
  {
    _id: ObjectId("64c17364fdd332ea482f9bc2"),
    name: 'Ayesha',
    age: 32,
    gpa: 2.8,
    fullTime: false,
    registerDate: ISODate("2023-07-26T19:26:28.369Z"),
    graduationdate: null,
    courses: [ 'Datastructure', 'oop', 'Calculus' ],
    address: { street: '123 ned university', city: 'Karachi', zip: 23456 }
  }
]
school>
```

**Finding only one/first document:**

The findOne() method finds and returns one document that matches the given selection criteria. If multiple documents satisfy the given query expression, then this method will return the first document according to the natural order which reflects the order of documents on the disk. If no document matches the selection criteria, then this method will return null.

**school>db.student.findOne()**

```
school> db.student.findOne()
{
  _id: ObjectId("64c16dc2fdd332ea482f9bbe"),
  name: 'shiza',
  age: 30,
  gpa: 3.2
}
school>
```

**Adding arguments to find():**

To find specific entries in the database, we will query as

**school>db.student.find({name: “shiza”})**

```
school> db.student.find({name:"shiza"})
[
  {
    _id: ObjectId("64c16dc2fdd332ea482f9bbe"),
    name: 'shiza',
    age: 30,
    gpa: 3.2
  }
]
school>
```

**school>db.student.find({gpa: 3.2, fullTime: false})** No Result as there is no entry that matches the criteria

**school>db.student.find({gpa: 3.2, fullTime: false})**

```
school> db.student.find({gpa: 3.2, fullTime: false})

school> db.student.find({gpa: 2.8, fullTime: false})
[
  {
    _id: ObjectId("64c17364fdd332ea482f9bc2"),
    name: 'Ayesha',
    age: 32,
    gpa: 2.8,
    fullTime: false,
    registerDate: ISODate("2023-07-26T19:26:28.369Z"),
    graduationdate: null,
    courses: [ 'Datastructure', 'oop', 'Calculus' ],
    address: { street: '123 ned university', city: 'Karachi', zip: 23456 }
  }
]
school>
```

### Query and Projection Argument in find()

The projection arguments takes up values and produces answer if true or false.

**school>db.student.find({}, {name: true})**

```
school> db.student.find({}, {name: true})
[
  { _id: ObjectId("64c16dc2fdd332ea482f9bbe"), name: 'shiza' },
  { _id: ObjectId("64c16e50fdd332ea482f9bbf"), name: 'amna' },
  { _id: ObjectId("64c16e50fdd332ea482f9bc0"), name: 'hina' },
  { _id: ObjectId("64c16e50fdd332ea482f9bc1"), name: 'simra' },
  { _id: ObjectId("64c17364fdd332ea482f9bc2"), name: 'Ayesha' }
]
school>
```

### Optimizing the query

**school>db.student.find({}, {_id: false, name: true})**

```
school> db.student.find({}, {_id: false, name: true})
[
  { name: 'shiza' },
  { name: 'amna' },
  { name: 'hina' },
  { name: 'simra' },
  { name: 'Ayesha' }
]
school>
```

**school>db.student.find({}, {_id: false, name: true, gpa: true})**

```
school> db.student.find({}, {_id: false, name: true, gpa: true})
[
  { name: 'shiza', gpa: 3.2 },
  { name: 'amna', gpa: 1.5 },
  { name: 'hina', gpa: 4 },
  { name: 'simra', gpa: 2.5 },
  { name: 'Ayesha', gpa: 2.8 }
]
school>
```

### Limiting documents:

We can limit the number of entries shown in the output

**school>db.student.find().limit(1);** this will result to only 1 output sorted through a manner such as object ID's.

```
school> db.student.find().limit(1)
[
  {
    _id: ObjectId("64c16dc2fdd332ea482f9bbe"),
    name: 'shiza',
    age: 30,
    gpa: 3.2
  }
]
school>
```

**Combining both sort and limit:**

Finding the student with highest GPA:
**school>db.student.find().sort({gpa:-1}).limit(1)**

```
school> db.student.find().sort({gpa:-1}). limit(1)
[
  {
    _id: ObjectId("64c16e50fdd332ea482f9bc0"),
    name: 'hina',
    age: 27,
    gpa: 4
  }
]
school>
```

Finding the student with lowest GPA:
**school>db.student.find().sort({gpa:1}).limit(1)**

```
school> db.student.find().sort({gpa:1}). limit(1)
[
  {
    _id: ObjectId("64c16e50fdd332ea482f9bbf"),
    name: 'amna',
    age: 38,
    gpa: 1.5
  }
]
school>
```

**Update the documents:**

To update a document we need to pass two arguments to the query

**school>db.student.updateOne({filter}, {update})**

**school>db.student.updateOne({name: “shiza”}, {$set:{fullTime: true}})**

```
school> db.student.updateOne({name: "shiza"}, {$set:{fullTime: true}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
school>
```

**To verify that the record has been updated:**

**school>db. student.find({name: “shiza”})**

```
school> db.student.find({name: "shiza"})
[
  {
    _id: ObjectId("64c16dc2fdd332ea482f9bbe"),
    name: 'shiza',
    age: 30,
    gpa: 3.2,
    fullTime: true
  }
]
school>
```

**To update using Object ID:**

Object ID’s are unique for all records, hence it is always better to update a record using an object ID.

**school>db. student. updateOne({_id: ObjectId(“656……..bbe”)}, {$set: {fullTime: false}})**

```
school> db. student. updateOne({_id: ObjectId("64c16dc2fdd332ea482f9bbe")}, {$set: {fullTime: false}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
school>
```

**To verify that the record has been updated:**

**school>db. student.find({name: “shiza”})**

```
school> db.student.find({name: "shiza"})
[
  {
    _id: ObjectId("64c16dc2fdd332ea482f9bbe"),
    name: 'shiza',
    age: 30,
    gpa: 3.2,
    fullTime: false
  }
]
school>
```

**To unset any record:**

**school>db. student. updateOne({_id: ObjectId("656……..bbe")}, {$unset: {fullTime: ""}})**

This query will remove the fullTime field entirely from the record of the Object ID given in the filter argument.

```
school> db. student. updateOne({_id: ObjectId("64c16dc2fdd332ea482f9bbe")}, {$unset: {fullTime:""}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
school>
```

**To verify that the record has been updated:**

**school>db. student.find({name: "shiza"})** (You can also use Object Id to find the changes)

```
school> db.student.find({name: "shiza"})
[
  {
    _id: ObjectId("64c16dc2fdd332ea482f9bbe"),
    name: 'shiza',
    age: 30,
    gpa: 3.2
  }
]
school>
```

**To Update Many records:**

In order to update many records at one time, we can use the query.

This will filter out records will the field fullTime and if any record does not have the required field, it will added to that field.

**school> db. student. updateMany({fullTime: {$exists:false}}, {$set: { fullTime: true}})**

```
school> db.student.updateMany({fullTime: {$exists: false}}, {$set: { fullTime:true}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 4,
  modifiedCount: 4,
  upsertedCount: 0
}
school>
```

**To verify that the record has been updated:**

**school>db. student.find()**

```
school> db.student.find()
[
  {
    _id: ObjectId("64c16dc2fdd332ea482f9bbe"),
    name: 'shiza',
    age: 30,
    gpa: 3.2,
    fullTime: true
  },
  {
    _id: ObjectId("64c16e50fdd332ea482f9bbf"),
    name: 'amna',
    age: 38,
    gpa: 1.5,
    fullTime: true
  },
  {
    _id: ObjectId("64c16e50fdd332ea482f9bc0"),
    name: 'hina',
    age: 27,
    gpa: 4,
    fullTime: true
```

```
  },
  {
    _id: ObjectId("64c16e50fdd332ea482f9bc1"),
    name: 'simra',
    age: 18,
    gpa: 2.5,
    fullTime: true
  },
  {
    _id: ObjectId("64c17364fdd332ea482f9bc2"),
    name: 'Ayesha',
    age: 32,
    gpa: 2.8,
    fullTime: false,
    registerDate: ISODate("2023-07-26T19:26:28.369Z"),
    graduationdate: null,
    courses: [ 'Datastructure', 'oop', 'Calculus' ],
    address: { street: '123 ned university', city: 'Karachi', zip: 23456 }
  }
]
school>
```

**To delete a document:**

To delete one document at a time we will use deleteOne. This can be queried as

**school>db. student. deleteOne({name: “shiza”})**

```
school> db.student.deleteOne({name: "shiza"})
{ acknowledged: true, deletedCount: 1 }
```

**To verify that the record has been updated:**

**school>db. student.find()**

We will not be able to find the document who has name: “shiza”

```
school> db.student.find()
[
  {
    _id: ObjectId("64c16e50fdd332ea482f9bbf"),
    name: 'amna',
    age: 38,
    gpa: 1.5,
    fullTime: true
  },
  {
    _id: ObjectId("64c16e50fdd332ea482f9bc0"),
    name: 'hina',
    age: 27,
    gpa: 4,
    fullTime: true
  },
  {
    _id: ObjectId("64c16e50fdd332ea482f9bc1"),
    name: 'simra',
    age: 18,
    gpa: 2.5,
    fullTime: true
  },
  {
    _id: ObjectId("64c17364fdd332ea482f9bc2"),
    name: 'Ayesha',
```

```
  {
    _id: ObjectId("64c17364fdd332ea482f9bc2"),
    name: 'Ayesha',
    age: 32,
    gpa: 2.8,
    fullTime: false,
    registerDate: ISODate("2023-07-26T19:26:28.369Z"),
    graduationdate: null,
    courses: [ 'Datastructure', 'oop', 'Calculus' ],
    address: { street: '123 ned university', city: 'Karachi', zip: 23456 }
  }
]
school>

school>
```

To delete many records at the same time, we can use

**school>db. student. deleteMany({fullTime: false})**

This will delete all documents where the fullTime field has been set to false

```
school> db.student.deleteMany({fullTime:false})
{ acknowledged: true, deletedCount: 1 }
school>
```

**To verify that the record has been updated:**

**school>db. student.find()**

We will not be able to find the document who has fullTime field set to “false”

```
school> db.student.find()
[
  {
    _id: ObjectId("64c16e50fdd332ea482f9bbf"),
    name: 'amna',
    age: 38,
    gpa: 1.5,
    fullTime: true
  },
  {
    _id: ObjectId("64c16e50fdd332ea482f9bc0"),
    name: 'hina',
    age: 27,
    gpa: 4,
    fullTime: true
  },
  {
    _id: ObjectId("64c16e50fdd332ea482f9bc1"),
    name: 'simra',
    age: 18,
    gpa: 2.5,
    fullTime: true
  }
]
school>
```

We can also check and then delete any document

**school>db. student. deleteMany({registerDate:{$exists: false}})**

```
school> db.student.deleteMany({registerDate:{$exists: false}})
{ acknowledged: true, deletedCount: 3 }
school>
```

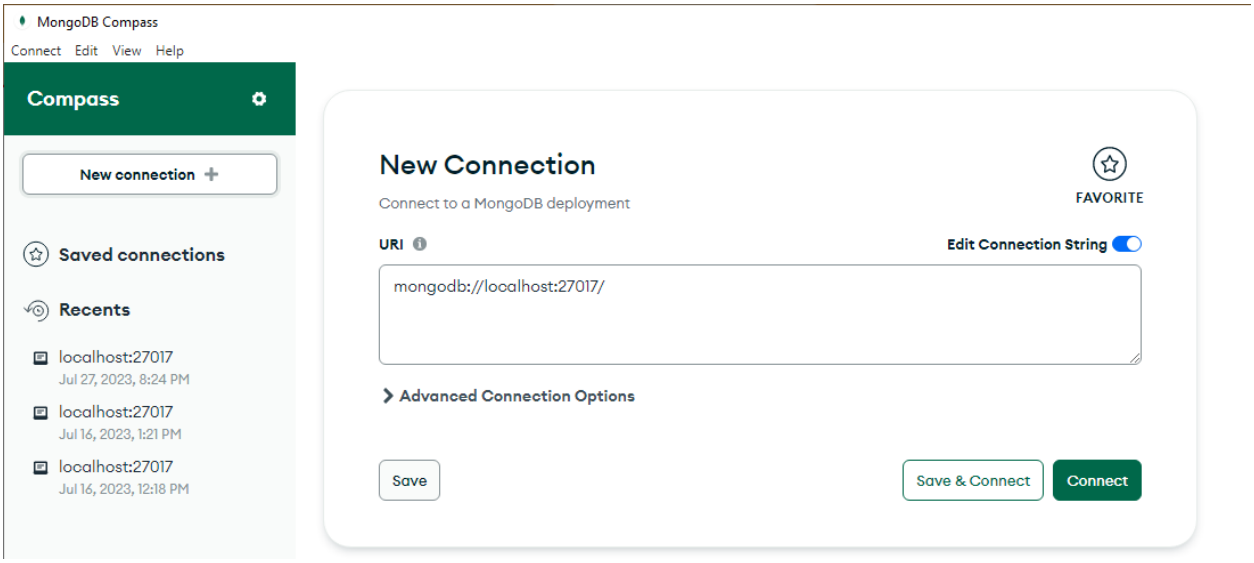
## LAB No 12.

**Objective: Working with MongoDB GUI "Compass"**

**Theory:**

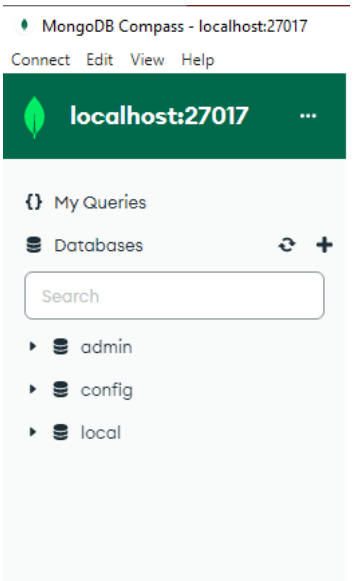
**Syntax of Commands**

Create a connection to MongoDB by clicking on Connect. This will connect you to the local host database

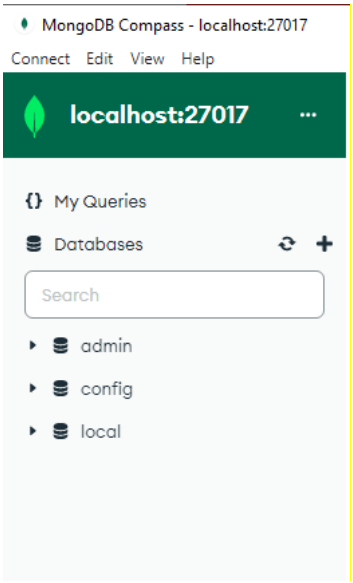



**To view all databases in MongoDB Compass:**

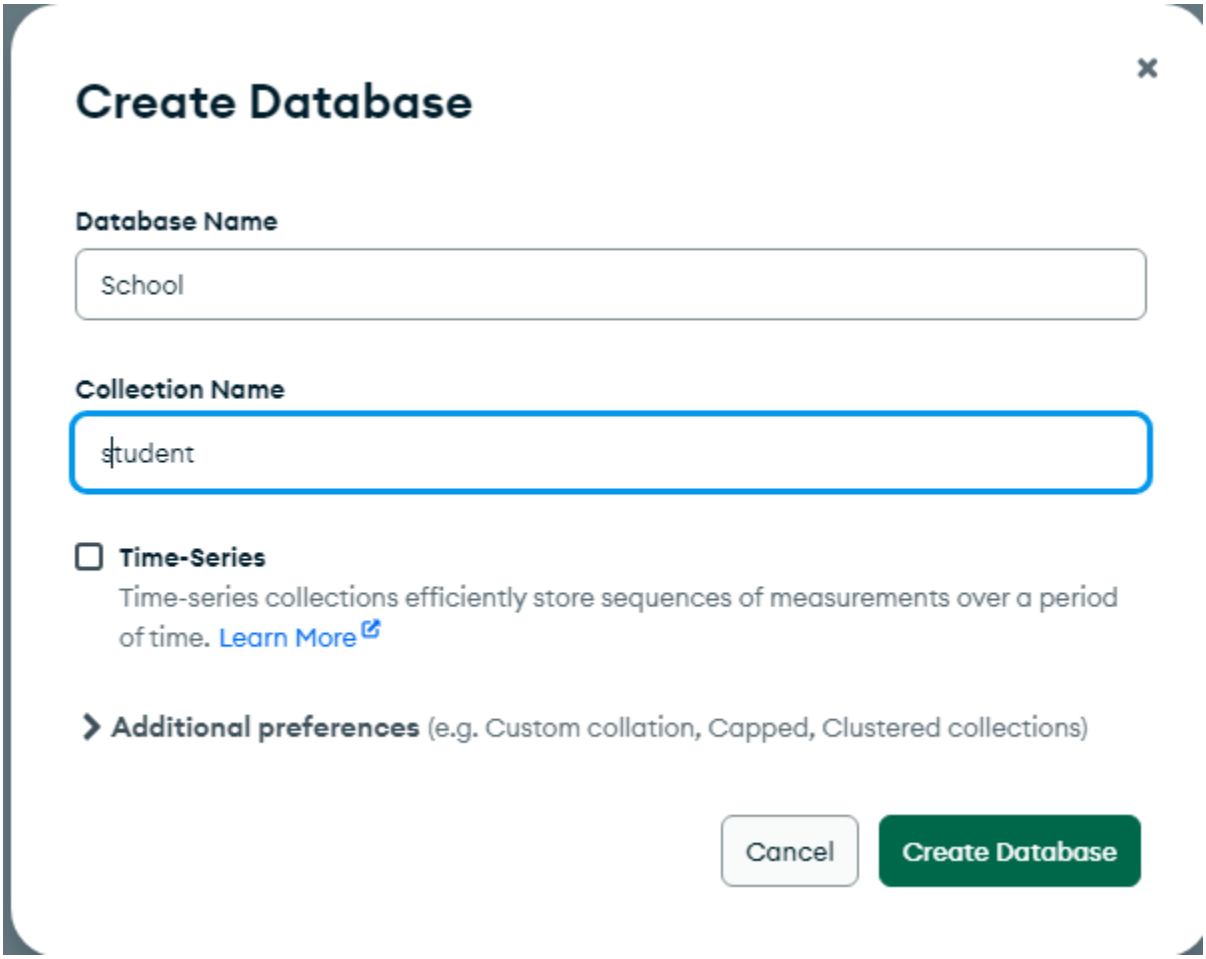
You can view the already existing databses on the left panel of the MongoDB Compass.



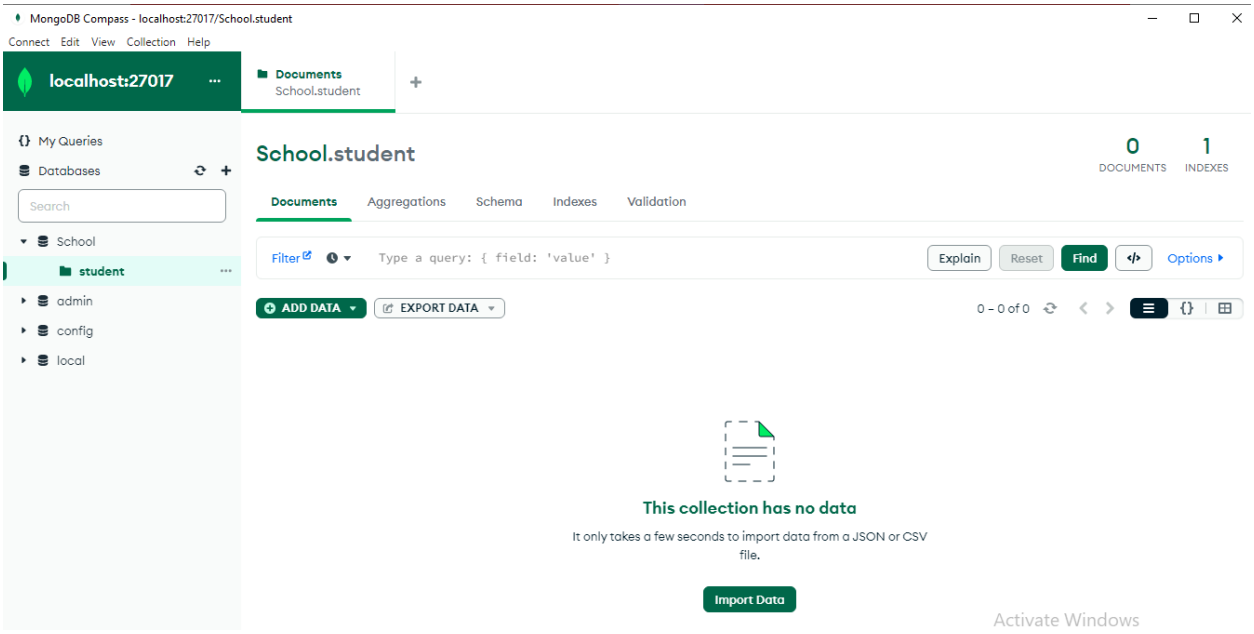
**To Create a new database**, click on the + sign in the Databases tab




Enter the Database and Collection name. Once entered click on Create Database option.

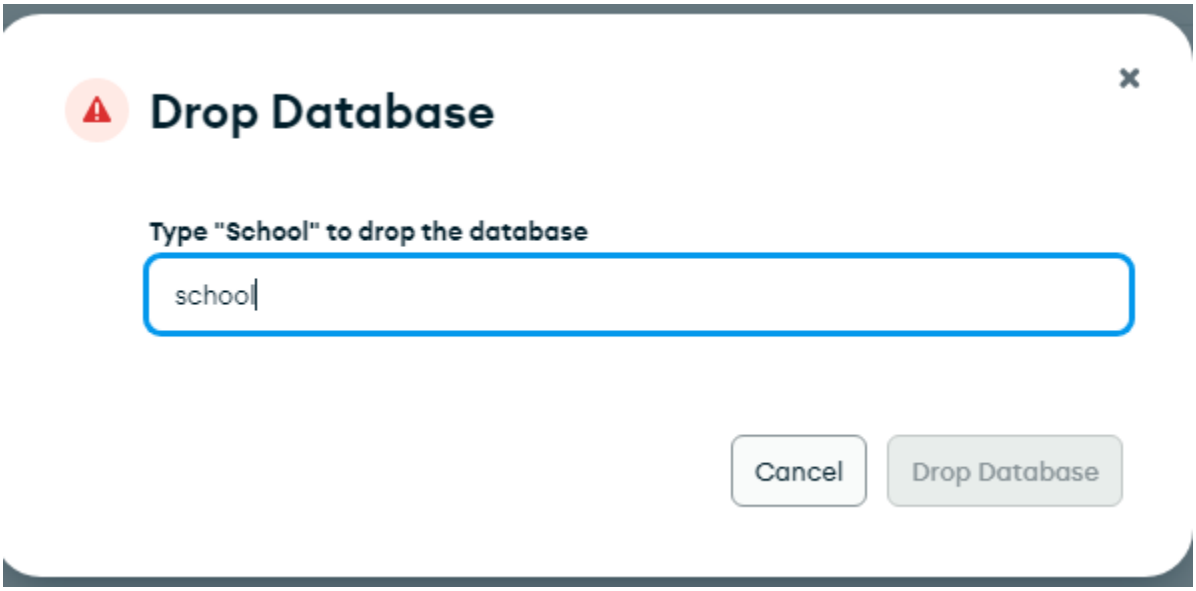


Now you can view the School Database and “student” Collection




**To delete a database in MongoDB Compass:**

Click on the database and then click on the trash can icon. Type the name of the database in the new prompt and select delete database. The database will be deleted.

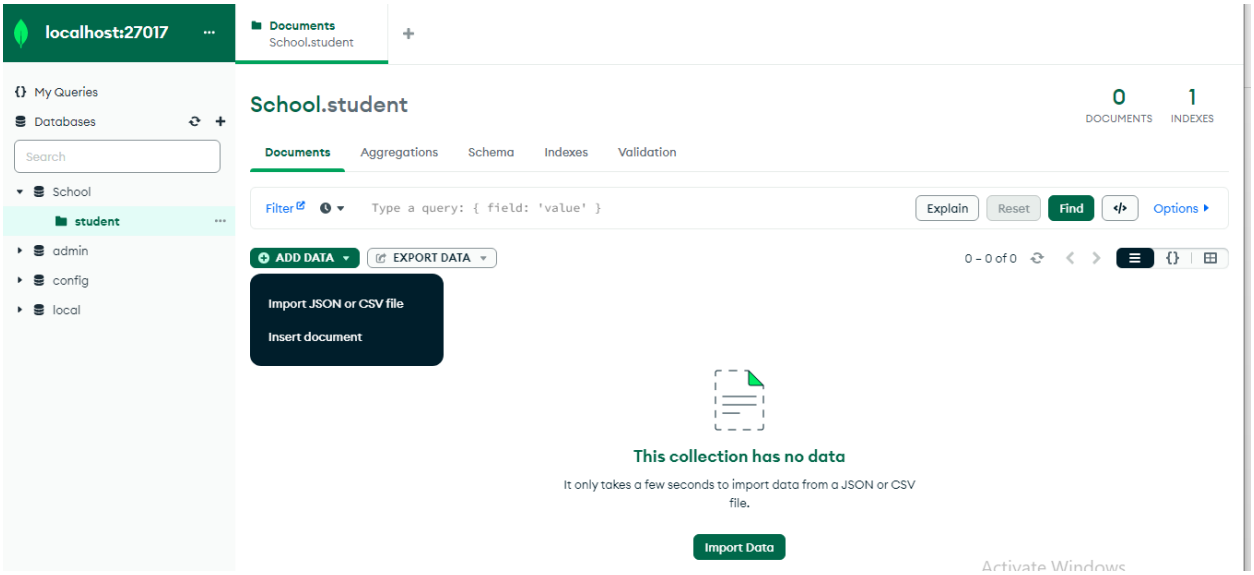


**To insert data in MongoDB Compass:**

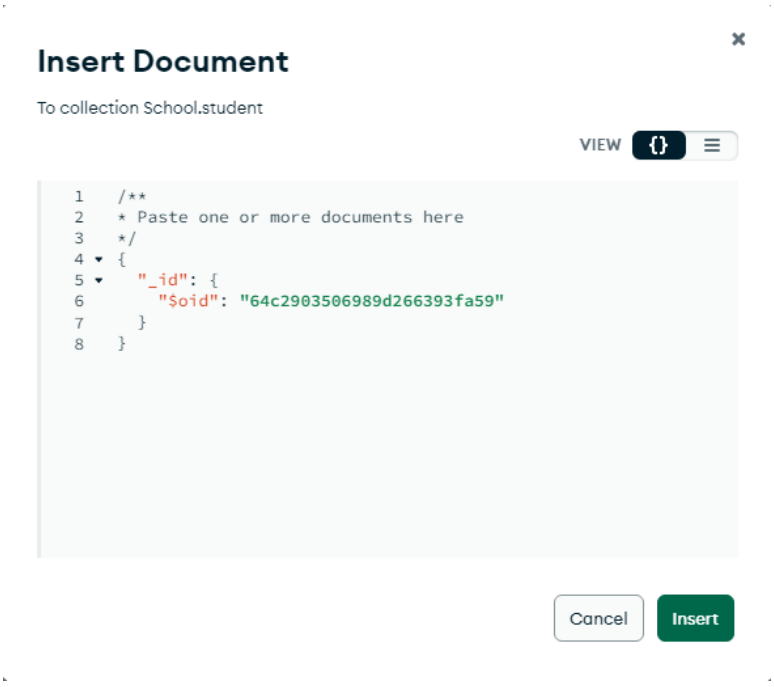
Click on the database and then select the collection where you want to add the data. We select **"School"** database and **"student"** Collection.

Then you click on the add data option. This gives you two options:

i. Either to import a JSON or CSV File or
ii. Insert a document Manually




Select the insert document from the options and you will see a prompt appearing on your screen

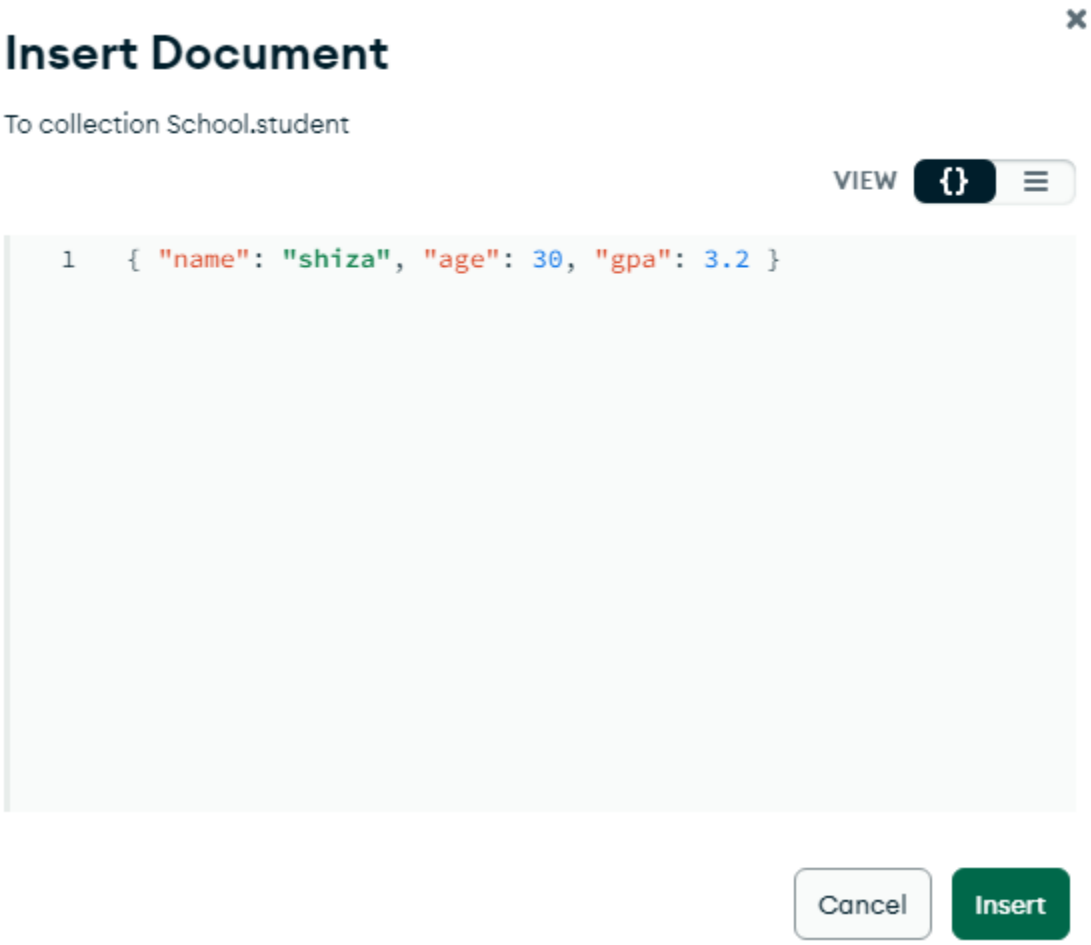


**To insert One Data at a time:**

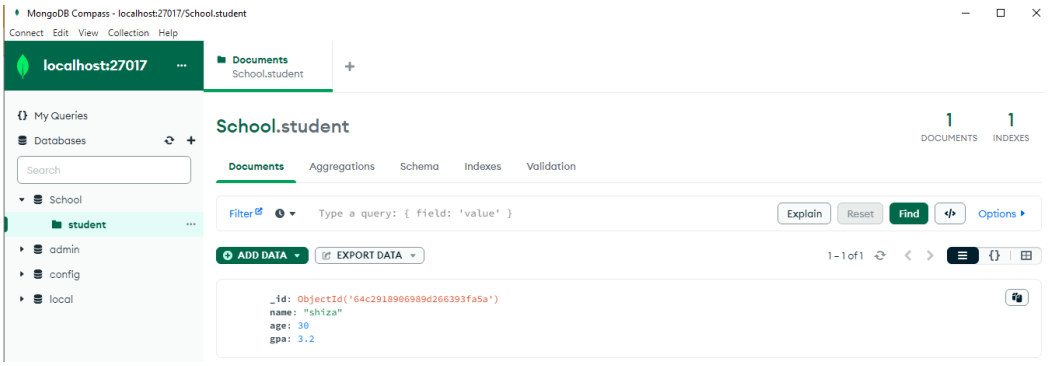
Delete the already existing text and write another query to insert one data

**{name: “shiza”, age: 30, gpa: 3.2}**

**Once the query is written, click on the format icon on the right of the prompt and your record will be inserted.**




**One record has been inserted**

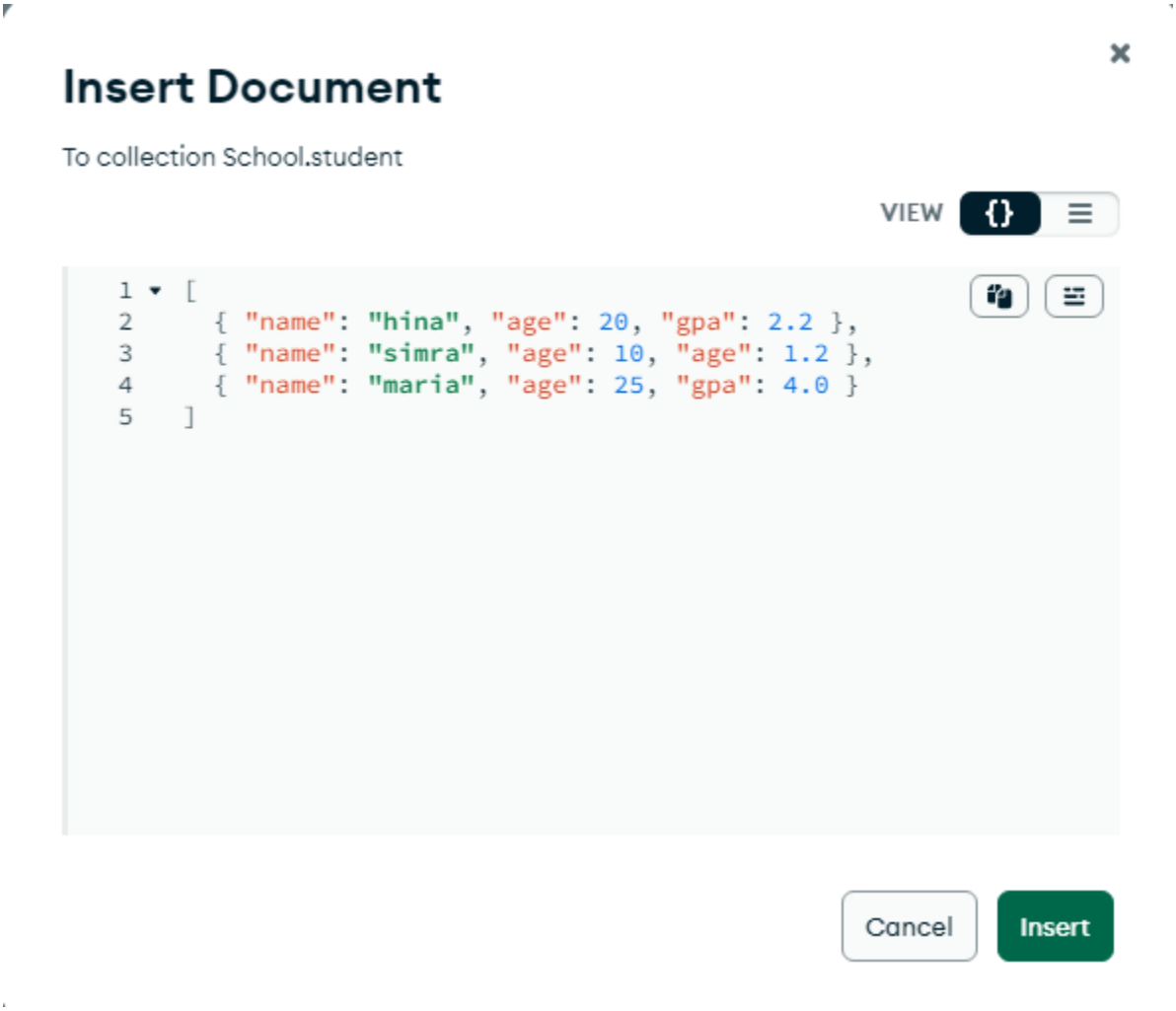


**To insert more than one documents:**

**In order to insert more than one document in MongoDB Compass, we enclose the documents in an array.**

**Click on the Add Data option and then insert document**

**[{name: “hina”, age: 20, gpa: 2.2}, {name: “simra”, age: 10, gpa: 1.4}, {name: “maria”, age: 25, gpa: 4.0}]**




**Nested Documents:**

In order to nest multiple documents, we use an array.

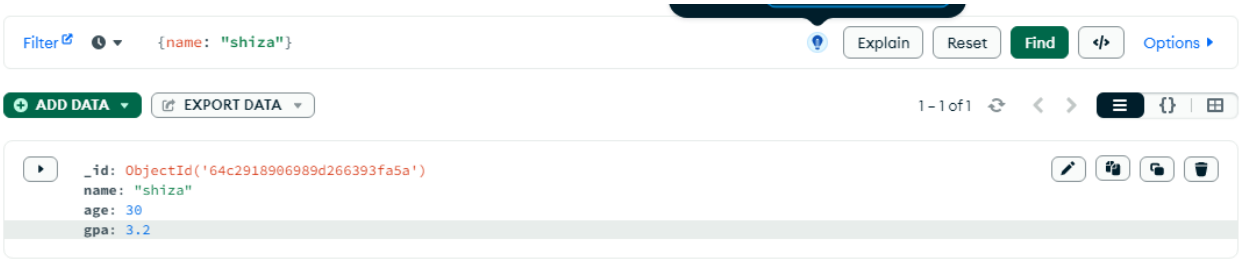
**[{name:"ayesha", age:32, gpa:2.8, fullTime: false, graduationdate: null, courses: ["datastructures", "oop", "calculus"], address: {street: "123 ned university", city: "karachi", zip:23456}}]**

```
_id: ObjectId('64c296f706989d266393fa60')
name: "ayesha"
age: 32
gpa: 2.8
fullTime: false
graduationdate: null
courses: Array (3)
  0: "datastructures"
  1: "oop"
  2: "calculus"
address: Object
  street: "neduniversity"
  city: "karachi"
  zip: 23456
```

**Finding a record/document:**

To find any document in MongoDB Compass, you can write the query in filter tab and select find option on the right. In this way you can find any specific record.

**{name: “shiza}**

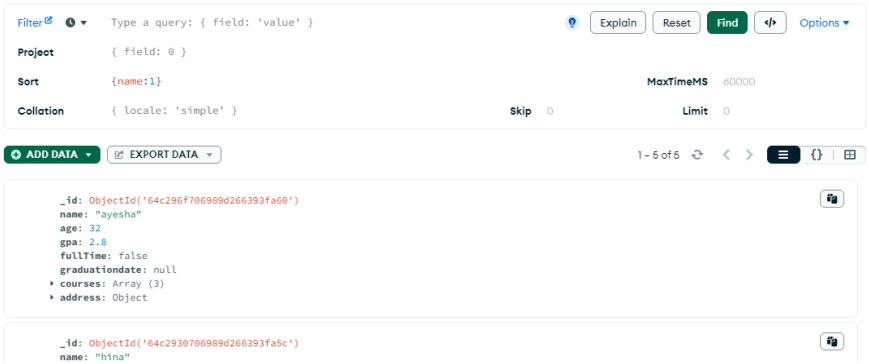


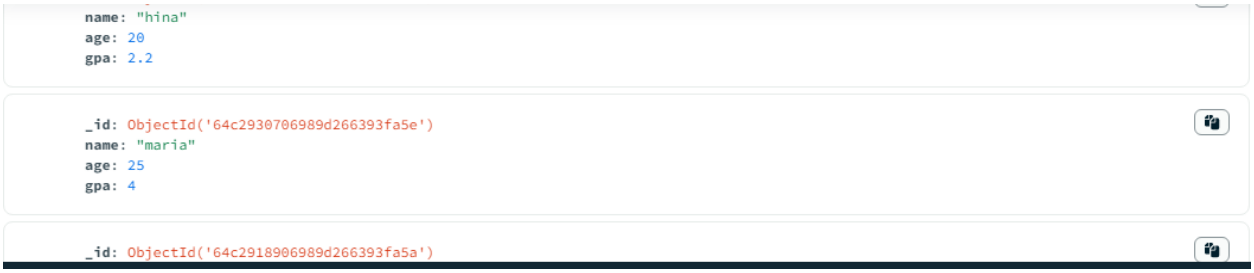

**Sorting data:**

To sort data, click on more options. You will find the Sort field. Type the required query and click on find.

**{name:1}**

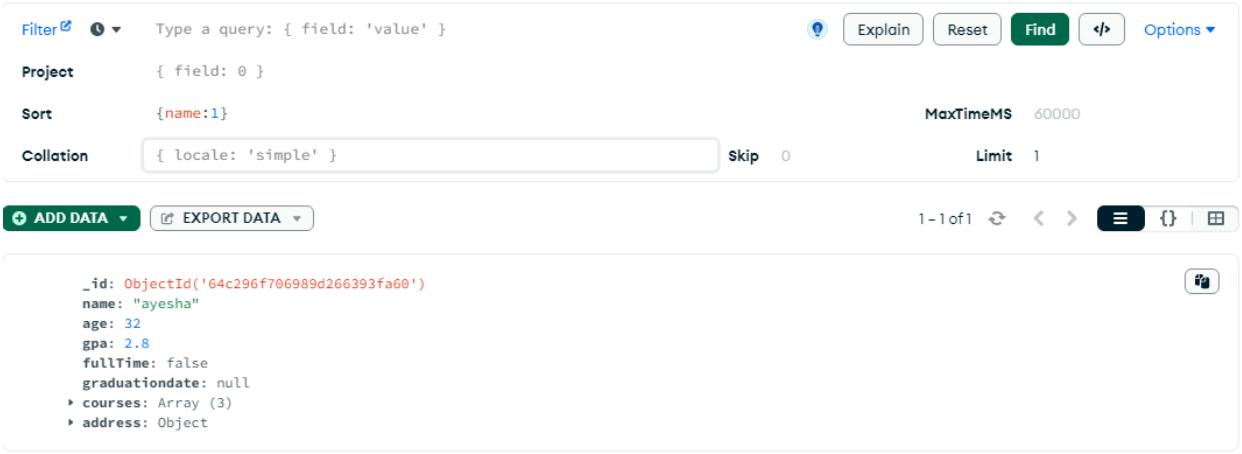
For Ascending Sort you will use 1 and for descending use -1.






**Limit the record:**

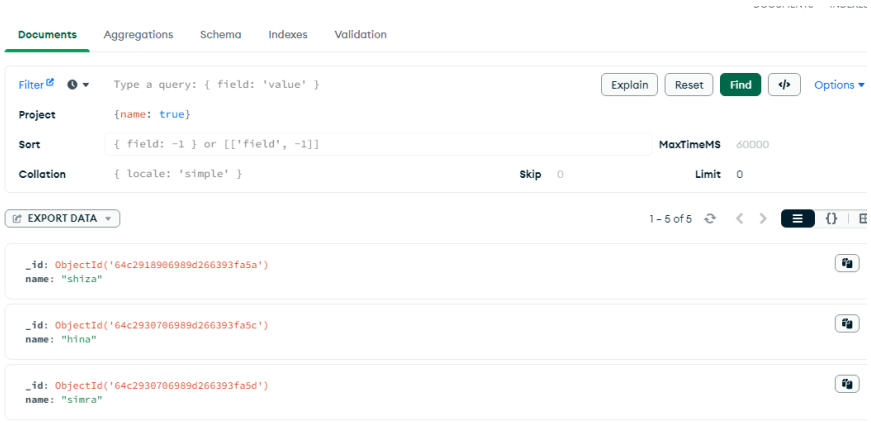
In order to limit the record, you can simple enter the number of entries to the limit box on the right to which you want the record to be limited.




**Optimizing the query:**

When using find, we are getting the entire data about a specific record. If we want to view only particular information about any record, we can use the projection field in MongoDB compass.

**{name: true}**



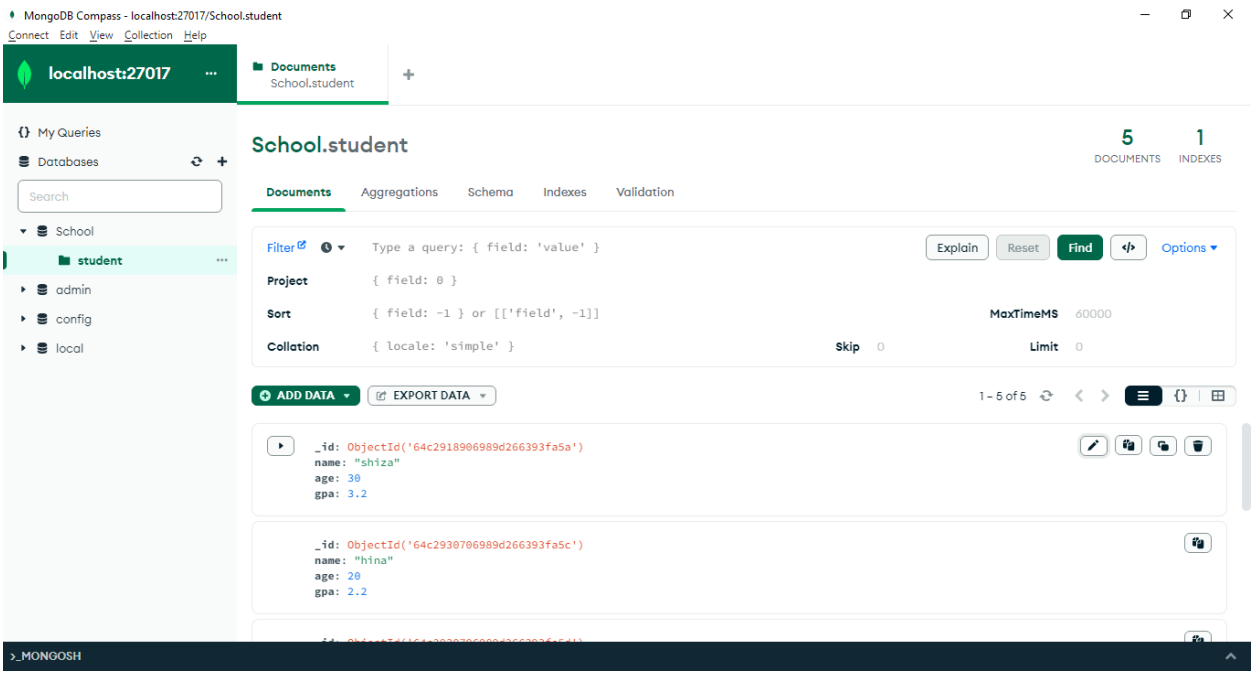
Here we are still able to view the Object ID as it is unique for all records. If we do not want to see the Object ID, then we can further optimize the query.

**{name: true, _id: false}**

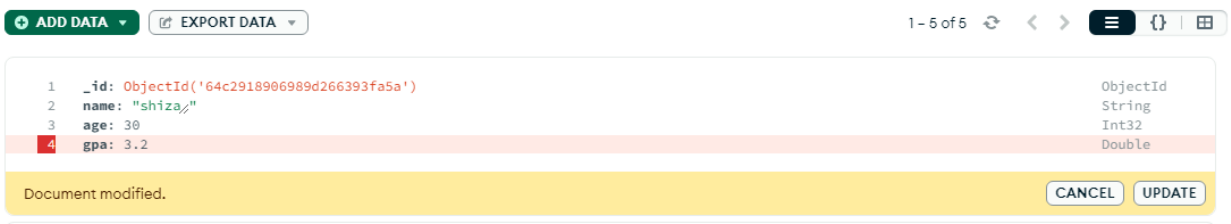
```
name: "shiza"
name: "hina"
name: "simra"
name: "maria"
```

## Updating a document:

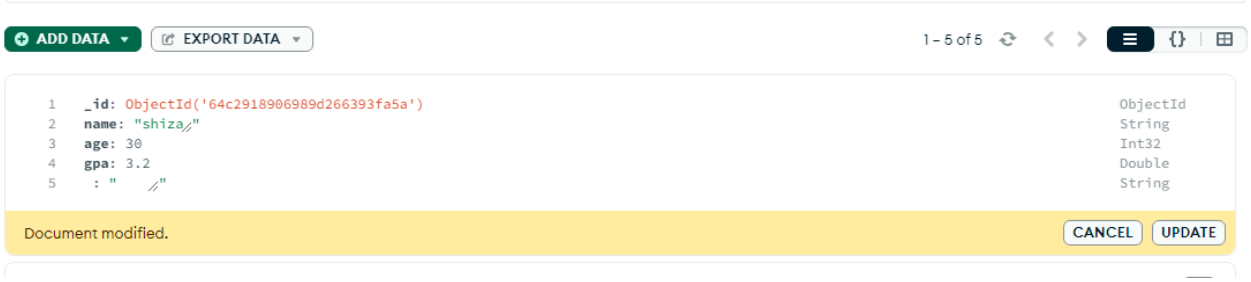
In MongoDb Compass, updating a document is very easy. We only have to click on the pencil icon to update any document.



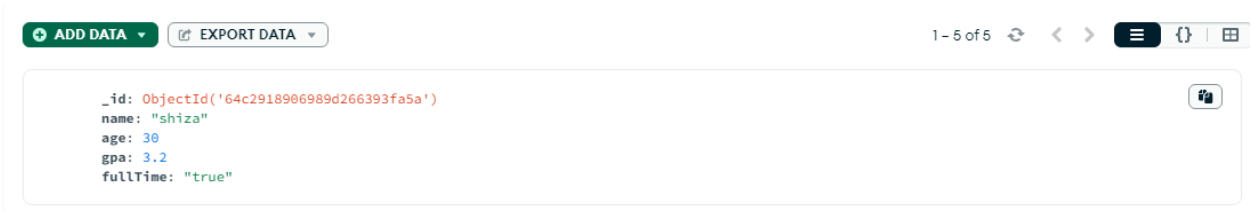

Once we click on the pencil icon to edit our document, we can delet any field using the trash can icon on the left of the field records and then click on the update option.



Similarly we can add a new field by clicking on the + icon besides the trash icon and then click on update. This will add a new data to our document.




The fullTime field has been added to the document with the name **“shiza”**



## LAB No 13.

**Objective: Comparison, Logical Operators and Indexes in NoSQL Databases**

**Theory:**

The insert, update, delete, find and limit etc are basic commands in NoSQL Databases. However, using logical operators, comparisons and indexes require some advance skills.

**Syntax of Commands**

**Comparison Operator:**

Comparison operators are used in programming and databases to compare two values and evaluate whether a particular condition is true or false.

***In Mongosh***

- **The Not Equal != Comparison operator:**

If we want to find any name in the database except for a specific one. For example, we want to find all names in the database except for **"shiza",** we use the Not Equal Comparison Operator.

**school>db.students.find ({ name: {$ne:"shiza"}})**

```
School> db.student.find({name: {$ne: "shiza"}})
[
  {
    _id: ObjectId("64c2930706989d266393fa5c"),
    name: 'hina',
    age: 20,
    gpa: 2.2
  },
  {
    _id: ObjectId("64c2930706989d266393fa5d"),
    name: 'simra',
    age: 1.2
  },
  {
    _id: ObjectId("64c2930706989d266393fa5e"),
    name: 'maria',
    age: 25,
    gpa: 4
  },
  {
    _id: ObjectId("64c296f706989d266393fa60"),
    name: 'ayesha',
    age: 32,
    gpa: 2.8,
    fullTime: false,
    graduationdate: null,
    courses: [ 'datastructures', 'oop', 'calculus' ],
    address: { street: 'neduniversity', city: 'karachi', zip: 23456 }
  }
```

- **Less than operator:**

To find out age less than any specified number, we can use the less than operator.

**school>db.students.find ({ age: {$lt: 20}})**

```
School> db.student.find({age: {$lt:20}})
[
  {
    _id: ObjectId("64c2930706989d266393fa5d"),
    name: 'simra',
    age: 1.2
  }
]
School>
```

- **Less than equal to Operator:**

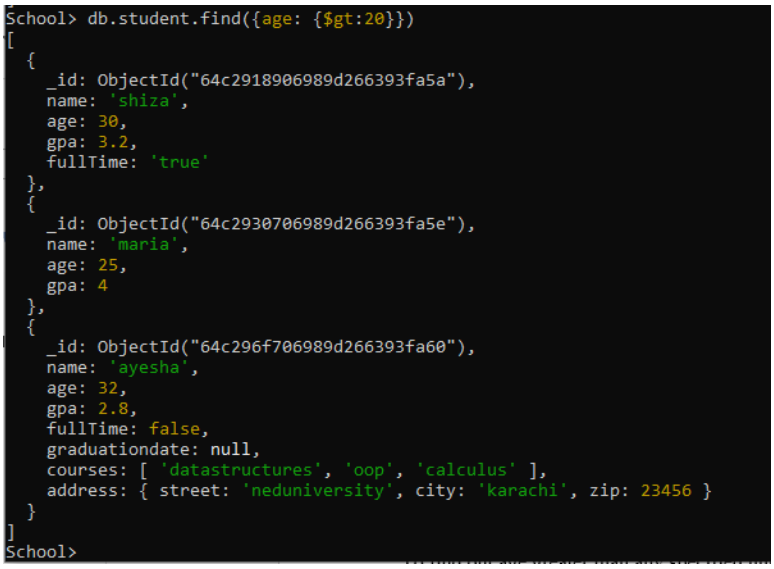
To find out age  less than or equal to any specified number, we can use the less than operator.

**school>db.students.find ({ age: {$lte: 20}})**


- **Greater than operator:**

To find out age greater than any specified number, we can use the greater than operator.

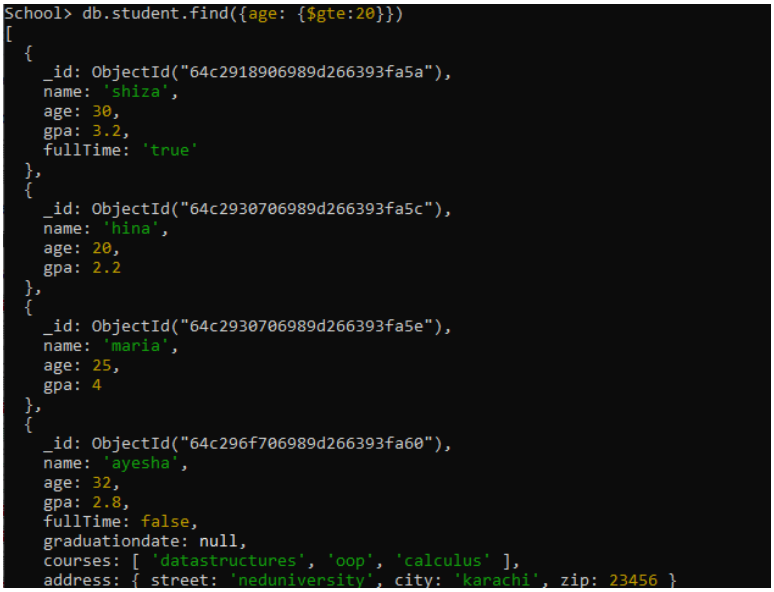
**school>db.students.find ({ age: {$gt: 20}})**




- **Less than equal to Operator:**

To find out age greater than or equal to any specified number, we can use the greater than equal to operator.

**school>db.students.find ({ age: {$gte: 20}})**




- **In Between Operator:**

To find the values that are in between a range

**school>db.students.find ({ gpa: {$gte: 3, $lte:4}})**


- **In operator:**

To find required value in a set of values. We use an array in this operator.

**school>db.students.find ({ name: {$in: [“shiza”, “hina”, “aamna”]}})**

```
School> db.student.find({name: {$in: ["shiza", "hina", "aamna"]}})
[
  {
    _id: ObjectId("64c2918906989d266393fa5a"),
    name: 'shiza',
    age: 30,
    gpa: 3.2,
    fullTime: 'true'
  },
  {
    _id: ObjectId("64c2930706989d266393fa5c"),
    name: 'hina',
    age: 20,
    gpa: 2.2
  }
]
School>
```

- **Not In operator:**

To disregard or overlook a required value in a set of values. We use an array in this operator.

**school>db.students.find ({ name: {$nin: [“shiza”, “hina”, “aamna”]}})**

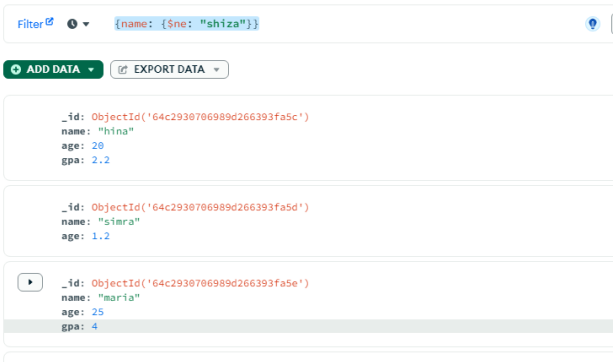
```
School> db.student.find({name: {$nin: ["shiza", "hina", "aamna"]}})
[
  {
    _id: ObjectId("64c2930706989d266393fa5d"),
    name: 'simra',
    age: 1.2
  },
  {
    _id: ObjectId("64c2930706989d266393fa5e"),
    name: 'maria',
    age: 25,
    gpa: 4
  },
  {
    _id: ObjectId("64c296f706989d266393fa60"),
    name: 'ayesha',
    age: 32,
    gpa: 2.8,
    fullTime: false,
    graduationdate: null,
    courses: [ 'datastructures', 'oop', 'calculus' ],
    address: { street: 'neduniversity', city: 'karachi', zip: 23456 }
  }
]
School>
```

***<u>In MongoDB Compass</u>***

- **The Not Equal != Comparison operator:**

If we want to find any name in the database except for a specific one. For example, we want to find all names in the database except for **“shiza”,** we use the Not Equal Comparison Operator. We write the following query in the filter bar and press the Find option.

**{ name: {$ne:”shiza”}}**

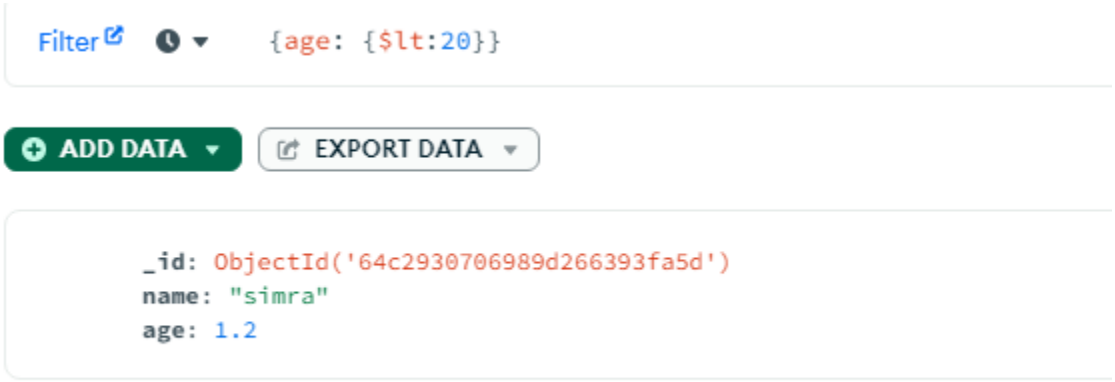


It will show all records except for those that contain “shiza” in the name field.

- **Less than operator:**

To find out age less than any specified number, we can use the less than operator. We write the following query in the filter bar and press the Find option.

**{ age: {$lt: 20}}**

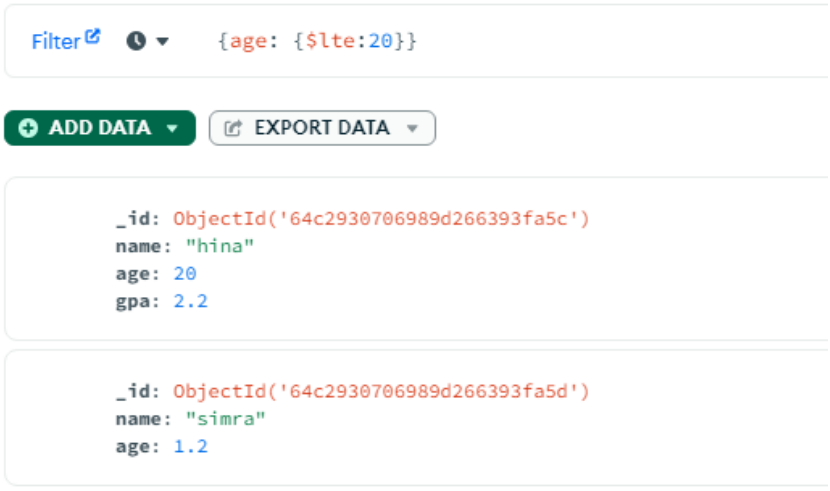



This gives the records that have age less than 20.

- **Less than equal to Operator:**

To find out age less than or equal to any specified number, we can use the less than operator. We write the following query in the filter bar and press the Find option.

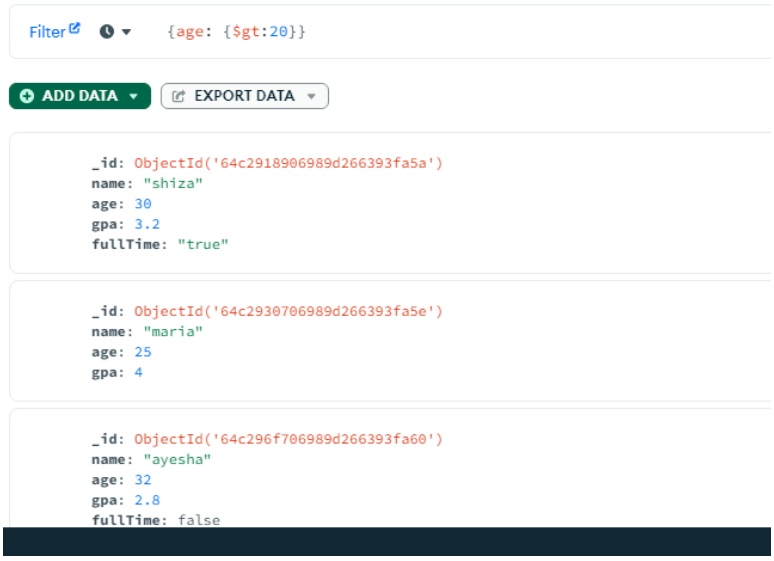
**{age: {$lte: 20}}**



- **Greater than operator:**

To find out age greater than any specified number, we can use the greater than operator. We write the following query in the filter bar and press the Find option.

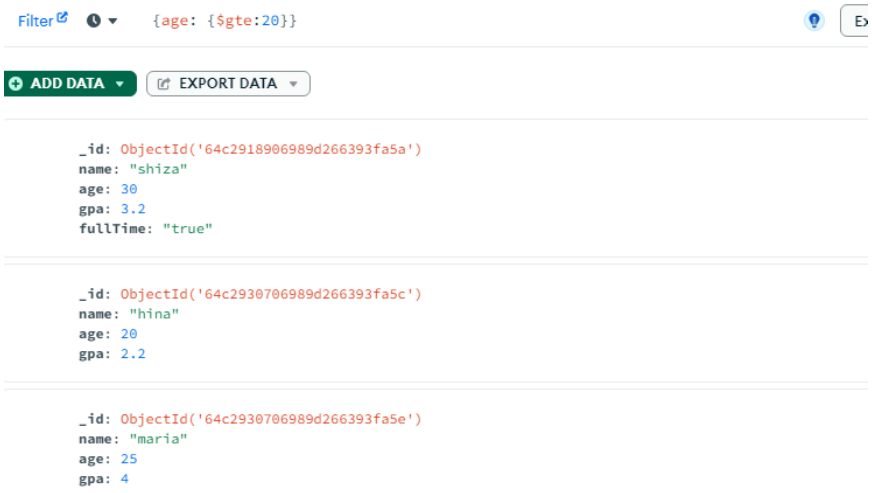
**{ age: {$gt: 20}}**




- **Less than equal to Operator:**

To find out age greater than or equal to any specified number, we can use the greater than equal to operator. We write the following query in the filter bar and press the Find option.

**{ age: {$gte: 20}}**

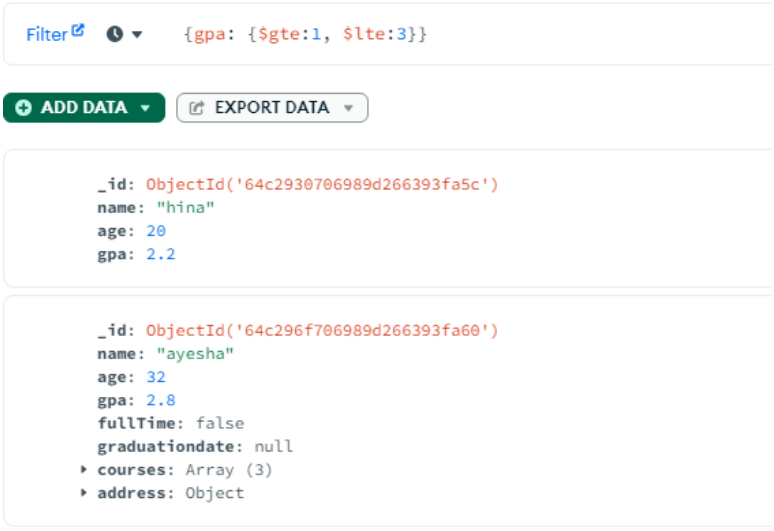



Here the record of hina I added after we used the greater than or equal to operator.

- **In Between Operator:**

To find the values that are in between a range. We write the following query in the filter bar and press the Find option.

**{ gpa: {$gte: 1, $lte:3}}**




- **In operator:**

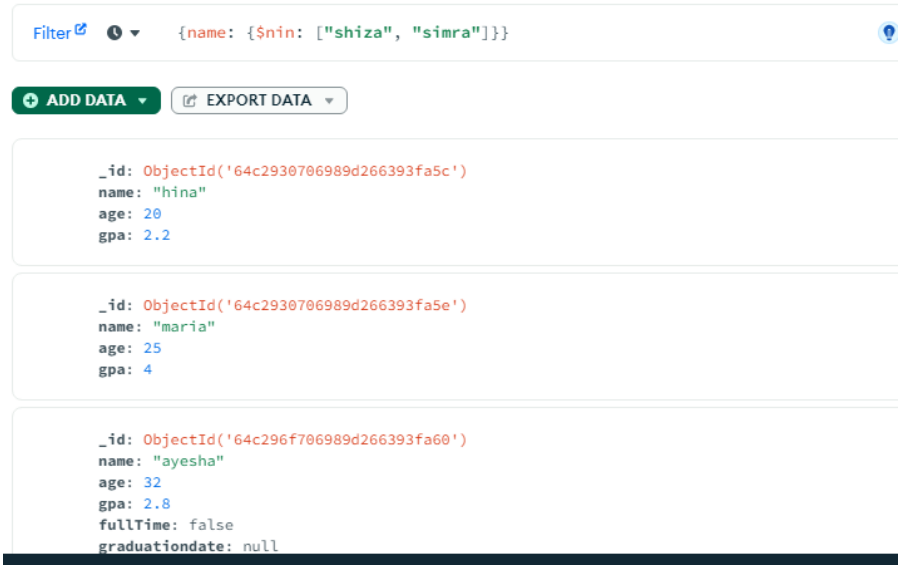
To find required value in a set of values. We use an array in this operator. We write the following query in the filter bar and press the Find option.

**{name: {$in: [“shiza”, “simra”]}})**

- **Not In operator:**

To disregard or overlook a required value in a set of values. We use an array in this operator. following query in the filter bar and press the Find option.

**{ name: {$nin: [“shiza”, “hina”, “aamna”]}}**




**Logical Operators:**

Logical operators are used in programming and databases to combine multiple conditions or expressions and evaluate them to determine whether an overall condition is true or false.

***<u>In Mongosh</u>***

- **AND operator**

Produces the output to be true only if all the conditions are true.

**school>db.student.find({$and: [{name: “simra”}, {age:{$lte:22}}]})**

```
School> db.student.find({$and: [{name: "simra"}, {age: {$lte:22}}]})
[
  {
    _id: ObjectId("64c2930706989d266393fa5d"),
    name: 'simra',
    age: 1.2
  }
]
```

- **OR operator**

Produces the output to be true if any of the condition is true.

**school>db.student.find({$or: [{name: true}, {age:{$lte:22}}]})**

```
School> db.student.find({$or: [{name: true}, {age: {$lte:22}}]})
[
  {
    _id: ObjectId("64c2930706989d266393fa5c"),
    name: 'hina',
    age: 20,
    gpa: 2.2
  },
  {
    _id: ObjectId("64c2930706989d266393fa5d"),
    name: 'simra',
    age: 1.2
  }
]
School>
```

- **NOR operator**

Produces the output to be true if all of the conditions are false

**school>db.student.find({$nor: [{name:true}, {age:{$lte:22}}]})**

```
School> db.student.find({$nor: [{name: true}, {age: {$lte:22}}]})
[
  {
    _id: ObjectId("64c2918906989d266393fa5a"),
    name: 'shiza',
    age: 30,
    gpa: 3.2,
    fullTime: 'true'
  },
  {
    _id: ObjectId("64c2930706989d266393fa5e"),
    name: 'maria',
    age: 25,
    gpa: 4
  },
  {
    _id: ObjectId("64c296f706989d266393fa60"),
    name: 'ayesha',
    age: 32,
    gpa: 2.8,
    fullTime: false,
    graduationdate: null,
    courses: [ 'datastructures', 'oop', 'calculus' ],
    address: { street: 'neduniversity', city: 'karachi', zip: 23456 }
  }
]
School>
```

- **NOT operator**

Produces the output if the condition is false

**school>db.student.find({age: {$not: {$gte:30}}})**

This query will return all the values where age is not greater than or equal to 30

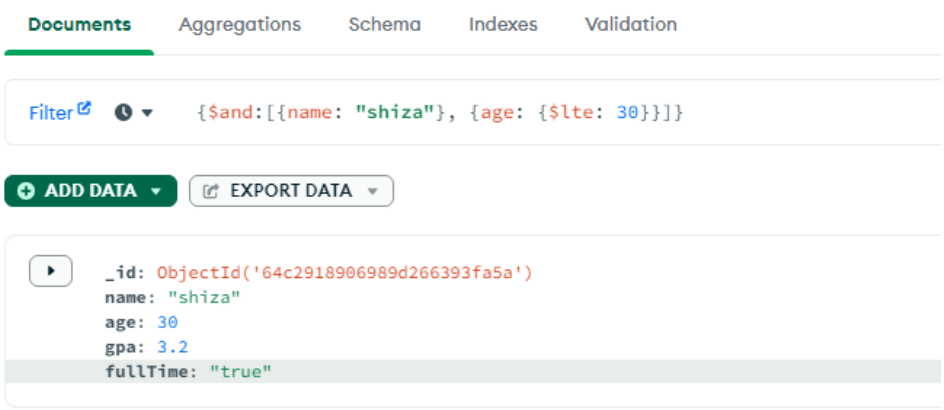
```
School> db.student.find({age: {$not: {$gte:30}}})
[
  {
    _id: ObjectId("64c2930706989d266393fa5c"),
    name: 'hina',
    age: 20,
    gpa: 2.2
  },
  {
    _id: ObjectId("64c2930706989d266393fa5d"),
    name: 'simra',
    age: 1.2
  },
  {
    _id: ObjectId("64c2930706989d266393fa5e"),
    name: 'maria',
    age: 25,
    gpa: 4
  }
]
School>
```

***<u>In MongoDB Compass</u>***

- **AND operator**

Produces the output to be true only if all the conditions are true.

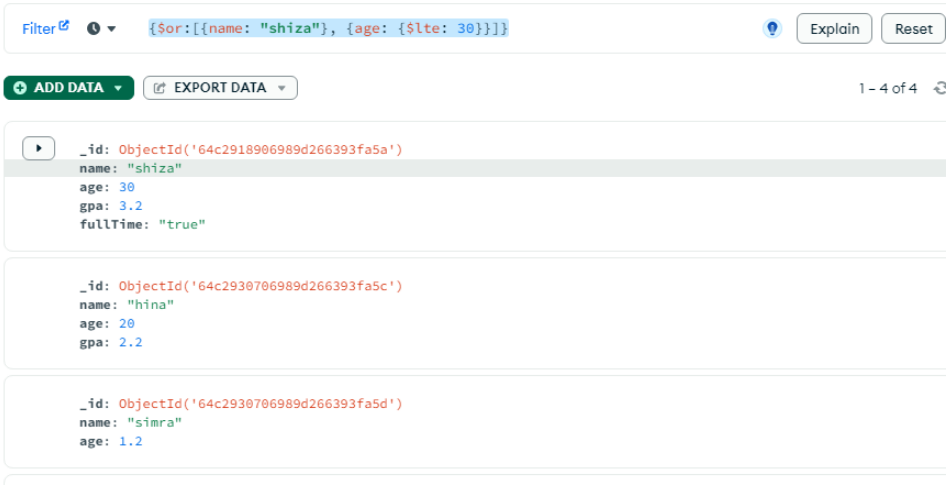
**{$and: [{name: "shiza"}, {age: {$lte: 30}}]}**



- **OR operator**

Produces the output to be true if any of the condition is true.

**{$or:[{name: "shiza"}, {age: {$lte: 30}}]}**




- **NOR operator**

Produces the output to be true if all of the conditions are false

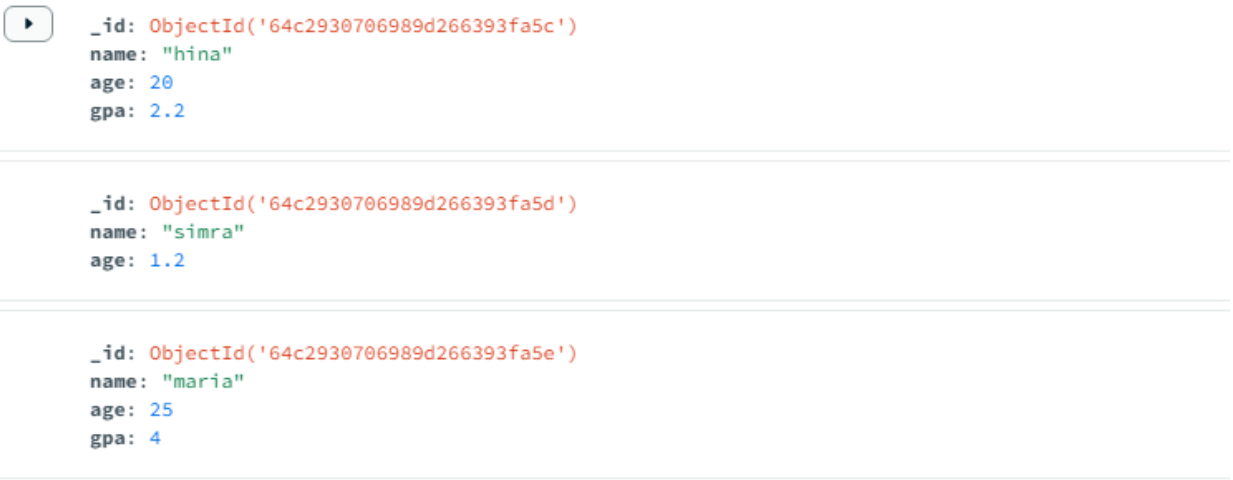
**{$nor:[{name: "shiza"}, {age: {$lte: 30}}]}**

- **NOT operator**

Produces the output if the condition is false

**{age: {$not: {$gte:30}}}**

This query will return all the values where age is not greater than or equal to 30.




**Indexes:**

In MongoDB, indexes are data structures that improve the efficiency and speed of query operations on a collection. They work similarly to indexes in traditional databases, helping to optimize queries by reducing the number of documents that need to be scanned to find matching data. Indexes in MongoDB are stored in a B-tree data structure, which allows for fast access to the data.

When you create an index on a field or a set of fields in a collection, MongoDB creates a separate index that stores the values of those fields along with pointers to the corresponding documents in the collection. This makes it faster to search, sort, and filter the data based on the indexed fields.

**school>db.student.find({name: “shiza”}). explain(“executionStats”)**

If we run this query it will go through all the data in a linear search and then find out the answer. Right now we have limited data but with humongous data, this task becomes arduous. Thus indexes make such tasks easier for us.

**In Mongosh:**

**Creating Index:**

In order to create an index, we use the following query.

**school>db. student.createIndex({name: 1})**

The 1 here is for ascending index application.

**To get all Indexes in the database:**

In order to get all indexes, we use the following query.

**school>db. student.getIndexes()**

**To drop an Index:**

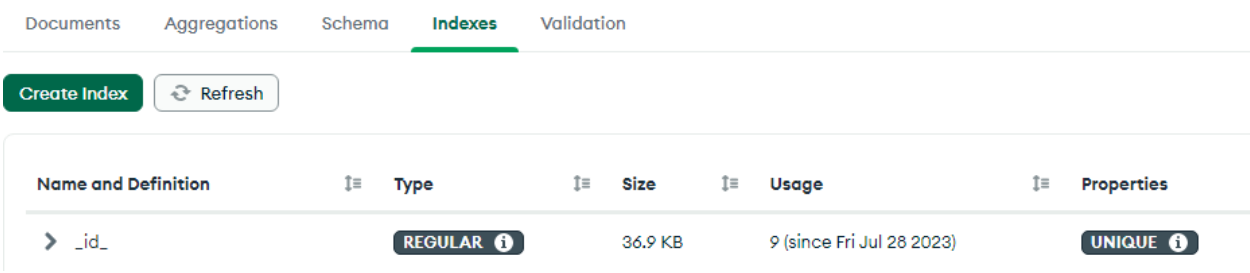
In order to drop an index, we use the following query.

**school>db. student.dropIndex("name_1")**

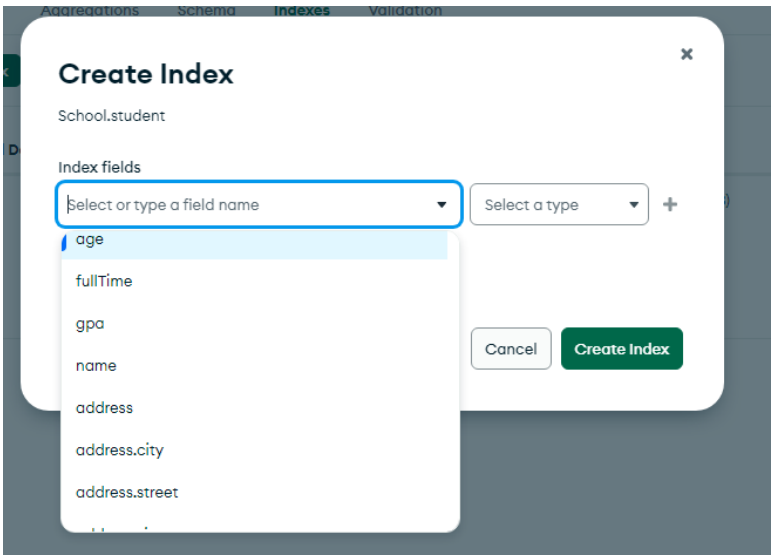
**In MongoDB Compass:**

**Creating an Index:**

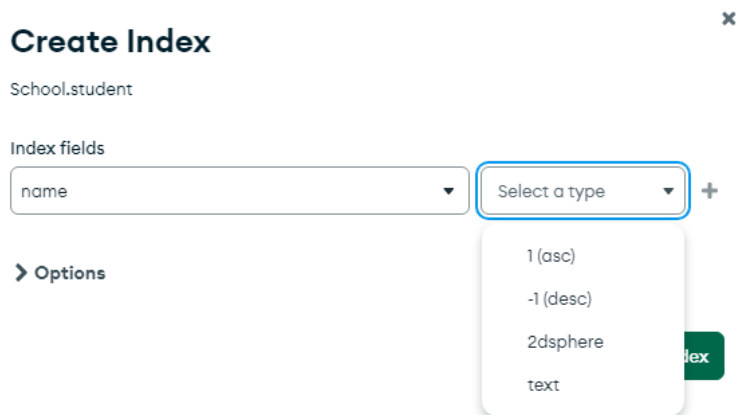
In order to create an index, we go to the indexes tab in MongoDB compass.




Then we click onto Create Index tab and select the field we want to create index for.



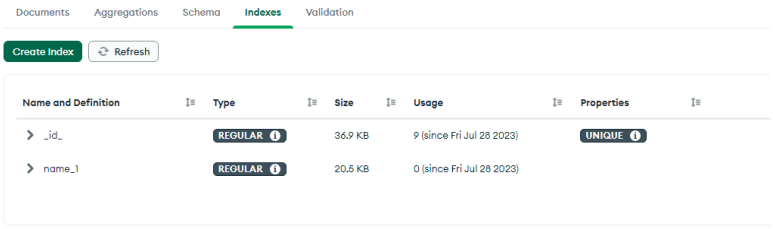
Then we select the type, where we tell the the Compass to create index with name field in which format( ascending, descending etc.)




Once the type is chosen, click on the create index button and your index will be created.

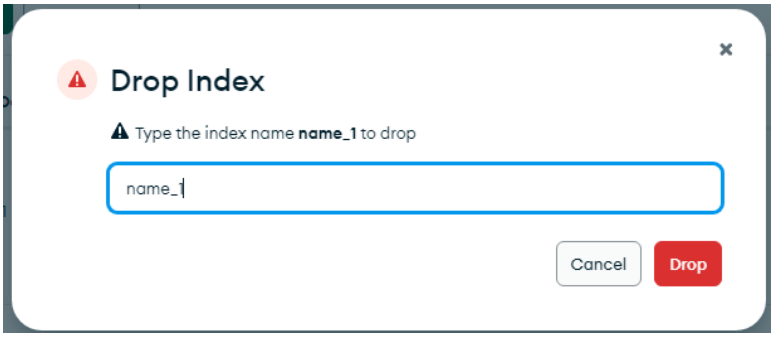
**To get all Indexes in the database:**

All the indexes created are displayed in the indexes tab.




**To drop an Index:**

In order to drop an index, we simple click on the trash icon located on the left of the index. Once clicked, a prompt appears which asks for the name of the index to be dropped. After entering the name, click on the Drop button and the index will be deleted.



## LAB No 14.

**Objective: Open Ended Lab**

**PROBLEM STATEMENT:**
Design and Implement a Database Management System for a Real-World Application.

**PROBLEM DESCRIPTION**:
In this laboratory assignment, you are tasked with designing and implementing a Database Management System (DBMS) for a real-world application of your choice. The goal is to demonstrate your understanding of database design principles, schema normalization, query optimization, and practical implementation of a DBMS.

**INSTRUCTIONS:**

1. Select a real-world application that requires a database system. Examples may include an online e-commerce platform, a library management system, a hospital patient information system, or a social networking platform. Ensure that the chosen application is sufficiently complex to showcase your skills in database design and implementation.
2. Design the database schema for the selected application. Start by identifying the entities, attributes, and relationships involved. Use appropriate entity-relationship (ER) modeling techniques to represent the database schema.
3. Normalize the database schema to ensure that it is in an optimal form, minimizing data redundancy and ensuring data integrity. Apply normalization techniques such as First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF) as applicable.
4. Implement the database schema using a relational DBMS. Create the necessary tables, define appropriate data types and constraints, and establish the relationships between tables.
5. Populate the database with representative data to simulate real-world scenarios. Ensure that the data reflects various aspects of the selected application, such as different user profiles, transactions, or entities.
6. Develop SQL queries that showcase the functionality of the DBMS. Include queries for data retrieval, data manipulation, and complex operations that highlight the capabilities of the DBMS and demonstrate your proficiency in SQL.
7. Optimize the query performance by analyzing query execution plans, creating appropriate indexes, and utilizing database optimization techniques such as query rewriting, de-normalization, or materialized views.
8. Test and validate the database system by executing the implemented queries and performing various operations on the database. Verify that the system functions correctly, handles concurrency, and maintains data consistency.
9. Document your database design, implementation decisions, and any challenges faced during the process. Provide a detailed explanation of the schema, relationships, constraints, and optimizations performed.
10. Present your database system, highlighting its features, performance, and usability.

**Rubrics for Evaluating Complex Engineering Activity**
**NED University of Engineering & Technology**
**Department of Software Engineering**

F/OBEM 01/18/00

**Course Code &Title: SE-204 Database Management Systems**
**Assessment Rubric for OEL**

Batch: ____ Class: _____ Semester: _____

| Criterion | LevelofAttainment | | | | |
|---|---|---|---|---|---|
| | Below Average (0) | Average (2) | Good (3) | VeryGood (4) | Excellent (5) |
| **Understanding of the selected real-world application and its requirements** | Absolutely no understanding of the problem | Limited understanding of the problem | Adequate understanding of the problem | Thorough understanding of the problem | Comprehensive understanding of the problem |
| **Design of the database schema** | No schema design developed | Basic schema design with minimal attributes and relationships | Adequate schema design with most required attributes and | Structured schema design | Well-structured and comprehensive schema design |
| **Representation of real-world data** | No representation | Limited representation of real-world data | Moderate representation of some aspects of the real-world | Adequate representation of most aspects of the real-world | Comprehensive representation of various aspects of the real-world data |
| **Functionality and Complexity of SQL queries** | Limited or incorrect implementation of queries | Simple queries without complex operations | Adequate implementation of basic queries | Moderate complexity with joins, subqueries, or aggregate | Advanced complexity with complex joins, nested queries |
| **Optimization techniques** | No or incorrect implementation of optimization techniques | Partial implementation of optimization techniques | Adequate implementation of optimization techniques | High implementation of optimization techniques | Effective implementation of optimization techniques resulting in improved query |

| | | | | | |
|---|---|---|---|---|---|
| **Validation of System Functionality** | Lack of testing or validation for critical functionalities | Partial validation of system functionality | Moderate validation of system functionality | Reasonable validation of system functionality with demonstrated handling error scenarios. | Comprehensive testing or validation for critical functionalities, ensuring robustness and reliability of the system. |

Student's Name:_____RollNo.:_____

Total Score: _____

Instructor's Signature: _____